

INDEX

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.		DBMS intro	1-2	
2.		Database system & its types	3-5	
3.		File system VS DBMS	5-7	
4.		2 tier & 3 tier architecture	8-10	
5.		Schema	10-11	
6.		3 level of Abstrac ⁿ (or) 3 schema Arch.	11-13	
7.		Data Independence	13-15	
8.		Candidate key & Primary key	15-16	
9.		Primary key	16-18	
10.		Foreign key	18-19	
11.		Foreign key (Referential Integrity)	19-21	
12.		Ques ⁿ of Referential Integrity	22	
13.		Super key in DBMS	22-25	
14.		E-R Model	25-26	
15.		Types of attributes in ER Model	27-28	
16.		Degree of Relo ⁿ ship (Cardinality)	28-30	
17.		One to one Relo ⁿ ship	31-32	
18.		Many to many Relo ⁿ ship	32-33	
19.		Ques ⁿ practice	33-34	
20.		Normalization	34-39	
21.		First normal form (1NF)	39-40	
22.		Closure method	41-44	
23.		Functional Dependency	44-47	
24.		2nd Normal form (2NF)	47-50	
25.		3rd Normal form (3NF)	50-52	
26.		BCNF (Boyce Codd Normal Form)	52-54	
27.		Lossless & Lossy Join Decomposition	54-57	
28.		All normal forms with Real Life Examples	57-58	

(Live SOL)

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
29.		Minimal cover	38-60	
30.		Question on Normalization	61-64	
31.		Q1.- Find normal form of a Rel ⁿ	65-69	
32.		Normalization Questions	69-70	
33.		Ques ⁿ Explained on Normalisation	70-74	
34.		Cover & Equivalence of F.D.	74-76	
35.		Dependency preserving Decomposi ⁿ	76-79	
36.		" " " " Example	79-80	
37.		Joins & Its Types	81-82	
38.		Natural Join	82-84	
39.		Self Join	84-86	
40.		Equi-Join	86-87	
41.		Left Outer Join	88-89	
42.		Right Outer Join	89-90	
43.		Rel ⁿ al Algebra (R.A.)	90-91	
44.		projection in Relational Algebra	91-92	
45.		Selection in " "	92-93	
46.		Cross/Cartesian product in " " " "	93-94	
47.		Set Diff. in R.A.	94-95	
48.		Union opera ⁿ in R.A.	96-97	
49.		Division opera ⁿ " "	97-99	
50.		Tuple Calculus in DBMS	99-103	
51.		Intro to SQL	104-106	
52.		All types of SQL Commands	107-108	
53.		Create table in SQL with execution	108-109	
54.		Alter Command (DDL) in SQL	109-110	
55.		D/W Alter & update	111-	
56.		D/W Delete, Drop & Truncate	112-113	
57.		Constraints in SQL	113-115	
58.		SQL Queries & Sub-Queries (i) & (ii)	115-116	
59.		(iii) & (iv) part.	116-117	
60.		(v) part	117-118	
61.		(vi) part	118	
62.		(vii) part	120	

S. No.	Topic Name	Page No.
63.	Use of IN and Not IN	121-122
64.	Use of IN and Not IN in Subquery	122-123
65.	Exist & Not Exist Subqueries	123-124
66.	Aggregate Funcs in SQL	124-126
67.	Co-Related Subquery in SQL	126-127
68.	DB Joins, Nested Subquery & Co-related Subquery	127-129
69.	Find N th Highest Salary using SQL	129-132
70.	3 Imp Questions on SQL Basic Concepts	133-134
71.	PL-SQL	135
72.	Transac ⁿ Concurrency	136-137
73.	ACID properties of a Transac ⁿ	137-139
74.	Transaction States	139-141
75.	SCHEDULE (Serial Vs Parallel Schedules)	141-142
76.	Types of problems in Concurrency	142-144
77.	Read-Write Conflict (or) Unrepeatable Read problem	144-145
78.	Irrecoverable Vs Recoverable Schedule in Trans	146-
79.	Cascading Vs Cascadeless Schedule	147-149
80.	SERIALIZABILITY	150-152
81.	Conflict Equivalent Schedules	152-154

① Database Management System (DBMS) :-

→ Basic Introduction → 2 tier, 3 tier, (3 schema),
(3 level of abstraction)

Inf.
↓
(comes in Data Independence)

→ Various Data Models :-

Network, Hierarchical,
(Relational, ER), object oriented.

(DBMS) or (RDBMS) — Same.
↓
(Relational).

→ ER MODEL :-
(Conceptual (Entity-Relationship))

Attributes & its types,
Relationship — " — " Inf.

Inf.
→ Basics of keys :-

primary key & its characteristic.
Candidate key — " — "
Super key — " — "

2 Foreign key — " — "

→ Normalization →

closure method → to find candidate key,
functional dependencies

1st NF (normal form), 2NF,
3NF,
BCNF

→ Transaction Control & Concurrency →

ACID properties

R-W problem (Read-write)

W-R "

W-W "

Conflict Serializability

Recoverability

Concurrency → locks

2-PL, timestamp

(2 phase lock)

→ SQL Δ Relational algebra

SQL → Structured Query language
(It is a programming lang.)

D D L command (Data-Definition)

D M L (Data-Manipulation)

D C L

→ Constraint

→ Aggregate func.

→ Joins

→ Nested Query

→ In, Not in, any, all

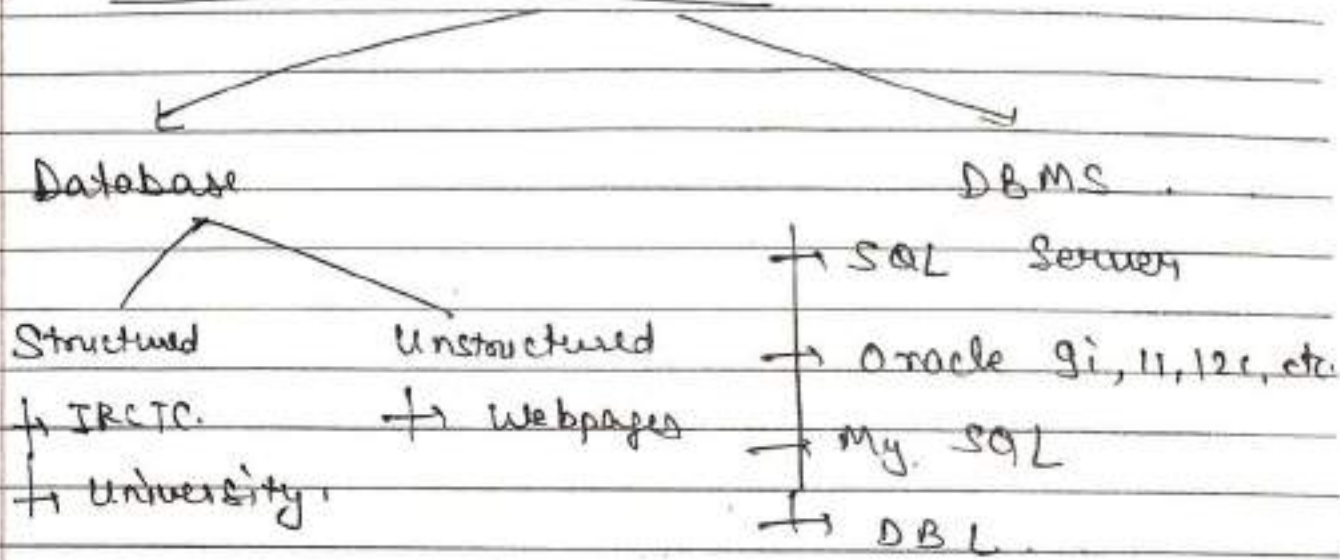
→ Indexing: → (Single level indexing)

→ primary, cluster & Secondary Indexing

→ B tree, B⁺ tree. (in Multi-level)

(2)

Database System ! →



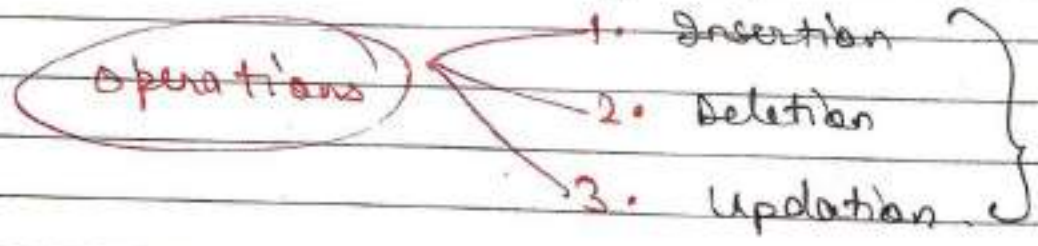
⇒ Database ! → "Collection" of Related Data.

Ex! Indian Railways & Indian passport have their different data.

⇒ Structured ! →

RDBMS → Relational Database Management Sys^m

⇒ DBMS ! → collection of operations



It provide easyness to perform operaⁿs.

⇒ Diff. companies have made diff. DBMS.

Ex!

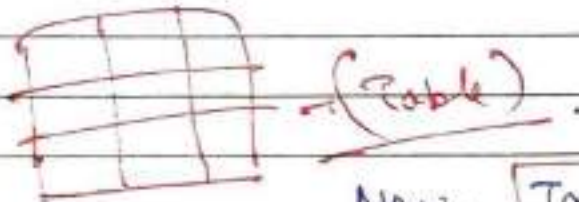
Microsoft Co. ⇒ SQL Server.
(Relational server).

⇒ Oracle ⇒ 9i, 11, 12c, etc.
My SQL.

⇒ IBM ⇒ DBL.

(H) Structured Data! → RDBMS
↳ Relation

Mean,
we stored in the table form.



Now, Table is technically called as Relo?

↳ "Relo" is most usable form.

Now

↳ Store Relo's & Access Relo's is done by the Management System i.e. RDBMS.

⇒ when it is used for Relations, it is called as RDBMS.

Hence,

→ we perform Insert, Delete & updation ops on Relo's.

(H) Unstructured Data! → There is not predefined structure. ↳! →

A webpage is a collecⁿ of photos, videos, chats, etc. i.e.

There is not a particular format in these.

→ Hence, RDBMS works only on the structured data.

[Note:] 90% of data on this earth is unstructured.

→ i.e., There is more technologies on unstructured data.

Ex: Bigdata, Hadoop, etc.

3

File System Vs DBMS :-

→ File System is used before DBMS.

→ user always manages its data in file form.
→ OS has inbuilt file system.

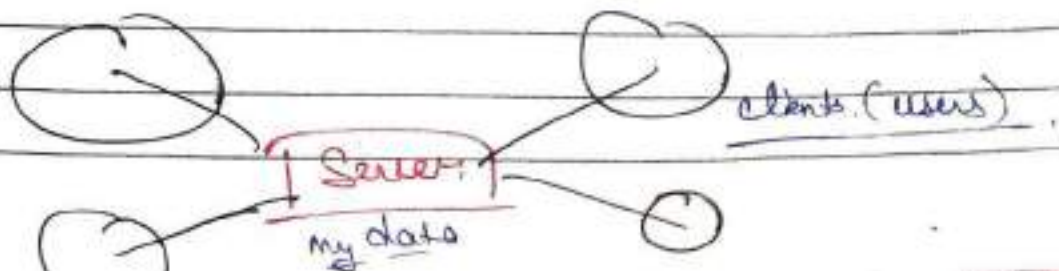
Ex: CIFS, NFS file systems.

→ File System :- We store our data in file form & then into our drives.

→ Why we use DBMS?

bcz, we are using the client-server architecture, means.

→ Our data is at a centralized place & all over the world users can access that.



→ Hence, we can't use file system here.
Now,
DBMS comes into picture.

1.) If we have to search only for 1 KB, then in file system, complete file comes to us (approx of 25 GB).

More memory usage. Now,
DBMS! → gives us only of 1KB data from the server.

(Searching is fast & Memory utilizeⁿ is efficient).

2.) In file system, we require attributes to search data, i.e.,
Attributes (name, locaⁿ, permission, etc.)

Meta Data! → Data about Data.
(Data of a particular file).

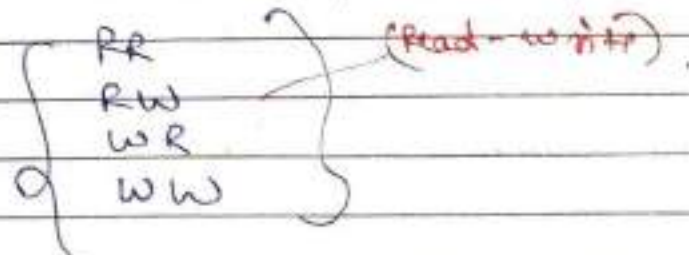
In DBMS, no locaⁿ, it is totally ~~comp~~ independent. We don't require any attributes here.
(Easeⁿess provide).

3.) Concurrency! → means Concurrent Access.
→ Multiple persons can access the data at the same time.

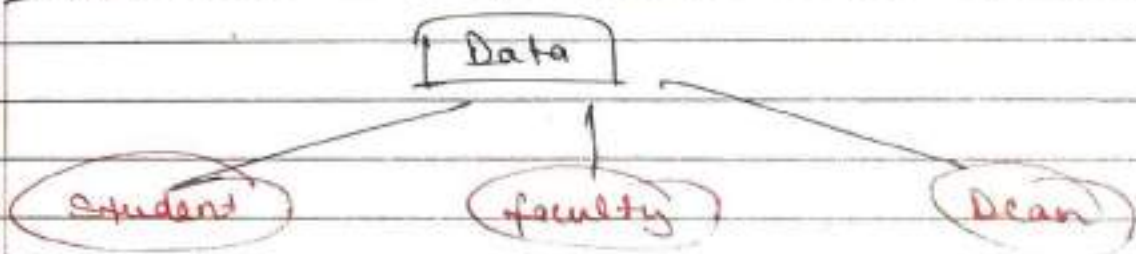
Ex! IRCTC (Indian Railway) System

File system may show inconsistency when multiple users want access of a file at the same time.

DBMS, have proper protocol for concurrency.



4.) Security! → Role based Security



(Role based Access Control.)

If we are user, we only get user data.
 " - faculty, " - faculty

(o.s.)
 File system, has no security for this.
 No level-by-level. No Hierarchical.
 DBMS has this role-based Security Syst.

5.) Data Redundancy! → (Duplication)

In File system, we can save same content by diff file names.

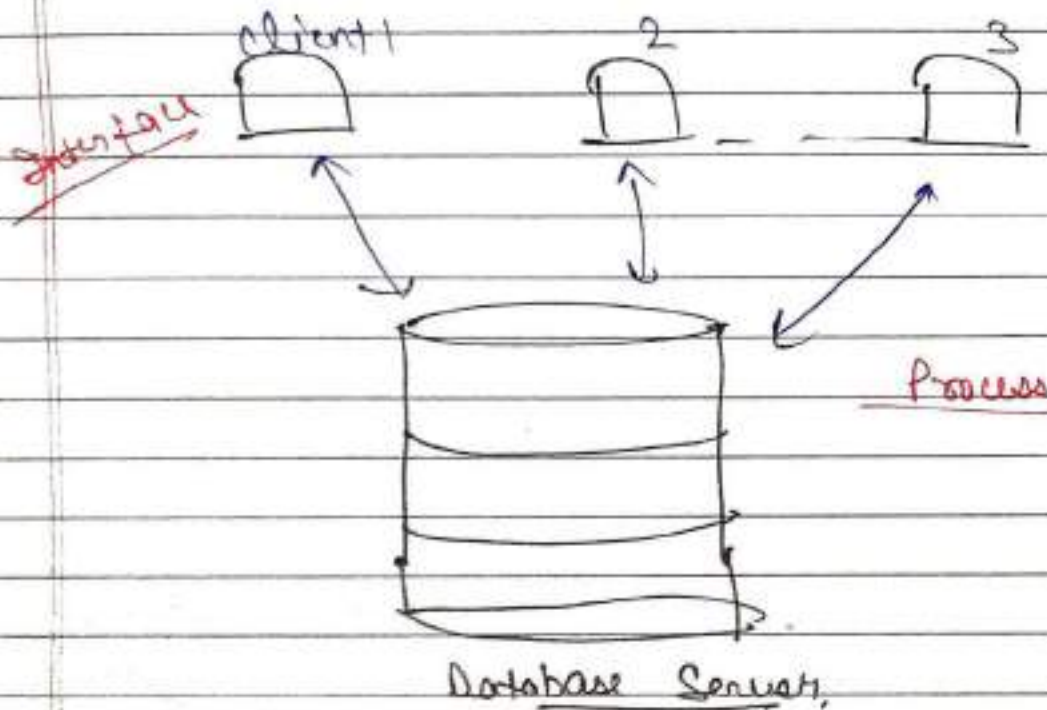
In DBMS, has many constraints to ensure unique data. Stops redundancy of data.

DBMS is at back end of every client server & web app.

④ 2 tier & 3 tier architecture in DBMS !

④ 2 tier means 2 layers.

1. client (machine) layer.
2. Database server, (Data layer).



→ 2 tier also called as client-server Architecture.

Ex: → ~~G)~~ Indian Railway :- If we reserve ticket by going to railway Staⁿ at ticket window.

cli: Bank :- When physically we draw or post some money.

(client → Request → Database server)
↓
Gives info

→ Hence, limited clients & limited Database to which we access, i.e., why maintenance is very easy.

But, the users are not limited. They are in big nos. Then, this 2 tier system fails. We call it Scalability.

Security : → not gud, bco clients are directly interact with the database.

Advantage : Maintenance is easy.

② 3-tier : → (Now, mostly uses).

- 1.) client layer
- 2.) Business layer
- 3.) Data layer.

→ In 2-tier, query is process in Database Server.
But,

→ here, Business layer supports interface.
Our query is process in Business layer
Hence,
we don't give load to Database Server here.

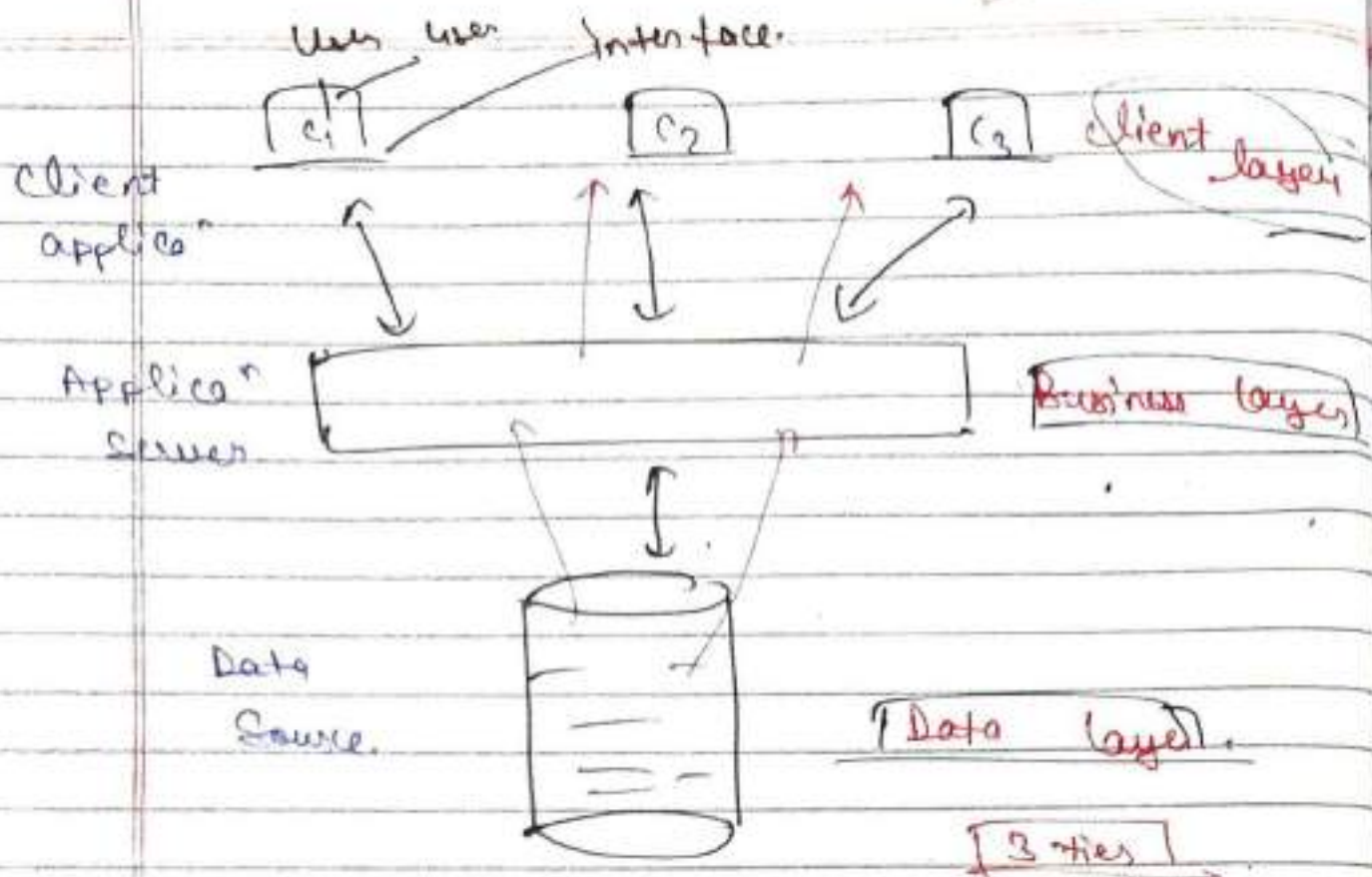
Hence, Business layer acts as Intermediate.

Ex: IRCTC & banking app & Gmail app

Advantage : 1.) Reliability

2.) Security

(no direct interact of data & user)



(Here, Maintenance is not Easy bcoz. it is Complex.)

Ex: Web Applications (Apps) are kind of 3-tier Architecture.

If we go physically to any bank or Railway Stn, then it is 2-tier Architecture.

(S)

Schema → Logical Representation of a database.

Ex: In RDBMS, data is stored in the form of Tables (Rel's).

RDBMS Access & Manage the data in this Schema (table) form. It is not actually stored in this table form in drives.

(Entity)

E-R

Ex: i) Schema of a Student →

Roll.No.	name	addresses
----------	------	-----------

✓ Relationship

ii) Course (Schema, Entity)

C. ID	name	Duration
-------	------	----------

✓ (Relationship)

↳ Logical Representation (Structure)

→ But, we implement it by SQL (Integer, character - -) (Sequel)

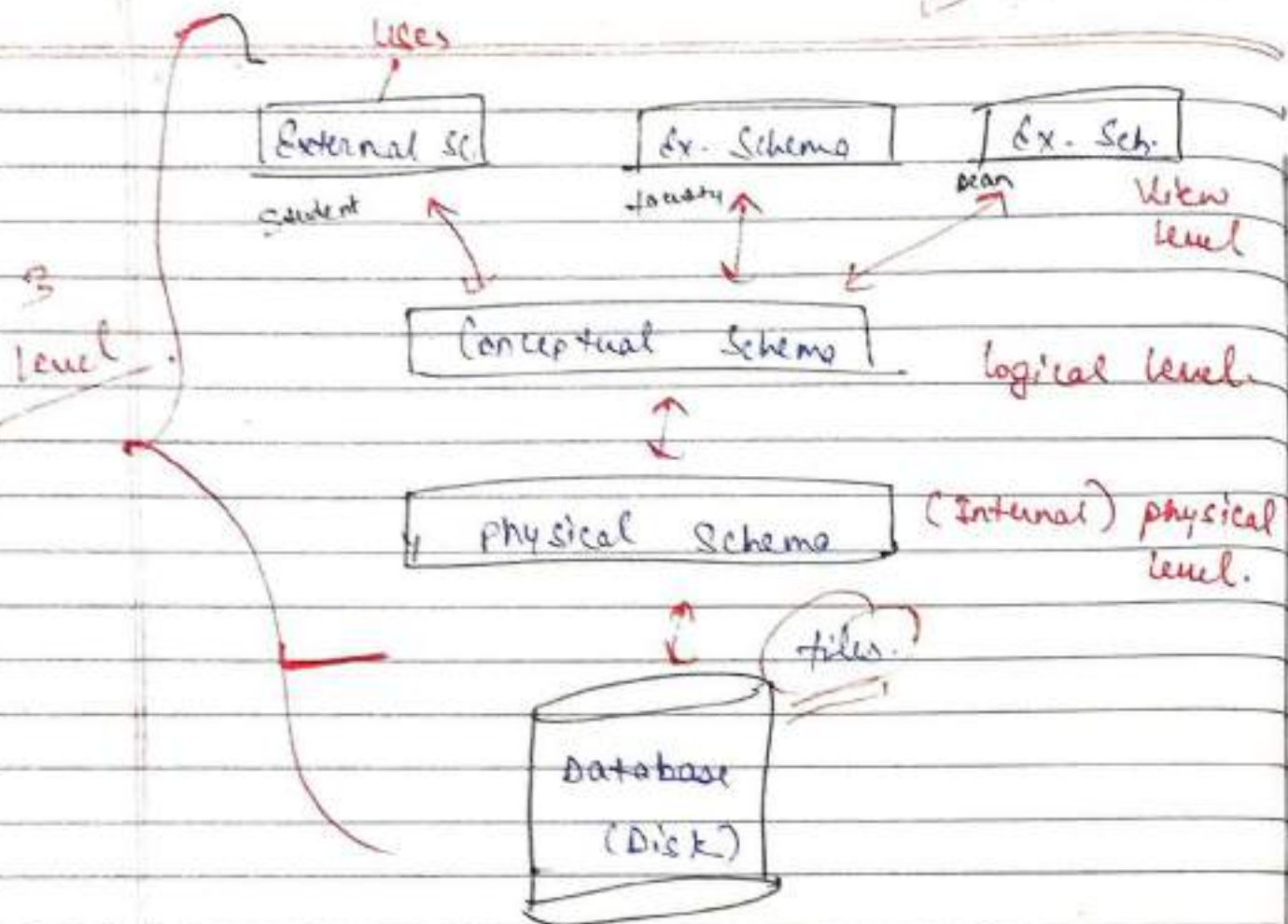
Data Defiⁿ language (DDL) → to implement Schema. (as to design)

Schema is simply a structure (table form)

(6) (Three level of Abstraction) (or) (Three Schema Archi.)

→ (we hide the locⁿ of where the data is stored, from the user)

Ex: our mails, we don't know physically (exact locⁿ) where our mails are stored. Ex- Delhi, UK, US, - etc.



External Sch! (View level) → View that will be given to the user.

{ Students have their View.
 Faculty have ——— }

Ex: → after login, which view comes to us is external schema.

Conceptual Sch! E-R Model

Ex: Student (Roll no, age, add, ---)

→ Inform of all tables that we use & their relationship. A type of Blueprint is this.

* physical Schema → where the data is actually physically present. The locaⁿ of data.

→ A Normal Data Base Designer is working at level of Conceptual Schema.

→ front End or Interface Developers are at level of Ext. Schema.

→ Database Administrator (who has all control of data) is at physical Schema.

Centralised — Data at one place.

Multiple — Data all over the world.

Imp. Note:

When we see the data as a user, then we see it in Table form. But, Actually in hard disk, data is stored as files. ~~And~~ and we apply the layer of DBMS on it.

There are 3 layers b/w the user & the data

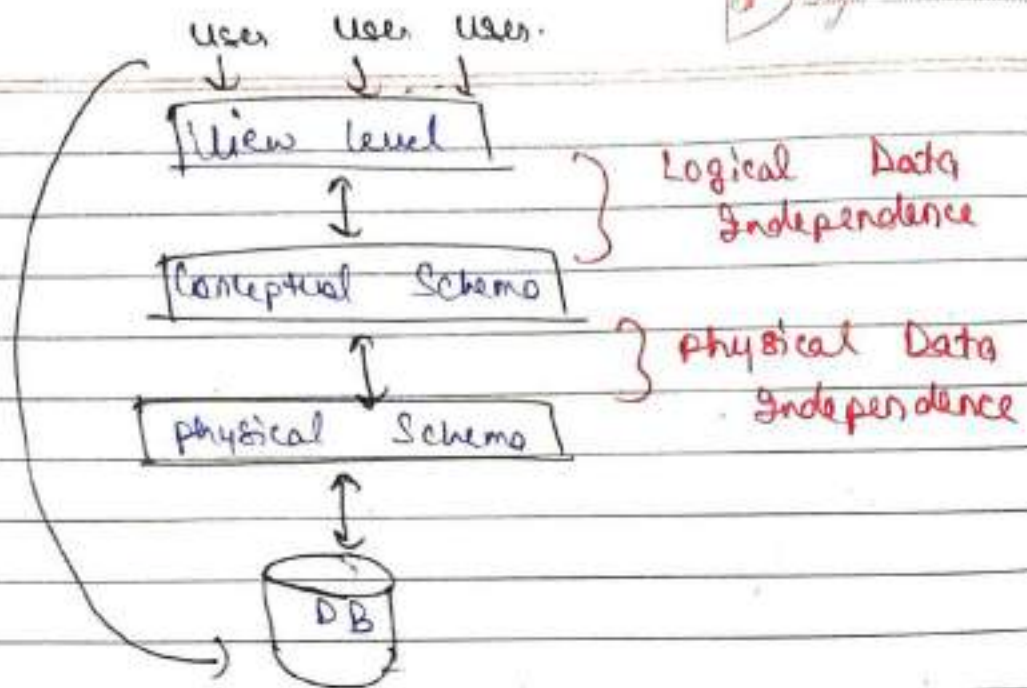
x ————— x

7

Data INDEPENDENCE : →

→ Make user independence of data. Hide its locaⁿ & etc.

→ Conceptual Schema : → which table we use, how many tables are there. how many attributes, relationship b/w them.



④ Logical Data Independence! →

Student Table → add by user 1.

Name	Age	Mob. No.

It will not affect the Application program.

→ we don't need to write the Appⁿ pgm again. If that user 1 changes want, we give him. But, It will not affect the actual logical structure.

mean, User 2 still can see only col 1 & col 2. Mob No. col^m is only for user 1.

→ We use this concept, by Views (Virtual Table).

→ In actual Table, may be we have 50 col's but user can only see 5-7 col's. So, user think there is only 5-7 col's. But, there are many.

→ Hence, View level don't change by users changes in Conceptual Schema.

Ex: UMS (University Management System), Shopping website → They can add or delete any col's.

Hence,

It is logical Data Independence.

physical Data Independence: → Schema Any change in physical data independence won't affect our Conceptual Schema.

Ex: If we take data from Hard Disk 1 to 2, then data not changes. Tables & structures remains the same.

Any change in Back End, won't affect the user. It is Data Independence.

§ Candidate key & Primary key! →

→ key! → It is one of the attribute in the table.

Use of key! To uniquely identify any 2 tuples in the table.

Roll.No	S.name	City	Age
1	Reddy	Shamli	20
2	Anurag	Kanpur	21
3	Reddy	Shamli	20

1) Same 1 student repeats
or
2) 2 diff. students with same attr.

4 To identify this, we must have a key.

Ex!.

Student Table

- 1.] Aadhar Card
- 2.] Roll No.
- 3.] Reg. No.
- 4.] Licence No.
- 5.] Voter Id.
- 6.] Phone No.
- 7.] Email - Id.

all
These attributes can
uniquely identify any 2
rows.

The set of all 1-7 values. If we make a set of all of them. Then we call it Candidate key.

<Aadhar Card, Roll No, Reg No, Licence No, Voter Id, Phone No, Email-Id>

Now, from above Candidate key set, we choose the one most appropriate & call it Primary key. & rest all keys known as Alternative keys.

9. PRIMARY key: →

(Every lock has its unique key.)
i.e.

we uniquely identify 2 things.

Ex!.

Any 2 student can have same name, age, D.O.B. but, some attributes like phone No., Aadhar Card, Licence No. are always unique.

2) Candidate keys! These are Unique.

phone no.
adhar card
Pan
Reg no.
Roll no.

Candidate keys.



phone no, adhar card, ugc no. को NULL नहीं रखा जा सकता। धरना ही पड़ेगा।

Case 1: We fill wrong. adhar card no.

Case 2: We don't take adms without adhar card no.

In student case, most appropriate key is
Reg no.
Roll no.

→ Primary key = { unique + NOT NULL }

We don't give primary key to them, they give to us.

Ex 1: university give us Reg. No.
passport office give us passport No.
license office give us license No.

- We ~~can~~ have only 1 primary key in a Database. Software don't allow us this.

(Why need 2 or more, when we only need 1)

10

Foreign key in DBMS: →

- Foreign ^(F.K.) key! → It is an attribute or set of attributes that references to primary key of ~~some~~ same table or another table (relation).

→ It maintains referential integrity.

Ex:-

Student →

Course →

P.K.
(primary key)

Roll no	name	Add.
1	A	Delhi
2	B	Chandigarh
3	A	Mumbai

(F.K.)

C.ID	C.name	Roll.No.
C ₁	DBMS	1
C ₂	networks	2

Roll.No is same b/w the student & course. It shows relationship b/w them.

→ In Table 2, Roll No. col^m value ^{take} references from Roll no attribute (primary key) in Table 1.

Q:- Can I write Roll.No. '10' in Foreign key (F.K.)

→ No, because it is not in Table 1 (Roll.No.) until now.

(with f.k.)

→ Table 2 is called Referencing Table.

→ Table 1 is called Referenced Table.
(with p.k.) or Base Table.

```
* Create table Course
(
    Course_id varchar(10),
    Course_name varchar(20),
    Rollno int references
    Student (Roll no);
);
```

→ Code Ex.

→ Now, how to write a query after the table is created.

```
→ Alter table Course
Add constraint f.k.
foreign key (Roll no.)
references Student (Roll no);
```

Note: (1) We don't have to keep the name 'Roll. No.' same of both the attributes necessarily. We can also take diff. names.

(2) In a table, there can be more than 1 foreign key.

11. Foreign key :- (Referential Integrity) :-

→ Integrity means same value for the database.

Ex 1 (12)

In mobile phones,
diff. price at Flipkart, Amazon & store
so, not integrity.

(2.) In university,
a student Reg. no. is same at every office
in the university, in library, in dean office.
so,
Integrity present here.

Ex 1 -

P.T.V

A.K.

Student (Base Table or Referenced Table)	Roll No.	name	add.	Deletion	C. Id	C. name	Roll. No.
	1	A	Delhi		C1	DBMS	1
	2	B	Mumbai		C2	networks	2
	3	A	Chd.		C3	C++	7
	4	C	Chd.				

Course (Referencing table)

(*) Referenced Table →

1.) Insert → No violation (we can easily add)

2.) Delete → May cause violation (bcoz, if we
~~delete~~ delete a row & with same roll.
no. a row is in referencing table. Then,
it's not possible).

Ref. Int.

1.) on delete cascade
(also delete from every table)

2.) on delete set Null
(insert NULL in other tables)

Roll. No.
NULL

f.k. can take reference from P.K. of same table.
also.



Date: _____
Page: _____

(21)

But, if same colⁿ is also a primary key in that other table. Then, we can't use this colⁿ, bcs.

primary key cannot be NULL.
<unique, not NULL>

3.) On delete No Action.

(In this colⁿ, our row don't get deleted, first we have to delete from other tables, then we can delete in our table.)

3.) updatation: → may cause violaⁿ (bcs, if we make '20' of '2', then how can we get Roll No '2' in referencing table).

Solⁿ! →

- 1.) On update Cascade
- 2.) On update Set NULL
- 3.) On update No Action.

⑧ Referencing table: →

1.) Insert: → May cause violaⁿ (bcs, if '1' is not base table, then how it is referencing table).

2.) Delete: → No violation

3.) updatation: → May cause violation (bcs, we can't make '20' of 2 if '20' is not in base table).

Note!

- 1.) Same key can be foreign & primary key in a table.
- 2.) Many table can have a foreign key from a single table by take reference of its P.K.

12.

Q.17

Let $R_1(a, b, c)$ and $R_2(x, y, z)$ be 2 relations in which 'a' is foreign key in R_1 that refers to primary key of R_2 . Consider, 4 options $\rightarrow$

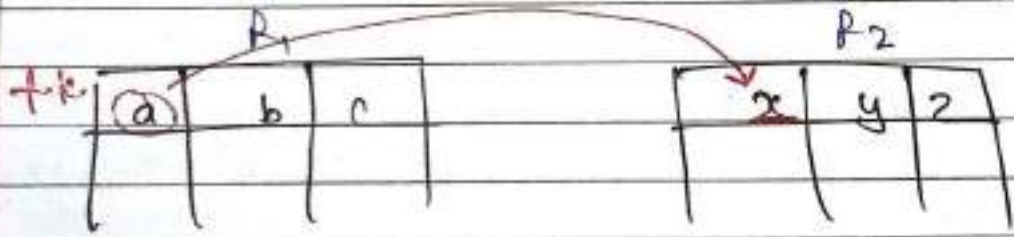
- a) Insert into R_1 x
- b) Insert into R_2 ✓
- c) Delete from R_1 ✓
- d) Delete from R_2 x

Which is correct regarding referential integrity?

- 1.) option a & b cause violⁿ
- 2.) option b & c ^{will} cause violⁿ
- 3.) option c & d " "
- 4.) option d & a " " $\rightarrow$ (ok)

Let, x be p.k. in R_2 .

Soln:



Referencing Table.

Base Table
(or) Referenced Table.

Note! $\rightarrow$ If f.k. is not there, then we can do anything (insert, delete, update) without any violⁿ.

13.

SUPER KEY in DBMS $\rightarrow$

Q.

1. Super key is a combination of all possible attributes which can uniquely identify 2 tuples (row) in a table.

→ Candidate key is minimal.

Ex: Candidate key (C.K.) = Roll.No. | Roll no. | name | age

(also Super key)

{ Roll no, name
 Roll no, age
 Roll no, name, age } all are Super key.

Candidate key (C.K.) is that key that add any attribute, it is not super key.

2. name, age × (not Super key).

⇒ Super set of any Candidate key is Super key.

Q: If $R(A_1, A_2, \dots, A_n)$ then how many Super keys are possible
 If $\rightarrow A_1$ is Candidate key.
 $\rightarrow A_1, A_2$ are candidate keys.

Ans: power set $\rightarrow$ how many subsets can be possible of given set.

Ex: $\rightarrow A_1, A_2, A_3$

$\rightarrow 2 \times 2 \times 2 = 8$

(Either take or not take, 2 possibility)

$\therefore 2^n$

Now, Solⁿ $\rightarrow R(A_1, A_2, \dots, A_n)$

i) (यहाँ जो A_1 ली आया है)

A_1
 A_1, A_2
 A_1, A_3
 A_1, A_2, A_3, A_4

$R(A_1, A_2, A_3, \dots, A_n)$
 Compulsory.
 $1 \times 2 \times 2 \times \dots \times 2$

So, $[2^{n-1}]$ Ans.

ii) $R(A_1, A_2, A_3, \dots, A_n)$

~~For A_1, A_2 both C.K.~~

A_1, A_2 both C.K.

when $A_1 \rightarrow$	when $A_2 \rightarrow$
A_1 A_1, A_2 A_1, A_2, A_3	A_2 A_2, A_1 A_2, A_1, A_3
2^{n-1}	2^{n-1}

But, some sets are common (b/c, $A_1 A_2 = A_2 A_1$).

So, Common are those who has both A_1, A_2 .

So, there are 2^{n-2}

Now,

$$\text{ii)} \quad \frac{2^n + 2^{n-1} - 2^{n-2}}{2^{n-2}} \quad \underline{\underline{\quad}}$$

iii) A_1, A_2 combined is C.K.
 $\frac{2^{n-2}}{2^{n-2}} \quad \underline{\underline{\quad}}$

iv) $\frac{A_1 A_2}{2^{n-2}}, \frac{A_3 A_4}{2^{n-2}}$ are C.K.

Now,

$$\begin{array}{c|c} A_1 A_2 A_3 A_4 & A_1 A_2 A_3 A_4 \\ A_1 A_2 A_3 A_4 - & A_3 A_4 A_1 A_2 - \end{array}$$

To remove these common Elements.

$$\frac{A_1 A_2 A_3 A_4 - A_3 A_4 A_1 A_2 -}{2^{n-4}} \quad \underline{\underline{\quad}}$$

$$\text{v)} \quad \frac{2^{n-2} + 2^{n-2} - 2^{n-4}}{2^{n-4}} \quad \underline{\underline{\quad}}$$

14. E-R Model' → (Entity Relationship Model)

→ used for logical representation.

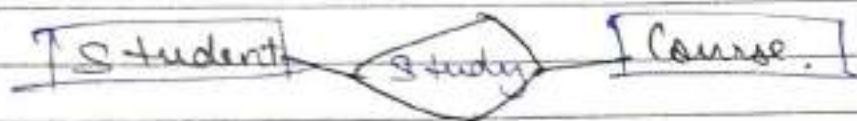
→ To see logical structure before implementation (Design).

→ It does the job of database design.

Entity - Any object which has physical existence is Entity.

Ex - Student (roll no, age, address)
(Entity) attributes

→ Relationship → Relationship b/w 2 or more Entities



Relationship b/w student & course is of study.

⇒ { Student (roll no, age, address) }
Entity type (schema).

⇒ We implement these by using SQL
(Structured Query lang.).

→ Entity.

→ Attributes - characteristics of Entity.
(types)

→ Relationship.

(types)

1 to 1

1 to many

many to 1

many to many

4 types

Entity → Student → represented by rectangle []

Attribute → ○

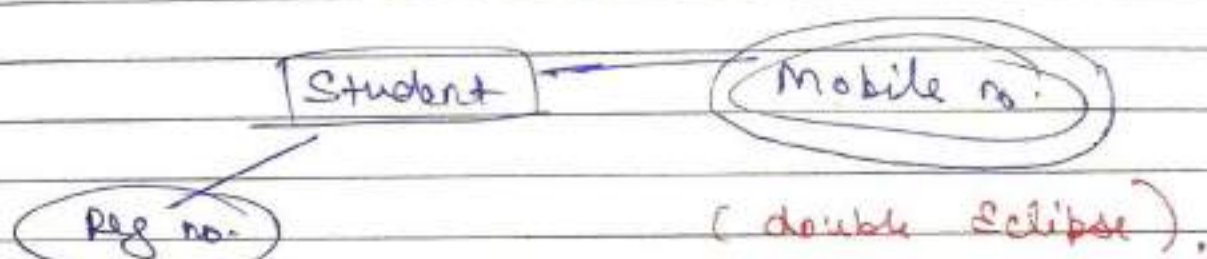
Relationship → ◇

15. Types of Attributes in ER Model

[Student]

1.) Single vs Multivalued attributes :-

↓
Reg no. ↓
Mobile no. (May be more than 1)
or address (C.No. of a student)

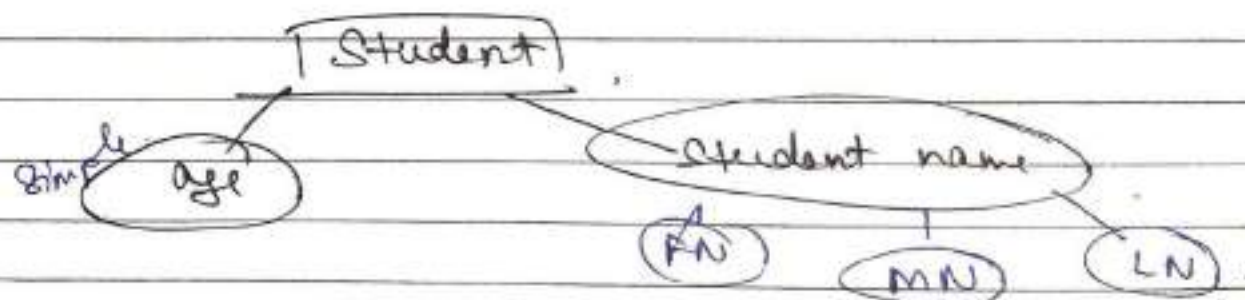


2.) Simple vs Composite Attributes

Simple - Can't be further divided

Composite - Composed of more than 1 value

Ex:- name (first name, middle name, last name)

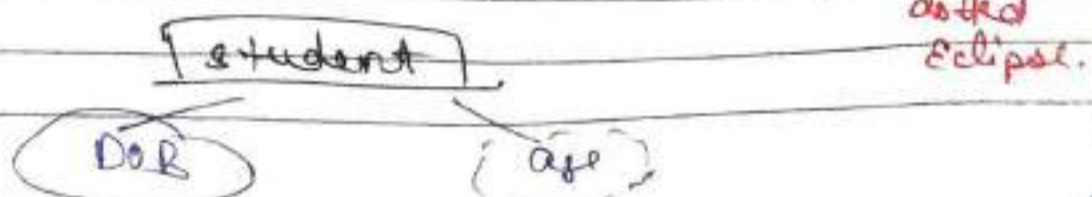


3.) Stored & Derived Attributes :-

Stored → These are stored & can't be derived

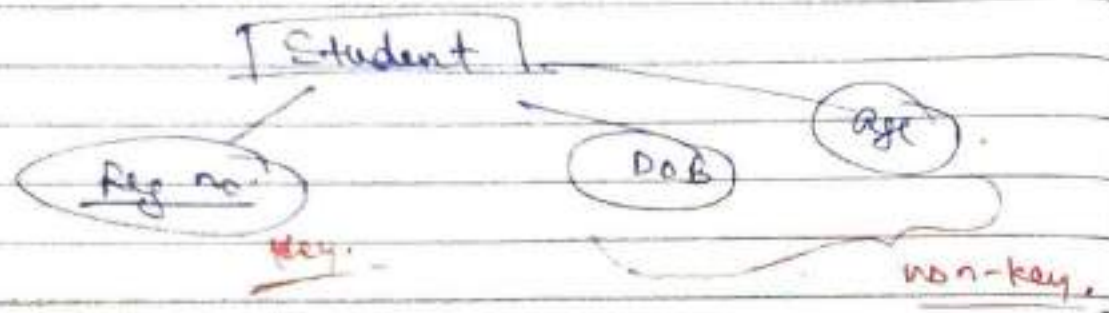
Ex - DOB

Derived → Ex - Age. (Derived from D.O.B.)



4.1) key vs Non-key Attributes ! →
key - used to uniquely identified.
Unique (No Repetitⁿ)

Eg. - Reg No. is always unique for a student.
Represent with underline (_).

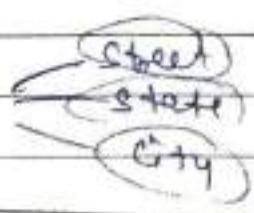


5) Required vs optional Attributes ! →

Required → These are mandatory (*) Name
optional → can also be leave D.OB, Add.

6.1) Complex Attribute ! →
(Composite + Multivalued)

Eg. - If a student have 2 Residential Add,
In Each Add, he have 2 phone no.
Add. is Composite Multivalued.



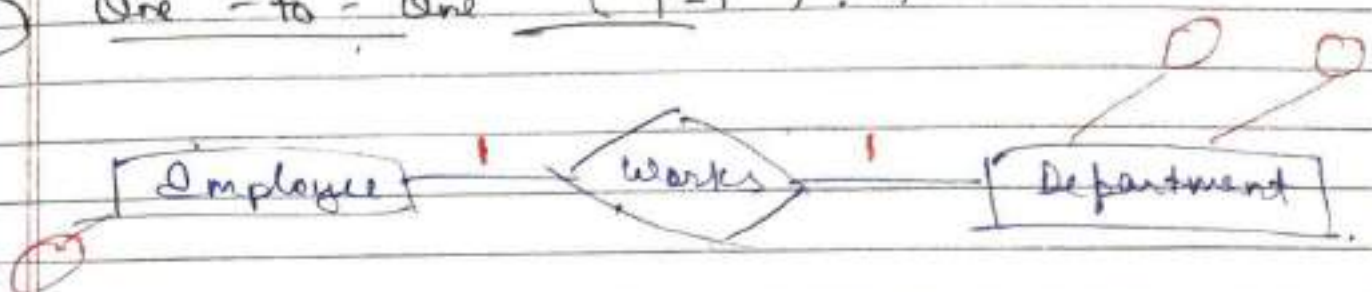
16. Degree of Relⁿship ! → (Cardinality).

→ how the Entities are connected with each other.

4 types! -

- 1.) 1-1
 - 2.) 1-m
 - 3.) m-1
 - 4.) m-m (m-n)
- one to one
many to many

④ One-to-One (1-1) :-



Convert Entity into Table :-

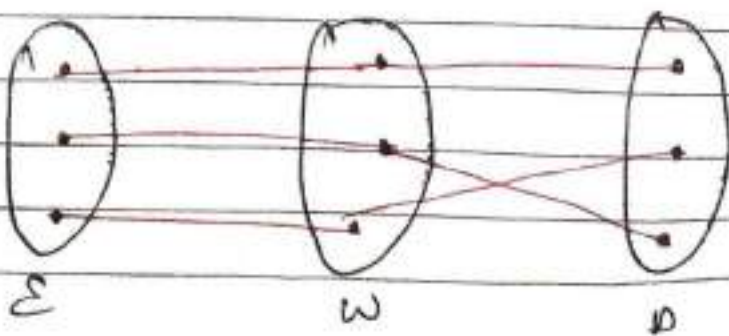
(Relationship of 2 table Employee & Department into Rel. Table)

Rel. Table :- Attributes
→ always (primary keys of both the table)

E.ID & D.ID

These, E.ID & D.ID works as a foreign key (F.K)

→ When we have to enter data in this relationship table, then we have to see relationship (1-1, 1-m, ...)



~~1-1~~

→ P. K. = Either E-ID or D-ID.
(Primary key)

<u>E-ID</u>	Ename	Age	<u>E-ID</u>	D-ID	<u>D-ID</u>	Dname	Loc
E ₁	A	20	E ₁	D ₁	D ₁	IT	Bang.
E ₂	B	25	E ₃	D ₂	D ₂	Prod.	Delhi
E ₃	C	28	E ₂	D ₃	D ₃	HR	Delhi
E ₄	A	24					
E ₅	B	25					

tot

PK = E-ID

Can use Merge?

Now, In Table 1 & Table 2, E-ID is the primary key.

Hence, we can merge Table 1 & 2.

<u>E-ID</u>	Ename	Age	D-ID
E ₁	A	20	D ₁
E ₂	B	25	D ₃
E ₃	C	28	D ₂
E ₄	A	24	—
E ₅	B	25	—

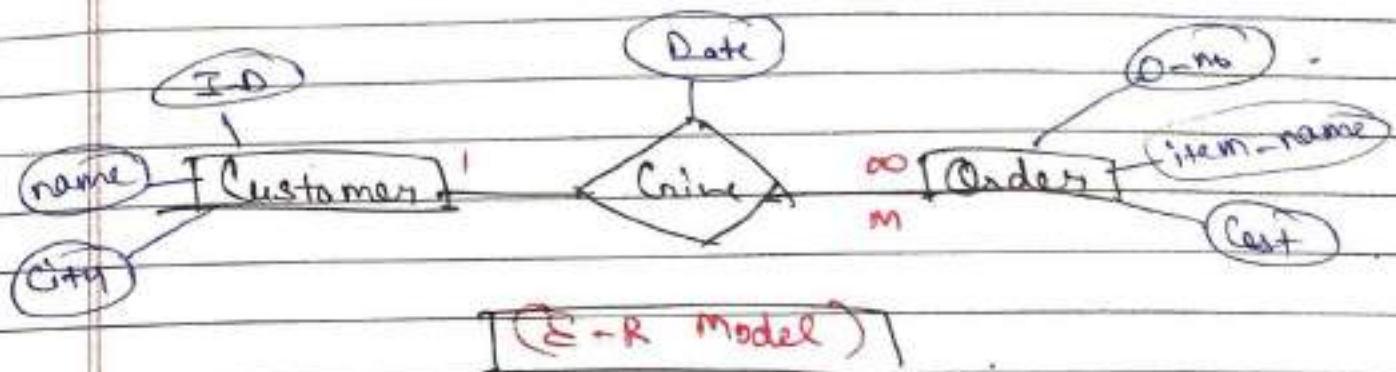
→ Now, we have 2 Table at Final! 3.
(Merge Table & Department Table)

Every Table must have its primary key.

31

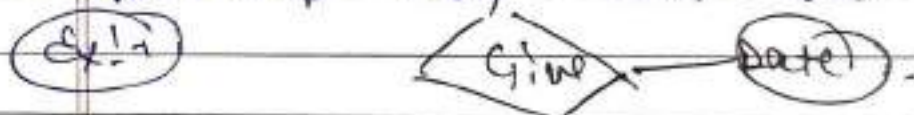
31

15) One to Many Relationship: $\rightarrow$
(1-M)



When we physically implement the ER Model, then we need Relational Model. & we use Tables in Relational Model.

2) Relationship may have its attribute.



& we call it Descriptive Attribute.

<u>Id</u>	Name	City	<u>I-d</u>	<u>O-no</u>	Date	<u>O-no</u>	Item name	Cost
C ₁	A	Tal.	C ₁	O ₁	~	O ₁	Pizza	100
C ₂	B	Delhi	C ₁	O ₂	~	O ₂	Burger	200
C ₃	C	Mumbai	C ₂	O ₃	~	O ₃	Pasta	300
C ₄	A	Mumbai	C ₂	O ₄	~	O ₄	Cold-Drink	400

Here, O-no is always diff. & so unique.

P.K. = (O.no).

Note! Always P.K. of the (many side) in (1-m) is also the P.K. of the Relationship Table.

Can we Merge Tables? (many diff side merge).
yes,

By Relationship & Order Table.
(then both have same P.K.).

→

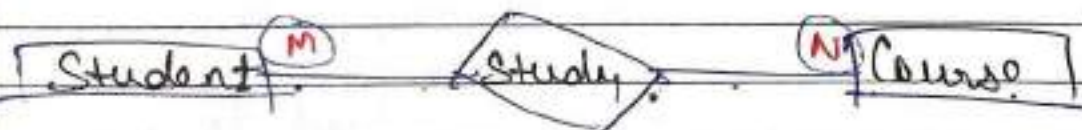
ID	O.no	item name	Cost	Date
~	~	~	~	~

Now,

2 Tables.

(M-N) is also same like this.

18. Many-to-Many Relationship →
(M-N)



Roll no	name	age
1	A	16
2	B	17
3	A	16
4	C	17
5	D	15

Base Table

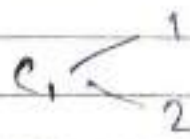
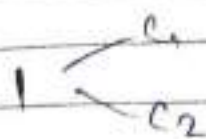
Roll no	C-id
1	C ₁
2	C ₂
1	C ₂
2	C ₁
3	C ₃

Referencing Table

C-id	name	Credit
C ₁	Maths	4
C ₂	phy.	4
C ₃	Chem.	4
C ₄	Hindi	4

Base Table

Many - Many



P.K. in Referencing Table (Relationship Table) :-

Roll. No repeats &
C-id also repeats

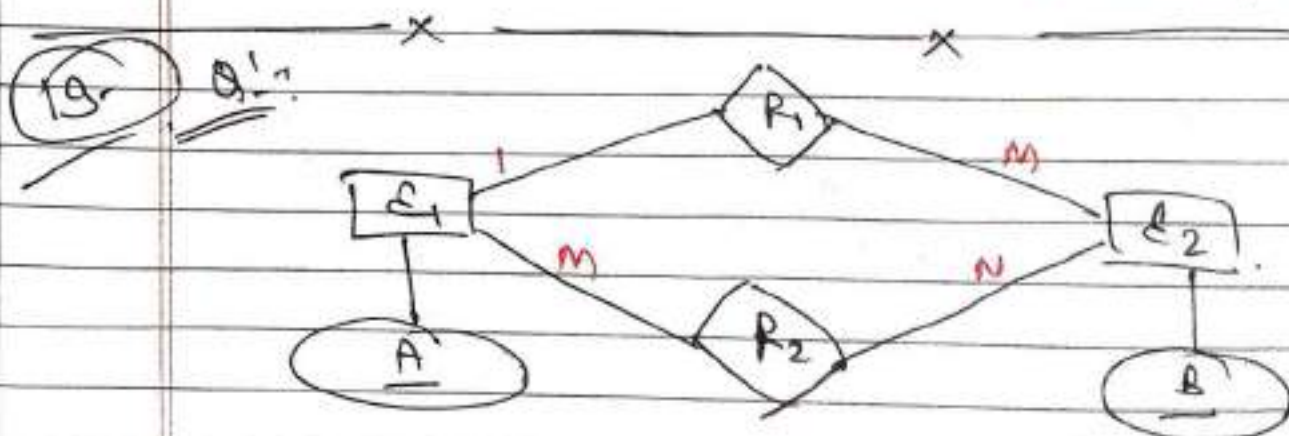
So, Roll no. & C-id both make P.K. combinedly
i.e.,

Composite key = Roll no. C-id

Can use Reduce Tables?

- No. bcoz, P.K. is combined.

Note: P.K. in Relationship Table depends on relationship (1-1, 1-m, ... m-m).

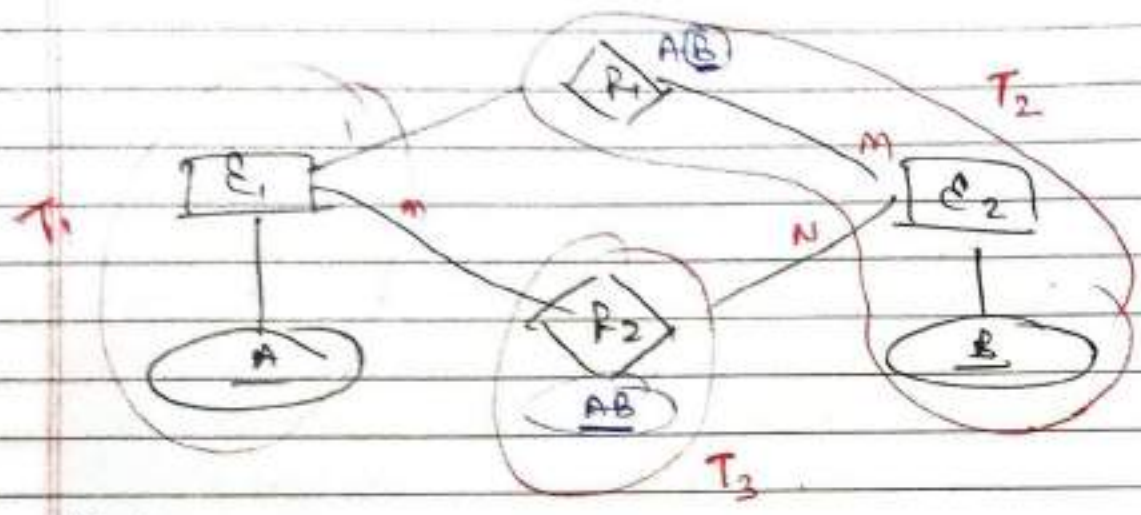


What is the min^m no. of tables required to represent the E-R model into Relational Model?

(5M) ^{value} _{stage}

a) 2 b) 3
c) 4 d) 5

$\left. \begin{matrix} P_1 \\ P_2 \\ P_3 \\ P_4 \end{matrix} \right\}$ 4 Tables. But minimum 3



Hence,

$\left[\begin{matrix} T_1 = E_1 \\ T_2 = R_1 E_2 \\ T_3 = R_2 \end{matrix} \right]$ 3 Tables sh.

E_2 की सभी attributes $R_1 E_2$ में Combined Table में जो जा जाएगी। So Now, we also don't need separate E_2 Table. [Min=3] ✓

Q20. Normalization →

It is a technique to remove or reduce Redundancy (Duplication) from a table.

There are 2 types of duplicacy in Database:-

1.) Row level

2.) Column "

① Row level:-

S-Id	S-name	Age
1	Ram	20
2	Varun	25
1	Ram	20

Same (Dupl. data)

Row level

We use the concept of primary key (P.K.)
We set a P.K. to any appropri. attribute:

Primary key (Unique + Not Null)

P.K. will take care of this duplicacy.

② Column-level:-

Student

Course

Faculty

(P.K.) S-Id	S-name	C-Id	C-name	F-Id	F-name	Salary
1	Ram	C ₁	DBMS	A	John	30,000
2	Ravi	C ₂	Java	F ₂	Bob	40,000
3	Nitin	C ₁	DBMS	F ₁	John	30,000
4	Anurag	C ₁	DBMS	F ₁	John	30,000
5	Varun	C ₁₀	MBBS			

3) 4 columns are same in many rows.

- ~~Insertion~~
- Insertion Anomaly
- Deletion "
- updation "

(Anomaly means problem, occurs on special occasion.)

Now,

Sid	Sname	Cid	Cname	Fid	Fname	Salary
1	Ram	C ₁	DBMS	F ₁	John	20k
2	Ravi	C ₂	Java	F ₂	Bob	40k
3	Nitin	C ₁	DBMS	F ₁	John	20k
4	Amritpal	C ₁	DBMS	F ₁	John	30k
5	Varun	C ₁₀	MBAS			

1) Insertion Anomaly: →

- We want to add data of a new st. let, Varun
- let

University starts a new Course,

C₁₀ - MBAS

- we can't insert this info. in table. Even, we use " — the new faculty data becs
- We only introduce the new Course C₁₀. we don't talk about the student, and we don't have S. Id. &
- We also remain it (S. Id) NULL — bec, it is a P.P.
- So, we can't insert directly.

It is the our Insertion Anomaly.

2.) Deletion Anomaly :-

We have simple query - Remove the database of Roll.No - 1.

Delete from student
where S-id = 1.

It will delete the whole row.

We don't face any problem here.

Now,

We have to delete the data of Roll.No 2.

Delete from student
where S-id = 2

Row 2 is deleted fully from database.

Now,

Row 2 is blank there.

Now,

tell us who is teaching to Roll No. 2
& what was the Course name of Roll.No 2

Likely be, It was only one student who was studying that particular course. & that particular faculty is teaching that course.

→ We here, only delete the detail of student but, bcz of him, all the info got deleted.

Course Info - lost }
Faculty Info - lost }

i.e., extra info is removed here & we can't recover it later.

3.) update Anomaly →

Simple query → [S-Id-4] change name from Amrit to Amritpal.

Update student
Set Sname = 'Amritpal'
Where S-Id = 4

→ Code

No problem here.

Now,

If we want to change the salary of faculty F₁ from 30k to 40k.

Change Salary of F₁ from 30k to 40k.

Now, how many times the F₁ repeats in table, the same no. of times update query runs & changes them all from 30k to 40k.

Note! There is only 1 Faculty F₁, then its salary ^{must} also change 1 times. But, due to the column level duplicacy, it runs no. of times. Hence, it takes more time.

It is update anomaly.

Now, Normalization removes Redundancy.

How?

A simple exⁿ may be, it use divide that table into multiple tables. like,

ex: S-id | S-name

f.k. C-id | C-name

f.k. F-id | F-name | salary.

This can be 1 of the exⁿ.

Now, we don't get any anomaly in insertion, deletion & updⁿ. There is no effect on others. Easy.

21. First Normal Form: → (1NF)

Ex: Codd → father of D.B.M.S

→ Table should not contain any multivalued attribute.

student

Roll No	Name	Course
1	Sai	C/C++
2	Anurag	Jana
3	Ankar	C/ABMS

→ Not in 1st NF

Null means not available



SDM
Lalaya

Now, how to convert in 1st NF: →

1st way

<u>Roll no</u>	Name	<u>Course</u>
1	Sai	C
1	Sai	C++
2	Anurag	Java
3	Ankur	C
3	Ankur	DBMS

primary key (P.K.) = Roll no, Course

Combined, it is composite P.K.

2nd way

<u>Roll no</u>	Name	Course 1	Course 2
1	Sai	C	C++
2	Anurag	Java	Null
3	Ankur	C	DBMS

P.K. = Roll no

2nd way

<u>Roll no</u>	Name	<u>Roll no</u>	<u>Course</u>
1	Sai	1	C
2	Anurag	1	C++
3	Ankur	2	Java
		3	C
		3	DBMS

Base Table

Referencing Table

P.K. = Roll no

P.K. = Roll no, Course

F.K. = Roll no

X

X

22

Closure Method : $\rightarrow$

helps

- 1 To find all the Candidate keys in the Table.

Sol: Candidate key (C.K.)

$R(ABCD)$

FD of $\{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

(Functional Dependency)

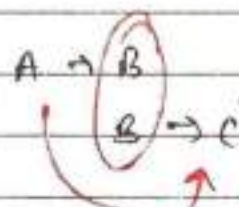
Meaning of closure is that what 'A' can determine

Here, A is determining B (from FD (i))

closure sign

$$A^+ = B$$

$$A^+ = BCDA$$



transitive

(A can determine itself also)

Sol: Roll no. can determine itself.

Now, $R(ABCD)$ has all 4 Attributes that are in $A^+ = BCDA$.

$\therefore$

A can determine all the Attributes of Table. This is the prop. of the Candidate Key.

Now,

$$B^+ = BCD$$

Here, A not determine A.

∴ B cannot be a C.K.

Q

$$C^+ = CD$$

$$D^+ = D$$

∴

only A is C.K.

$$CK = \{A\}$$

Prime att. = $\{A\}$

Non prime att. = $\{B, C, D\}$

∴

Note:

$$(AB)^+ = ABCD$$

$$\begin{pmatrix} AB \text{ - itself} \\ B \rightarrow C \\ C \rightarrow D \end{pmatrix}$$

Here,

AB can be a C.K.

But,

It is not a C.K.

bcz

C.K. is always minimal.

Q

In AB only A is C.K.

∴ AB is super key (S.K.).

$$\overline{AB}$$

Super key

∴

AB is a Super key.

(A se Sooth add any thing, becomes S.K.)

Ex:

$$R(ABCD)$$

$$FD = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$$

$$A^+ = ABCD$$

$$B^+ = BCDA$$

$$C^+ = CDAB$$

$$D^+ = DABC$$

$$\therefore \text{C.K.} = \{A, B, C, D\}$$

all 4.

prime Attribute → are Attribute which is used in making of the C.K.

prime att. = $\{A, B, C, D\}$. X bcz, all p
are c.k.

Non-prime att. = $\{ \emptyset \}$. $\rightarrow$ NULL

Q!- R(ABCDE).

$$FD = \{A \rightarrow \underline{B}, BC \rightarrow \underline{D}, E \rightarrow \underline{C}, D \rightarrow \underline{A}\}$$

2) Now, we have to check that which attributes are coming on the right side. i.e., attributes on the right side, it is to determine if it is no.

ϵ B D C A

$$A = B \cup C \cup E$$

(१६ वली आया)

Note: 1) Each & Every candidate key must contain ϵ .
 2. If ϵ is present on left side, then only it is written on right side also.

जो ~~अस~~ Right कद में नहीं आ रहा। यानि वो Left कद में लेना ही चाहिये। तो Candidate key बनाने में पर लेगा ही होगा।

Now,

$$Q^+ = Q_c$$

→ (e alone is not candidate key
but, it is used in making C.K.)

Now, Start!

With $n! \sim$

AE, AB, EC, D

$\therefore A \perp AR$

$BE^+ = BECDA$
 $CE^+ = CE$
 $DE^+ = DEABC$

C.K. = $\{AE, BE, DE\}$

Trick! - First, we got AE as C.K. So, check (A or E) on the right side of FD.

FD = $\{A \rightarrow B, BE \rightarrow D, E \rightarrow C, D \rightarrow A\}$

So,

directly, DE is also becomes the C.K., now check if D, it is depend on 2. So, check them with

prime Att. = $\{A, B, D, E\}$

non-prime att. = $\{C\}$

(used in making C.K.)

Ans

(23)

Functional Dependency! $\rightarrow$ (F.D.)

is the method which describes the relationship b/w the attributes.

$X \rightarrow Y$ $\rightarrow$ Dependent Att.
 Determinant $\rightarrow$ X determines Y (or)
 Y is determined by X.

Ex!

$S-id \rightarrow Sname$
 1 $\rightarrow$ Ranjit
 2 $\rightarrow$ Ranjit

valid.

} i.e. These 2 are diff.

Ex!

1 $\rightarrow$ Ranjit
 1 $\rightarrow$ Ranjit

} Same Student

Valid case

Ex: $\begin{matrix} 1 & \rightarrow & \text{Parrot} \\ 2 & \rightarrow & \text{Warren} \end{matrix} \quad \left. \vphantom{\begin{matrix} 1 \\ 2 \end{matrix}} \right\} \text{Valid.}$

Ex: $\begin{matrix} 1 & \rightarrow & \text{Parrot} \\ 1 & \rightarrow & \text{Warren} \end{matrix} \quad \left. \vphantom{\begin{matrix} 1 \\ 1 \end{matrix}} \right\} \text{Not Valid.}$

Q. F. D. are of 2 types: $\rightarrow$

- 1.) Trivial F.D.
- 2.) Non-Trivial F.D.

1.) Trivial F.D. $\rightarrow$

If $x \rightarrow y$
then,

y is subset of x .

These Trivial F.D. are Valid. (Always True).

Ex: $\frac{\text{Sid}}{x} \rightarrow \text{Sid}_y$ ✓

NOTE: $\rightarrow$

$x \rightarrow y$

$LHS \cap RHS \neq \emptyset$ (Never Null)

Ex: $\rightarrow$

$\text{Sid}_{\text{Sname}} \rightarrow \text{Sid.}$

$\cap \text{Sid.}$ ✓ Valid.

2.) Non-Trivial F.D. $\rightarrow$

If $x \rightarrow y$

then,

y is not a subset of x .

i.e. $X \cap Y = \phi$ (NULL)

Ex:-

Sid $\rightarrow$ Sname

Sid $\rightarrow$ phone no.

Sid $\rightarrow$ "Loca"

Now, In this we have to check cases.
to find which is valid or not.

④ properties of F.D:-

1.) Reflexivity: If Y is subset of X , Trivial
then $X \rightarrow Y$ (Sid $\rightarrow$ Sid)

2.) Augmentation:

If $X \rightarrow Y$, then $XZ \rightarrow YZ$

(Sid $\rightarrow$ Sname
Sid phone $\rightarrow$ Sname phone)

3.) Transitive:

If $X \rightarrow Y$ & $Y \rightarrow Z$

then,

$X \rightarrow Z$

Sid $\rightarrow$ Sname & Sname $\rightarrow$ City
Sid $\rightarrow$ City.

4.) Union:-

If $X \rightarrow Y$ & $X \rightarrow Z$

then

$X \rightarrow YZ$

5.) Decomposition! If

$$\underline{X} \rightarrow YZ \quad \text{then}$$

$$X \rightarrow Y \quad \text{and} \quad X \rightarrow Z$$

But,

$$\underline{XY} \rightarrow Z \quad X \rightarrow Z, \& \quad Y \rightarrow Z \quad X$$

6.) Pseudo Transitive!

If

$$X \rightarrow Y \quad \& \quad \underline{WY} \rightarrow Z \quad \text{then}$$

$$\underline{WX} \rightarrow Z$$

7.) Composition!

$$\text{If } \underline{X} \rightarrow Y \quad \& \quad \underline{Z} \rightarrow W$$

then

$$\underline{XZ} \rightarrow YW$$

24.) 2nd Normal Form! $\rightarrow$ (2nd NF).

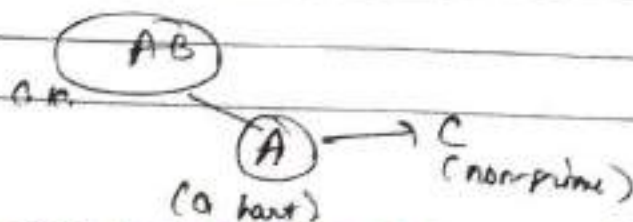
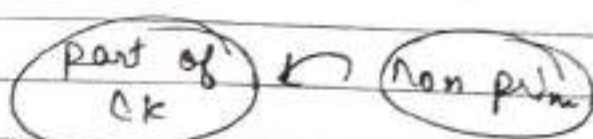
2 Rules

$\rightarrow$ Tables as Relⁿ must be in 1st NF.

$\rightarrow$ All the non-prime attributes should be fully functional dependent on Candidate key (C.K.) or (P.K.).

[There should be no partial dependency in the Relⁿ].

Ex:



If it's partial depend.

is not 2nd NF.

Ex: 1

Customer

<u>Customer - Id</u>	<u>Store - Id</u>	Loca ⁿ
1	1	Delhi
1	3	Mumbai
2	1	Delhi
3	2	Bangalore
4	3	Mumbai

C.P. : Customer - Id Store Id

Prime attributes : C - Id
Store - Id.
Non prime : Locaⁿ

Here, Locaⁿ is only depend on Store - Id.
i.e. partial dependency.

But,
(to be in 2nd NF it should depend on the both C-id & S-id (bcoz both are C.P.)).

Now →

Convert it into 2nd NF : →

Here, we make 2 Table.

<u>C-Id</u>	<u>Store-Id</u>
1	1
1	3
2	1
3	2
4	3

<u>Store-Id</u>	Loca ⁿ
1	Delhi
2	Bangalore
3	Mumbai

(Here, it is fully dependency bcoz only 1 C.P. is here)

2nd NF

Q!! R (A B C D E F)

FD: $\{c \rightarrow f, e \rightarrow a, e \rightarrow d, a \rightarrow b\}$

Ans: [C.K. 2]

First, check Right hand side

Step 1:

f A D B

means
Now, these attr are
determined by some of
the values.

So,

On LHS, there must be CE.

CE = FADB

(C.F. जो भी बनेगी,
उसके CE तो होगा
ही)

Now,

$EC^+ = EC F A D B$ (all 6 are present)

$\therefore$, EC is C.K.

Now, Use Trick

Either E or C must be present at
the RHS of any F.D.

but

not, neither E nor C, none is present.

So,

there is only 1 C.K. in this table.

i.e.

[C.K. = $\{EC\}$]

Ans.

Proof! check

Step 1 is comp. here.

After finding C.K. = $\{EC\}$

$A^+ = AB$
 $B^+ = B$
 $C^+ = CF$

$\therefore$, Proved.

proper subset is always less than a set.

$X \subset XY \rightarrow$ proper subset
 $X \subseteq XY \rightarrow$ subset

Step 2 $\rightarrow$ prime Attributes: $\{E, C\}$

non-prime attributes: $\{A, B, D, F\}$

Step 3 $\rightarrow$ C.K. = $\{E, C\}$

What is proper subset of C.K.

$\rightarrow$ Either 'E' or 'C'.

Now check:-

F.D. = $\{C \rightarrow F, E \rightarrow A, EC \rightarrow D, A \rightarrow B\}$

Check

(Either proper subset of C.K.)

for partial dependency
 i.e.,
 not in 2nd NF

Check on LHS $\rightarrow$ Either 'E' or 'C' AND

check on RHS $\rightarrow$ whether it is non-prime att.

we get,

$C \rightarrow F$

$\hookrightarrow$ partial dependency,

So,

Table is not in 2nd NF.

$C \rightarrow F$	$\propto$ PD	partial
$E \rightarrow A$	$\propto$ PD	
$EC \rightarrow D$	$\propto$ FD	fully
$A \rightarrow B$	$\propto$ FD	

1 st P.D.
 Hence, Table is not in 2nd NF

25. 3rd NF $\rightarrow$

1. Table or Rel must be in 2nd NF.

2.

$\rightarrow$ There should be no transitive dependency in table.

Non prime or Non-unique
prime or unique.

gm

Date: _____

Page: _____

91

not sufficient cardin

* Mean, Non-prime att. ko chahi Non-prime att. determine nhi kar sakata.

(C.K. & Prime att. (P.A.) ko kar sakate h determine NPA ko).

Ex 1:

Roll no.	State	City
1	punjab	Mohali
2	Haryana	Ambala
3	punjab	Mohali
4	Haryana	Ambala
5	Bihar	patna

C.K. = α Roll no.
F.D $\rightarrow \alpha$ Roll no + state,
state $\rightarrow$ city

- 1) PA = α Roll no.
- 2) NPA = α state, city

Here, Roll $\rightarrow$ State and State $\rightarrow$ city.

It is Transitive dependency & we don't want that.

so,

It is not in 3rd NF.

Ex 2: R (ABCD)

FD: $\{ AB \rightarrow C, C \rightarrow D \}$

- 1) C.K. = α AB
- 2) PA = α A, B
- 3) NPA = α C, D

Transitive
AB $\rightarrow$ ABCD
 $\swarrow$
C $\rightarrow$ D
NPA NPA

It is not in 3rd NF

C.K. + any thing = S.K.

Super Key



Date: _____

Page: _____

* Ex 1

R (A B C D)

FD: (A B → C D, D → A)

Soln

C.K.: {A B, D B}

PA: {A, B, D}

NPA: {C}

(B not on RHS)

$B^+ = B$

$AB^+ = A B C D$

Now, A on RHS

$DB^+ = D B A C$

is also C.K.

Step

Now, for each F.D.

LHS → C.K. on S.K

OR

RHS → be a P.A.

check only 1 be OR

Then, Table is in 3rd NF

FD: (A B → C D, D → A)

✓

✓

Table is in 3rd NF.

be

(NPA → NPA) is not present here.

26

BCNF. (Boyce Codd Normal Form) ! →

also called as special case of 3rd NF.

How,

~~Table is in 3rd NF. But it is not in BCNF.~~

Student

Roll-no	Name	Uaten-id	Age
1	Ravi	K0123	20
2	Harun	M034	21
3	Ravi	K786	23
4	Rahul	D286	21

Table is in 3rd NF already.

C.K. = { Roll no, water - Id }

f.D. : = { Roll no $\rightarrow$ name
Roll no $\rightarrow$ waterid
waterid $\rightarrow$ age
waterid $\rightarrow$ Roll no. }

Note! LHS of each FD should be C.K. or S.K.

Here, 3NF is the (OR) valid condition
हटा दी, जिसमें RHS के P.A. होने से भी
काम चल जाता था।

Here,

we only want C.K. or S.K. in L.H.S. &
RHS में कोई-लैंग देना नहीं।

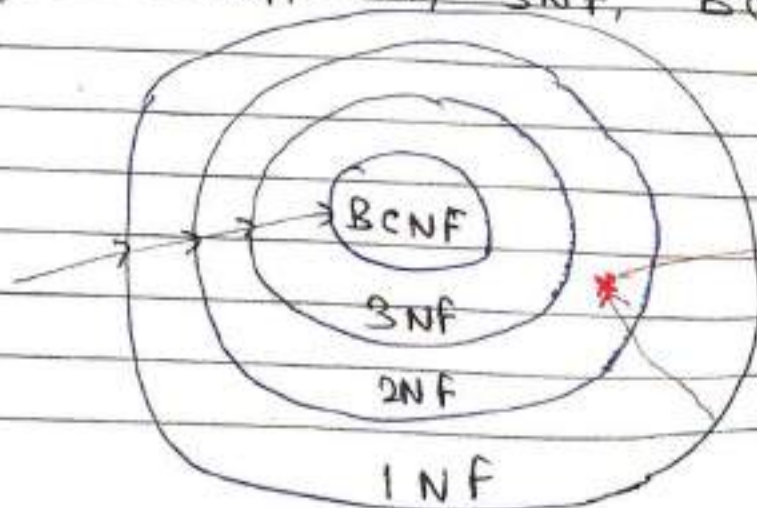
Soln!

So, check all the f.D. one-by-one.

In all the 4 f.D, the LHS is C.K.

So, It is in BCNF form.

Comparison: 1NF, 2NF, 3NF, BCNF !



It is outside
3NF &
in 2NF &
1NF both.

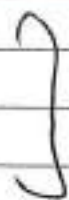


Date: _____

Page: _____

10

1NF.

2NF = 1NF + condiⁿ3NF = 2NF + condiⁿBCNF = 3NF + condiⁿ

(27)

Lossless & Lossy Decomposiⁿ ! →

We normalise table → or we decompose table into 1NF, ... forms.

R	A	B	C
	1	2	1
	2	2	2
	3	3	2

→ R₁ (AB)

→ R₂ (BC)

We divide this table into R₁ & R₂

Q

B is common in both the Table.

R ₁	A	B
	1	2
	2	2
	3	3

R ₂	B	C
	2	1
	2	2
	3	2

⇒ find the value of C if the value of A = 1.

Now, for this we have to join R₁ & R₂ tables.

So,

Select $R_2.C$ from R_2 Natural Join R_1
where $R_1.A = '1'$

(1st row of multiply all rows at start of Table 2)

cross product: - If R_1 has x rows &
 R_2 has y rows

then their join has $x.y$ rows.

condiⁿ: - Common ~~element~~ col^m of both
Tables (R_1 & R_2) - here (B) has the
same value in join Table.

Natural Join = Cross product + Condiⁿ.

Now,

	R_1		R_2		
	A	B	B	C	
{	1	2	2	1	✓
	1	2	2	2	✓
	1	2	3	2	
{	2	2	2	1	✓
	2	2	2	2	✓
	2	2	3	2	
{	3	3	2	1	
	3	3	2	2	
	3	3	3	2	✓

Now,

R_1

A	B	C
1	2	1
1	2	2
2	2	1
2	2	2
3	3	2

table after
Joining.

(Note).
Spurious of
tuples.

- In original Table (R), we have only 3 tuples (rows), but, after joining, in R' , we have 5 tuples.

It is a ^(conv) flaw, it is called the **Lossy Decomposition**.

- Why lossy?

Here, we get 2 extra rows, then, why lossy.

Here,

We don't talk about rows. We call it lossy because of inconsistency. There is a problem in Database.

⇒ In original, for $A=1$, $C=1$.
but

In join table, for $A=1$ $\left\{ \begin{array}{l} C=1 \\ C=2 \end{array} \right\}$

- ① Why we get Losses? (2 tuples more)

A bcz, we take B as Common in both Table.
But

Criteria for Common → Common Attribute should be C.K. or S.K. of either R_1 or R_2 or both.

So, we have to C.K. or S.K. of original Table.

1) Δ R has duplicacy in Table. We have to choose attri. 'A' for Pk'd Attr.

bcz, A is unique. $\langle 1, 2, 3 \rangle$.

for

R ₁	(A, B)
R ₂	(A, C)

→ We get 3 tuples also in joining table.

Condiⁿ for lossless Joining Decomposⁿ! →

1.) $R_1 \cup R_2 \equiv R$

$AB \cup AC \equiv ABC$.

2.) $R_1 \cap R_2 \neq \phi$

$AB \cap AC$

$A \neq \phi$

(C.K. of original Table)

3.) R_1 C.K. (or) R_2 C.K. (or) Both

To take Common attribute →

28. All normal forms with Real life Examples →
6 forms here

	1st NF	2nd NF	3rd NF
1st	→ No multivalued Attribute.	→ In 1st NF +	→ In 2nd NF +
2nd	→ only single valued.	→ No partial Dependency. * only full dependency.	→ No Transitive dependency. → No, NP A. should determine N.F. A.
		$(AB) \rightarrow C$	
		If A & B are 2 keys of a Co., then both will use the Emplo. 'C'.	

BCNF 4th N.F. 5th N.F.

→ In 3rd NF → In BCNF → In 4th NF

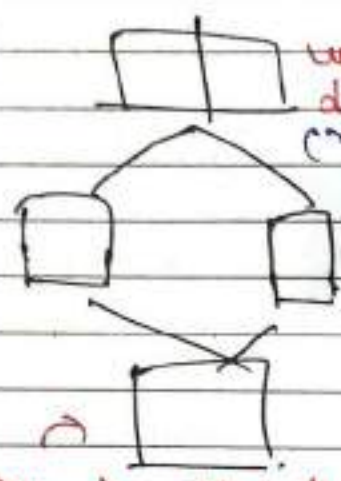
+ + +
 LHS must be C.F. or S.F. No multivalued Dependency lossless decomposition



$X \twoheadrightarrow Y$

Varun → 3 Phone no.
 → 3 Mail Id

(Varun depends on multiple attr. i.e. phone & mail)



vertical divide (rows) can

table

Varun	M ₁	E ₁
Varun	M ₁	E ₂
1	M ₁	E ₃
1	M ₂	E ₁
1	M ₂	E ₂
1	1	1

Very long table
 → take also don't able to form a key.

may be extra tuples come. So, make C.F. as common attribute in both tables.

So,

Make 2 table.



(Now, no multivalued dependency)

29. Minimal Cover : (Ir-reducible)!

Q! For the following functional dependencies, find the correct Minimal Cover →

{ $A \rightarrow B$, $C \rightarrow B$, $D \rightarrow ABC$, $AC \rightarrow D$ }

- $A \rightarrow B, C \rightarrow B, D \rightarrow A, AC \rightarrow D$
- $A \rightarrow B, C \rightarrow B, D \rightarrow C, AC \rightarrow D$
- $A \rightarrow BC, D \rightarrow CA, AC \rightarrow D$
- $A \rightarrow B, C \rightarrow B, D \rightarrow AC, AC \rightarrow D$

Ans: Our RHS in f.d. must be single.

Step 1 $\{A \rightarrow B, C \rightarrow B, D \rightarrow \underline{ABC}, AC \rightarrow D\}$

By decomposⁿ prop, separate them.

$A \rightarrow B, C \rightarrow B, D \rightarrow A, D \rightarrow B, D \rightarrow C, AC \rightarrow D$

Step 2 Remove the Redundant f.d. $\rightarrow$

$A \rightarrow B, C \rightarrow B, D \rightarrow A, \underline{D \rightarrow B}, D \rightarrow C, AC \rightarrow D$
 Let: $\times$ $\checkmark$ $\checkmark$ $\checkmark$ $\checkmark$ $\checkmark$

$\rightarrow$ Let, $A \rightarrow B$ को हटा दें, Now check the closure of A,

$A^+ = A$ (not all +)

$\therefore, A \rightarrow B$ is not Redundant. We can't remove it.

$\rightarrow$ Same check this for every f.d. $\rightarrow$

$D \rightarrow B$ is Redundant.

Let, $D \rightarrow B$ $\times$ then, $D^+ = DABC$

$\therefore$ $D \rightarrow B$ (Call r).
 [remove, $D \rightarrow B$]

उदा उदा आगे check करेंगे, वो इसको नहीं लेगे, इसको तो हटा ही remove कर दिया।

Now, for

$$AC \rightarrow D \quad \times$$

$$A^+ = ACB$$

also includes

$$AC \rightarrow D \quad \checkmark$$

Now, we get

$$\{A \rightarrow B, C \rightarrow B, D \rightarrow A, D \rightarrow C, AC \rightarrow D\}$$

Step 3

Now, use only want 1 Att in LHS.

Here,

$$AC \rightarrow D \quad \times$$

Now, check by remove A. & then check, closure of C.

$$C^+ = CB$$

जबकि C^+ में हमें 'A' का मिल जाना, तो हम A को हटा सकते हैं। लेकिन नहीं मिला, तो,

$$AC \rightarrow D \quad \text{ही रहेगा।}$$

Same check & A, by removing C.

$$A^+ = AB$$

∴ Can't remove C.

∴ $AC \rightarrow D$ can't be reduced.

∴

$$\{A \rightarrow B, C \rightarrow B, \underbrace{D \rightarrow A, D \rightarrow C}_{\text{complete}}, AC \rightarrow D\}$$

∴

$$\{A \rightarrow B, C \rightarrow B, D \rightarrow AC, AC \rightarrow D\}$$

Ans:

36

Question on Normalisation ! →

Q! R (AB C D E F), check the highest normal form ?

F.D. ! $\{ AB \rightarrow C, C \rightarrow DE, E \rightarrow F, F \rightarrow A \}$.

Ans! Step 1: Find all CKs in Relⁿ ! →
By closure Method ! →

(B is not on RHS.) So, B compulⁿ.

$$B^+ = B, \quad A^+ = A$$

$$AB^+ = ABCDEF \quad (\text{all } +)$$

1. $[AB \text{ is C.K.}]$

Now, check $\forall A$ on RHS.

we get, $F \rightarrow A$

2. $[FB \text{ is also C.F.}]$

Now, check F on RHS ! →

$E \rightarrow F$

So, $[EB \text{ is also C.K.}]$

Now, check E on RHS.

$C \rightarrow DE$

So, $[CB \text{ is also C.F.}]$

Now check $\forall C$ on RHS.

$$AB \rightarrow C$$

AB is already an C.F.
So, we get all C.F.

$$C.F. = \{ AB, FB, CB, CB \} \therefore 4 \text{ CF}$$

Step 2 $\rightarrow$ Write all prime att. & NPA! $\rightarrow$

$$P.A. = \{ A, B, C, E, F \}$$

$$NPA = \{ D \}$$

Step 3 $\rightarrow$ Now, check FD! $\rightarrow$

$$\{ AB \rightarrow C, C \rightarrow DC, E \rightarrow F, F \rightarrow A \}$$

Now,

check 1-by-1.

Highest NF $\rightarrow$ 1NF, 2NF, 3NF, BCNF,

↓
Redundancy decreases

$\rightarrow$ mean, when table is in BCNF, then redundancy is lowest. & in 1NF redundancy is highest.

$\rightarrow$ So, to check highest NF, we start from BCNF! $\rightarrow$

$\rightarrow$ In BCNF, we know, all LHS of all FD's should be CF or SF.

$\{ \underline{AB} \rightarrow C, \quad C \rightarrow DE, \quad E \rightarrow F, \quad F \rightarrow A \}$

$\downarrow$

It is not in BCNF form.

→ Now, 3NF! →

Check! → transitive dependency $(NPA \rightarrow NPA)$ x

LHS → C.P. or S.P.
 $\underline{AB}$
 RHS → is a P.A.

→ then it is 3NF.

Now, 1st Cond'n is already checked in BCNF.
 So, here check only 2nd ~~cond'n~~ cond'n that whether RHS is P.A. or not.

	$\{ \underline{AB} \rightarrow C$	$\{ C \rightarrow DE$	$\{ E \rightarrow F$	$\{ F \rightarrow A$
BCNF	✓	x	x	x
3NF	✓	x	✓	✓

∴ not in 3NF! →

→ Now, 2NF! →

Same thing → if there is already a tick in 3NF, then we don't have to check that for 2NF. If already 2NF is satisfied! Check only for 'x' tick.

Check!

LHS is proper subset of C.F.
RHS is non-prime att.

for partial dependency i.e.,
if true then not in 2nd NF.

	$AB \rightarrow C$	$C \rightarrow DE$	$E \rightarrow F$	$F \rightarrow A$
BCNF	✓	✗	✗	✗
3NF	✓	✗	✓	✓
2NF	✓	✗	✓	✓
1st NF	✓	✓	✓	✓

(check both cond'n for)
not in 2NF.

Ans is 1st NF.

why

bcz

for 1st NF that we don't want any multivalued attribute in the table. All att. must be single (atomic).

in

Table $\rightarrow$ R (A B C D E F)

all are general att.,

we can't tell them as atomic or multivalued by seeing them.

in

Table already 1st NF में ही मिलती है.
(assume already).

1st NF.

Sho?

proper subset is always less than a set.



CS

31. Find out Normal Form of a Relⁿ! $\rightarrow$
(from 1NF - BCNF).

Q:- R(ABCDE F),

FD's: $\{AB \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F, F \rightarrow A\}$

C.K. = $\{AB, FB, EB, CB\}$.

P.A. = $\{A, B, C, E, F\}$

NPA = $\{D\}$

Solⁿ: Now, let us assume that R(ABCDE F) is already in 1st NF.

$\rightarrow$ Check 2nd NF! $\rightarrow$

(partial depen) P.D.

Cond'n $\rightarrow$ $\left\{ \begin{array}{l} \text{LHS must be proper subset} \\ \text{of any C.K.} \\ \text{[and]} \\ \text{RHS must be a Non-P.A.} \end{array} \right\}$

FD's $\{AB \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F, F \rightarrow A\}$
 $\begin{array}{cccccc} \text{F} & \text{K} & \text{F} & \text{T} & \text{K} & \text{T} & \text{T} & \text{K} & \text{F} & \text{T} & \text{K} & \text{F} \\ \text{FO} & & & \text{P.D.} & & & \text{FO} & & \text{FO} & & \text{FO} & \end{array}$

(Full dependency)

$\therefore$ not in 2nd NF.

$\rightarrow$ make it in 2nd NF! $\rightarrow$

We do it by decompose the table.
(Divide into 2 parts).

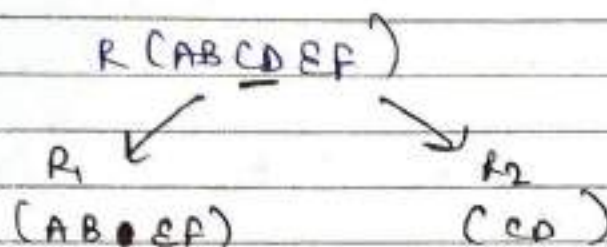
Cond'n! :-

Common attribute must be C.K.

- 1.) lossless decomposition
- 2.) Dependency should be preserved.

Now, let problem of 1st, $C \rightarrow D$ 3rd part
 Along with it, Along Table also it.

Now



Now, we have to make a common att. b/w these 2 table! -

Criteria & common! - can be C.K. of any R_1 & R_2 .

So, In $R_2 (CD)$

$C \rightarrow D$

$C = CD$ (all 2)
 $\therefore C$ is C.K.

So,

make 'C' common in both R_1 & R_2 .

Now,

again

$R(ABCD \underline{EF})$



R_1
 $\{ AB \rightarrow C, C \rightarrow E, E \rightarrow F, F \rightarrow A \}$

2nd NF

R_2

$\{ C \rightarrow D \}$

$\{ C, D, E, F, A \}$
 (F, D)

2nd NF

$\{ C \}$ itself is not its proper subset

Now!

check $\forall$ 3NF! $\rightarrow$

$\left\{ \begin{array}{l} \text{LHS must be C.K.} \\ \text{RHS must be P.A.} \end{array} \right\} \rightarrow \text{3NF}$

So,

R_1
(A B C E F)

$\{ AB \rightarrow C, C \rightarrow E, E \rightarrow F, F \rightarrow A \}$

$\text{C.K.} = \{ AB, FB, EB, CB \}$
 $\text{P.A.} = \{ A, B, C, E, F \}$

It is 3rd NF.

R_2
(C D)

$\{ C \rightarrow D \}$

$\text{C.K.} = \{ C \}$
 $\text{P.A.} = \{ C, D \}$

It is also 3rd NF.

Now!

check $\forall$ BCNF! $\rightarrow$

Condiⁿ $\rightarrow$ L.H.S. must be a C.K.

$R_1 \{ AB \rightarrow C, C \rightarrow E, E \rightarrow F, F \rightarrow A \}$

not in BCNF form.

There is redundancy.

Now,

Again Decompose R_1
(A B C E F)

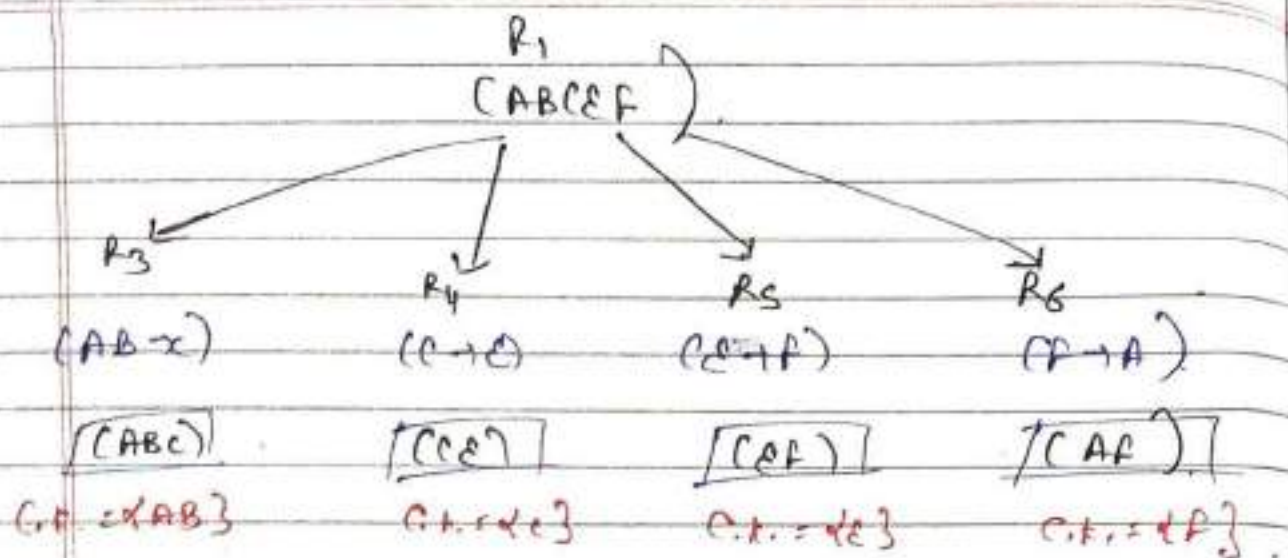
(will problem on 2nd, 3rd, 4th, 5th, 6th, 7th, 8th, 9th, 10th, 11th, 12th, 13th, 14th, 15th, 16th, 17th, 18th, 19th, 20th, 21st, 22nd, 23rd, 24th, 25th, 26th, 27th, 28th, 29th, 30th, 31st, 32nd, 33rd, 34th, 35th, 36th, 37th, 38th, 39th, 40th, 41st, 42nd, 43rd, 44th, 45th, 46th, 47th, 48th, 49th, 50th, 51st, 52nd, 53rd, 54th, 55th, 56th, 57th, 58th, 59th, 60th, 61st, 62nd, 63rd, 64th, 65th, 66th, 67th, 68th, 69th, 70th, 71st, 72nd, 73rd, 74th, 75th, 76th, 77th, 78th, 79th, 80th, 81st, 82nd, 83rd, 84th, 85th, 86th, 87th, 88th, 89th, 90th, 91st, 92nd, 93rd, 94th, 95th, 96th, 97th, 98th, 99th, 100th)

$R_2 \{ C \rightarrow D \}$

It is BCNF form.

Redundancy = 0%

Normaliseⁿ aim $\rightarrow$ To make redundancy 0%. SM $\rightarrow$ like decompose & normalise.



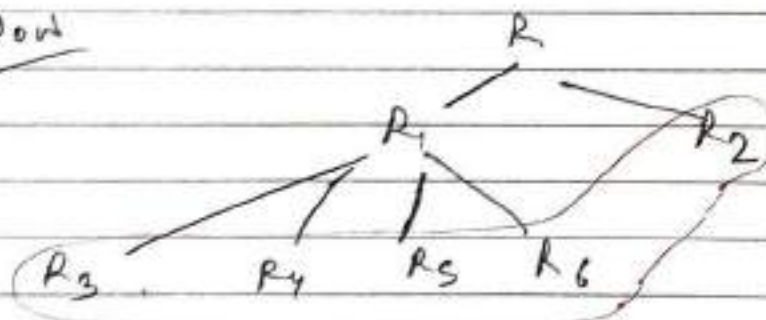
Now, these all 4 tables have C.K.

So,

These all 4 are in BCNF now

$\hookrightarrow$ (C, E) - common key in all 4 tables

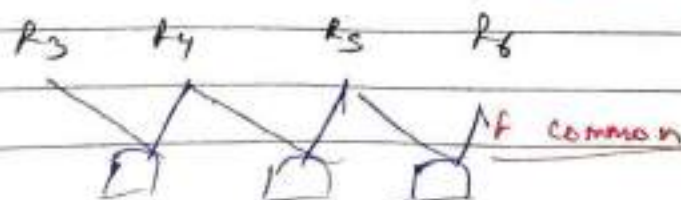
Now



Now, in all these 5 tables, Redundancy is 0%.

But, there are now multiple tables & if we join them again, it is complex.

Let's have to join R_3 & R_4 (both have common C).



→ Bat, John R_3 & R_5 .
(ABC) (EF).

(b/c nothing is common)

We can't join them directly. So, we take help from R_4 .

→ So, Combine R_3 & R_4 , let $R_3 R_4$ (ABCE)

Now,

we can combine it with R_5 b/c both have now 'E' as common attr.

$R_3 R_4 R_5$ (ABCEF)

→ E is C.K. now

32. Normalisation Questions: →

→ A relⁿ is in BCNF, then it is in 2NF also

Q: A Relⁿ R has 8 attr. (ABCDEFCH)

F = { CH → G, A → BC, B → CFH, E → A, A → EC }

how many candidate keys in R.

a) 3 b) 4 c) 5 d) 6

Solⁿ: First, check on RHS, which attr is absent.

D → so, now it comes with every.

$D^+ = D$

✓ $AD^+ = AD, BC, FH, GE$

(Call 3)

Now, check A on RHS,

so,

✓ ED^+ also a C.P.

Now, check E on RHS.

so,

✓ FD^+ also a C.P.

Now, check F on RHS.

so,

✓ BD^+ also a C.P.

Now, check B on RHS.

Now, completed,

so,

$$\boxed{AD, ED, FD, BD} \rightarrow \text{C.P.}$$

33. Ques Explained on Normalisation! →

Schema is a structure of a Table.

→ so, $\boxed{\text{Schema (or) Table}} \rightarrow \text{same.}$

→ Non trivial F.D. means in which.

$$\boxed{LHS \cap RHS = \phi} \rightarrow \text{we have to check that it is valid or not.}$$

→ Trivial F.D. are always valid.

Schema 1 : Registration (Roll. No., Courses)

Non trivial F.D of roll no $\rightarrow$ Courses

Ans- We have to check that this table is in which form.

$\rightarrow$ So, as always, start from higher forms.

$\rightarrow$ Check 4 BCNF:-

condiⁿ : LHS of every FD must be C.F. or S.F. or P.F.

and,

Roll. no is already given as a C.F.

So,

LHS is a C.F. of F.D.

So,

$\rightarrow$ Table is in BCNF form.

$\therefore$ also in 1st NF, 2nd NF & 3NF.

Schema 2 : Registration (roll no, Course id, email)

Non trivial FD of roll no, Course id $\rightarrow$ email
email $\rightarrow$ roll no.

Ans $\rightarrow$ C.F. = { roll no, Course id }.

PA = { roll no, Co. Id }.

NPA = { email }.

$\rightarrow$ Check BCNF $\rightarrow$ 1st F.D. LHS is C.F.

but not in 2nd F.D.

So,

not in BCNF form.

→ Check for 3NF!

Cond'n!

LHS must be a PK or SK or PK
OR
 RHS must be a f.a.

Now, 1st FD is already valid,
 2

2nd FD: email $\rightarrow$ roll no.

P.A.
 T.

f

2.

(T)

It is in 3NF.

OK

Scheme 3!

Register (roll no, Co-id, marks, grade).

Non trivial FD: { roll no, Co-id $\rightarrow$ marks, grade }
 marks $\rightarrow$ grade }

Ans!

C.f. = { roll no, Co-id }
 P.A = { roll no, Co-id }
 N.P.A = { marks, grade }

→ check for BCNF!

1st FD $\rightarrow$ valid.
 2nd FD $\rightarrow$ not valid.
 2
 not in BCNF.

→ check for 3rd NF!

1st FD $\rightarrow$ already valid
 2nd FD $\rightarrow$ marks $\rightarrow$ grade.
 f f G not in 3rd NF.
OK

→ check $\forall$ 2nd NF! →

cond'n:-

LHS must be a proper subset of (RHS)
[AND]
 RHS must NPA.

↳ for p.d. → i.e., not in 2nd NF.
 if both cond'n are True.

1st FD → already valid.
 2nd FD → marks → grade
 f T

(+f)

↳ not in partial dependency
 ∴ is in 2nd NF.

Ans

Schema 4: Register (sell no, C-Id, Credit)

Non trivial FD {
 sell no; C-Id → Credit
 C-Id → Credit
 }

Ans 1
 C.K. = { sell no, C-Id }
 P.A. = { sell no, C-Id }
 N.P.A. = { Credit }

→ check $\forall$ BCNF! →

1st FD → ✓
 2nd FD → X

} ∴ not in BCNF.

→ check $\forall$ 3NF! -

1st FD → ✓
 2nd FD → C-Id → Credit
 f f

(+f)

∴ not in 3rd NF.

→ Check $\nrightarrow$ 2NF! $\rightarrow$

1st FD $\rightarrow$ ✓

2nd FD $\rightarrow$ C-Id $\rightarrow$ Credit
 $\quad \quad \quad \uparrow \quad \quad \quad \uparrow$
 $\quad \quad \quad \text{C} \quad \quad \quad \text{P}$

(C, P)

→ It is Partial Dependency (P.D.)

so, It is not in 2nd NF. $\Rightarrow$

→ Table is in 1st NF. ids
 (we already assume it).

34. Cover & Equivalence of F.D. $\rightarrow$

$\left\{ \begin{array}{ll} \text{If } X \text{ covers } Y & Y \subseteq X \\ \text{If } Y \text{ covers } X & X \subseteq Y \\ \text{If both are true,} & \\ \text{then,} & [X \equiv Y] \rightarrow \text{Equivalent.} \end{array} \right.$

Q: $X = \{A \rightarrow B, B \rightarrow C\} \mid Y = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

i) X covers Y $\rightarrow$ (Y $\subseteq$ X)

(इसमें Y की FD में LHS का closure लेते लेते X वाले में से।)

so,

$$A^+ = ABC$$

$$B^+ = BC$$

$$Y = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$$

all 3 covers in these closure forms.

X covers Y. ✓

i) Y covers X:

$$X = \{ A \rightarrow B, B \rightarrow C \}$$

$$A^+ = A B C$$

$$B^+ = BC$$

(X ka closure
Y se hoga)

Y also covers X. ✓

both symbols covers each other & become Equivalent.

$$X \not\neq Y \quad Y \not\neq X$$

$$X \equiv Y$$

✓

ex. Q:- $X = \{ AB \rightarrow CD, B \rightarrow C, C \rightarrow D \}$

$$Y = \{ AB \rightarrow C, AB \rightarrow D, C \rightarrow D \}$$

2) X covers Y:

$$Y = \{ AB \rightarrow C, AB \rightarrow D, C \rightarrow D \}$$

$$AB^+ = ABCD$$

$$C^+ = CD$$

True

1) Y covers X!

$X = \{AB \rightarrow CD, B \rightarrow C, C \rightarrow D\}$

$AB^+ = ABCD$, $B^+ = B$, $C^+ = CD$ (from Y)

$\therefore$ Y not covers X.
It becomes false.

$[X \not\supseteq Y]$ ch.

&

$[X \supseteq Y]$

True

$[Y \supseteq X]$

X, false

✓

35.

Dependency Preserving Decomposition!

Defn!

Let, any table

$[R(ABCD)]$

&

also same F.D.

$[FD \{A \rightarrow B, B \rightarrow C\}]$

Now,

we find closure & then find c.k., &
then we also find hidden dependencies!

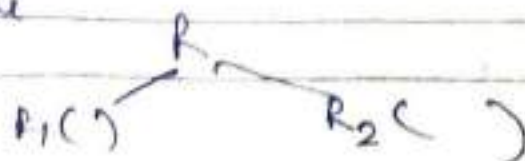
like.

$FD^+ \{A \rightarrow C\}$

by Transitive prop.

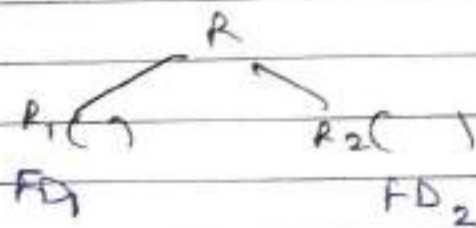
and,

In normalisaⁿ, we do decomposⁿ,
then, we divide



As we distribute attri. of R in R_1 & R_2 ,
 union of attr. of R_1 & R_2 , must be
 equal to the attr. of R .

Now, F.D. also divides, like.



Now,

Check! - Dependency should be preserved,

(check कि FD. lost ना हो रहा हो, यदि हो
 preserve करने करना है।)

ie.

$$\boxed{FD_1 \cup FD_2 = FD^*}$$

(original FD
 is equal.)

Q: Let $R(ABCD)$ with F.D.

$\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B\}$.

R is decomposed into $R_1(AB)$, $R_2(BC)$, $R_3(BCD)$.

$\Rightarrow$	$R_1(AB)$	$R_2(BC)$	$R_3(BCD)$
	$A \rightarrow A$	$B \rightarrow C$ ✓	$B \rightarrow D$ ✓
	$B \rightarrow B$	$C \rightarrow B$ ✓	$D \rightarrow B$ ✓
	$A \rightarrow B$ ✓		
	$B \rightarrow A$ ✗		

Key $\downarrow$
 $B^+ = BCD$
 (don't have A).

union of attr $R_1, R_2 \Delta R_3$ are
Equal to R_1 . So, true. (Right decomposition)

Now

→ check Dependency preservation : →

• $R_1(AB)$, So, It has only A, B attr.

So, whatever Dependency we can make from these 2, make them & write in table.

We don't want Trivial F.D. : →

→ Trivial F.D. : In which "intersec" (n)
is not null.

is, Ex-1 $A \rightarrow A$ has $n \neq \phi$
(common is A).

Ex-2 $AB \rightarrow A$ has $n \neq \phi$
(common is A).

We don't want these bcs, trivial are
by default, true. It's like

$A \rightarrow A$ is definitely true.

So,

→ only take non-trivial F.D. : →
($n = \phi$)

→ Now,

F.D: of $A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$.

$$A^+ = ABCD$$

$$B^+ = BCD$$

$$C^+ = CDB$$

$$D^+ = DB$$

So, acc. to these.

Select from the

~~original~~ non-trivial.

F.D. (which we finally select) from table. =

Now

F.D. which we get from Table: $\rightarrow$

$$A \rightarrow B \quad \checkmark$$

$$B \rightarrow C \quad \checkmark$$

$$C \rightarrow B$$

$$B \rightarrow D$$

$$D \rightarrow B \quad \checkmark$$

Now, check whether original F.D. are possible through them or not.

So, F.D: of $A \rightarrow B, B \rightarrow C, \underline{C \rightarrow D}, D \rightarrow B$.

Now,

$C \rightarrow D$ is not direct.

So, now take 'C' closure.

$$C^+ = C B D$$

$\therefore \underline{C \rightarrow D}$ also preserved.

$\therefore$ all F.D. are preserved. True, Ans.

36.

Dependency Preserving Decomposition Example

Q.2:

Let $R(ABCD)$ with F.D.

$\{AB \rightarrow CD, D \rightarrow A\}$

Decompose $R_1(AD)$ & $R_2(BCD)$.

$R_1(AD)$		$R_2(BCD)$	
$A \rightarrow D$	X	$B \rightarrow CD$	X
$D \rightarrow A$	✓	$C \rightarrow BD$	X
		$D \rightarrow CB$	X
		$BC \rightarrow D$	X
		$BD \rightarrow C$	✓
		$CD \rightarrow B$	X

attributes of $(R_1 \cup R_2)$ make f. $\rightarrow$

FO: $\{AB \rightarrow CD, D \rightarrow A\}$

$$A^+ = A, \quad B^+ = B, \quad C^+ = C, \quad D^+ = DA$$

$$BC^+ = BC, \quad BD^+ = BDAC, \quad CD^+ = CD A$$

So, f.d. we get from table! $\rightarrow$
 $\left\{ \begin{array}{l} D \rightarrow A \\ BD \rightarrow C \end{array} \right\}$

Now, check original f.d.!

$\{AB \rightarrow CD, D \rightarrow A\}$ from table $\rightarrow$

$$AB^+ = AB, \quad D^+ = DA$$

Use 'Card' get

$AB \rightarrow CD$, from our decomposed f.d.

Hence,

Dependency preserved not

Me

370 Joins & Its Types :→

1. Join: When we have to join 2 or more table, to get result.

J			+k		
<u>C.No</u>	<u>Ename</u>	<u>Addr.</u>	<u>Dep.No</u>	<u>Name</u>	<u>S.no</u>
1	Ram	Delhi	D ₁	HR	1
2	Manu	Chd.	D ₂	IT	2
3	Ravi	Chd.	D ₃	MRKT	4
4	Amrit	Delhi	D ₄	Finance	5
5	Nitin	Noida			

'Employee'

'Department'



Select Address from Employee where Ename = 'Manu'
Output → Chd.

↳ In these basic ques, we don't need Join, we easily done those by their separate tables.

Now:-

Q. find Ename of Emp whose working in HR.
Here, HR is in department Table
& Ename is in Employee Table.

Q.

Here, we need Join.

Note

There must be some common attr. in Tables to use Join. Here, E_no is common.

→ Join is Cross product
+
Select Statement (condition).

Its Types! →

- Cross Join (or) Cross product
- ~~Int~~ Natural Join
- Conditional Join
- Equi "
- Self "
- Outer "
 - left Outer.
 - Right "
 - full "

38

"Natural Join" ! →

Join = Cross product + "Andi".

m			n x n		
<u>E.No</u>	<u>E.name</u>	<u>Addr.</u>	<u>Dep No.</u>	<u>Name</u>	<u>Sno.</u>
1	Ram	Delhi	D ₁	HR	1
2	Manu	Chd.	D ₂	IT	2
3	Ravi	Chd.	D ₃	MKT	4
4	Amrit	Delhi			

Employee

Department

→ first find! - Table name

Q: "find the Emp. Names who is working in a department"

Here, we need both, - Table Employees & Department Table.

We do it by "Natural Join".

Common Att - 'E-No.'

So,

whenever we have to equalise (d2/d2) the values of the common Att., then, we always use NATURAL JOIN.

Now, Write Query :-

Select Ename from Emp, Dept where
Emp. Eno = Dept. Eno ;

Output :- Ram, Varun, Arun

'E-No.' must be same in both the tables.
ie (E-No / Emp-No.)

Emp		Dept	
E-No	Ename	Dep No.	E no.
1	Ram	D ₁	1
1	"	D ₂	2
1	"	D ₃	4
2	Varun	D ₁	1
2	"	D ₂	2
2	"	D ₃	4
3	Ravi	D ₁	1
3	"	D ₂	2
3	"	D ₃	4
4	Arun	D ₁	1
4	"	D ₂	2
4	"	D ₃	4

So, finally we get 3 rows.

Emp No	Emp Name	Dept No	Emp No
1	Ram	D ₁	1
2	Varun	D ₂	2
4	Amrit	D ₃	4

Ram
 Varun
 Amrit
 → Ans

Now, how to write Actual Query! →

→ Select Emp name from Emp Natural join Dept;

39. Self Join! →

→ in which the Table is joined to itself.

<u>S-id</u>	<u>C-id</u>	Since
S ₁	C ₁	2016
S ₂	C ₂	2014
S ₁	C ₂	2017

Student Course

Study

Q: find student id who is enrolled

→ Now, we do Self Join.

- SQL query always start from - 'from'
(sequence)

Query! →

Select (from study as T₁, study as T₂;

's' → means Cross product.

• So, here we make same Table 2 times
2 name as T₁ & T₂.

S ₁	C ₁	2016
S ₂	C ₂	2017
S ₁	C ₂	2017

T₂ (m)

<u>S-id</u>	<u>C-id</u>	Since
S ₁	C ₁	2016
S ₂	C ₂	2017
S ₁	C ₂	2017

T₁ (n)

m × n

T ₁		T ₂	
S ₁	C ₁	S ₁	C ₁
S ₁	C ₁	S ₂	C ₂
S ₁	C ₁	S ₁	C ₂
S ₂	C ₂	S ₁	C ₁
C ₂	C ₂	S ₂	C ₂
S ₂	C ₂	S ₁	C ₂
S ₁	C ₂	S ₁	C ₁
S ₁	C ₂	S ₂	C ₂
S ₁	C ₂	S ₁	C ₂

Cond'n! →

1 student (S-id)
need 2 course (C-id)

$< >$ → different (not equal to).

SM

Page

Now, Complete Query!

Select T1.Sid from study as T₁, study as T₂
where

T₁.Sid = T₂.Sid (Student name same)

and

T₁.Cid $< >$ T₂.Cid; (can't diff)

So, final output Table!

S ₁	C ₁	S ₁	C ₂
S ₁	C ₂	S ₁	C ₁

Now, we want S-id from this Table.

But T₁ & T₂ both have S-id.

So, we can use any T₁.Sid or T₂.Sid

Ans - S₁

40. EQUI - Join : (=)

(कोई भी ऑपरेटर नहीं है)

<u>E.No</u>	<u>Sname</u>	<u>Add.</u>
1	Ram	Delhi
2	Karan	Chd.
3	Ravi	Chd.
4	Ankur	Delhi

Employee

<u>Dep.No</u>	<u>Loco</u>	<u>E.no</u>
D ₁	Delhi	1
D ₂	Pune	2
D ₃	Patna	4

Department
Dept.

$f \Delta f = f$
 $f \Delta T = f$
 $T \Delta T = T$

$f \text{ or } f = f$
 $f \text{ or } T = T$
 $T \text{ or } T = T$

Note
 Day

84

Q: Find the Emp name who worked in a department having loca" same as their address?

Ans: By just seeing the 2 Table: →
Ram

Q: Query: →

Select Sname from Emp, Dept where

Emp. S-no = Dept. S-no , common
 and

Emp. Add = Dept. loca" ;

Emp.

Dept.

Delhi	1	Ram	D ₁	Delhi	1
"	1	"	D ₂	pune	2
"	1	"	D ₃	patna	4
Chd	2	Harun	D ₁	Delhi	1
"	2	"	D ₂	pune	2
"	2	"	D ₃	patna	4
"	3	Ravi	D ₁	Delhi	1
"	3	"	D ₂	pune	2
"	3	"	D ₃	patna	4
Delhi	4	Anurag	D ₁	Delhi	1
"	4	"	D ₂	pune	2
"	4	"	D ₃	patna	4

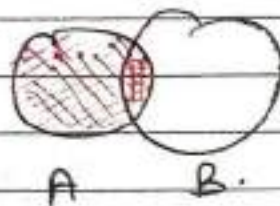
Ram

Ans:

Note: 1) Table ke common cell ke column me
 message (⇒) lagana h 2) ~~2~~ kisi koi 2 cell
 ke beech mein (⇒) lagana sakhte h.

41) Left Outer Join: →

- It gives the matching rows & the rows which are in left table but not in the right table.



Natural Join
+
something else

Emp			Dept		
Empno	Sname	Deptno	Deptno	Dname	Loc
E ₁	Umesh	D ₁	D ₁	IT	Delhi
E ₂	Amit	D ₂	D ₂	HR	Mysr
E ₃	Ravi	D ₁	D ₃	Finance	Pune
E ₄	Nitin	—			

Query! →

Select empno, sname, dname, loc from
emp left outer join dept on

(emp.deptno = dept.deptno)

cond'n of
Natural join
(common attr. of
table 1 & 2 on 1.)

Relational Database always shows output in Table form.

50%

89%

89%

output Table: 1

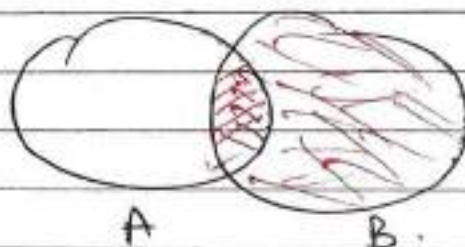
Emp-no	E-name	d-name	Loc ⁿ
E ₁	Usman	IT	Delhi
E ₂	Anant	HR	Hyd.
E ₃	Ravi	IT	Delhi
E ₄	Nitin	-	-

→ (Left side of
Right Row
30/1/11)

92%

Right Outer Join: →

It gives the matching rows & the rows which are in Right Table but not in left Table.



(Emp)			(Dept)		
Emp-no	E-name	Dept-no	Dept-No	D-name	Loc ⁿ
E ₁	Usman	D ₁	D ₁	IT	Delhi
E ₂	Anant	D ₂	D ₂	HR	Hyd
E ₃	Ravi	D ₃	D ₃	Finance	Rural
			D ₄	Testing	Meida

Query:-

Select emp-no, e-name, d-name, loc from
emp Right Outer join dept on
(emp.dept-no = dept-dept-no.)

natural join.

→ Output Table ! →

Emp-no	E-name	D-name	Loc ⁿ
E ₁	Uday	IT	Delhi
E ₂	Amrit	HR	Hyd.
E ₃	Ravi	Finance	Pune
-	-	Testing	Noida

Right Table
all rows.

Full-Outer Join ! - Take the union of left outer join & Right outer join rows.

$((\text{Left O.J.}) \cup (\text{Right O.J.}))$

(43) Relational Algebra ! → (1970).

(procedural lang. or formal Query lang.) also.

→ have to do these things in Query ! →

- 1.) What to do.
- 2.) How to do.

→ SQL language base is Relⁿal Algebra.

(Collection of mathematical types).

Relⁿal algebra → SQL → No SQL

In today's time.

*1 "operators"

Basic operator

- projection (π)
- Selection (σ)
- Cross product ($\times$)
- Union ($\cup$)
- Rename (ρ)
- Set Difference ($-$)

Derived operator.

- Join ($\Join$)
- intersect ($\cap$)
- $(x \cap y) = x - (x - y)$
- Division ($/, \div$)

2) If we understand Relational Algebra very clearly, then we don't face any problem in understanding SQL.

44- Projection in Relational algebra: $\rightarrow$

- Projection (π): $\rightarrow$

π func. is to retrieve the data.

Roll no.	Name	Age
1	A	20
2	B	21
3	A	19
1	!	!

$\times$ (Student)

- Query: retrieve the roll no. from table (Student)

Q. $\pi_{rollno, name}(Student)$

Roll no.
1
2
3

→ Query! $\pi_{rollno, name}(Student)$

We fetch the 2 colⁿ (rollno & name) from the student (table).

Output →

Rollno	Name
1	A
2	B
3	A

Note: It only gives unique Answer.

Q. # Query! - $\pi_{name}(Student)$

Name
A
B

(Only name here)

→ "project", we use it in last & before this we use other operators.

45. Selection in Relⁿal algebra → (CR.A.).
 σ (Sigma).

#

Rollno	Name	Age
1	A	20
2	B	21
3	A	19

→ It works on Tuples (rows).

→ First, It found rows.

π operator.

* Query: Retrieve the name of student whose Roll no = '2'

→ $\sigma_{\text{rollno}='2'}(\text{Student}) \rightarrow$

2	B	21
---	---	----

↓
Final.

→ $\pi_{\text{name}}(\sigma_{\text{rollno}='2'}(\text{Student})) \rightarrow$

B

π always last it operate $\rightarrow$!

* We can also find 2 at a time.
Ex: $\pi_{\text{name, Age}}(\sigma_{\text{rollno}='2'}(\text{Student})) \rightarrow$

B	21
---	----

Note: Projection always works on Columns.

Selection (σ) always works on Rows. (Tuples).

46

Cross / Cartesian product in Relational algebra

a)

A	B	C
1	2	3
2	1	4

R_1

C	D	E
3	4	5
2	1	2

R_2

To join, we must have to cross product.

1.

$$3 + 3 = 6$$

(m+n)

$$2 \times 2 = 4$$

(m x n)

A	B	C	C	D	E
1	2	3	3	4	5
1	2	3	2	1	2
2	1	4	3	4	5
2	1	4	2	1	2

 $\rightarrow (R_1 \times R_2)$
 (R_1)
 (R_2)

→ We need a common attr. to join 2 tables. (Here, c)

(42)

Set Difference in R. A ! $\rightarrow$

$$(A - B) = A \text{ but not } B.$$

$$= A \cap B'$$

Ex:

 $\{1, 2, 3\}$
 S_1
 $\{3, 4\}$
 S_2

$$S_1 - S_2 = \{1, 2\}$$

$$S_2 - S_1 = \{4\}$$

*

It is not Commutative ! $\rightarrow$

i.e.

$$A - B \neq B - A$$

Condition:-

- 1.) No. of column must be same in no.
- 2.) Domain of every column must be same.
(data of same type). (Ex - Numeric) $(1+A) \times$

Ex 1

Roll no	Name
1	A
2	B
3	C

(Student)

Emp. No.	Name
2	E
1	A

(Employee)

→ (Student) - (Employee)

(1 A EE Staff)

→

Roll no	Name
2	B
3	C

→ (By default, first table in a column not set)

Q:- Find the name of a person who is a student but not employee.

→ (Student - Employee)

→ How to write?

$(\pi_{\text{Name}}(\text{Student}) - \pi_{\text{Name}}(\text{Employee}))$

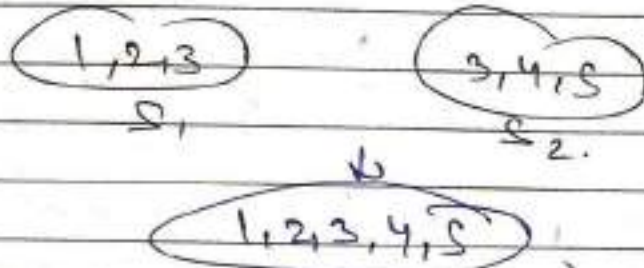
→ Output:-

Name
B
C

$((A, B, C) - (E, A))$

48) Union Opero in R.A. $\rightarrow$

Same as in Set. ($\cup$).



Ex: $\rightarrow$

Roll no.	Name
1	A
2	B
3	C

(Student)

Emp. No.	Name
7	D
1	A

(Employee)

Condⁿ :

1. No. of colⁿ must be same in no.
2. Domain of every colⁿ must be same.

(Student) $\cup$ (Employee)

Roll no.	Name.
1	A
2	B
3	C
7	D

$\rightarrow$ (left side ^{Table} and A).

Long Trick!

2) $\pi_{name} (Student) \cup \pi_{name} (Employee)$

Name
A
B
C
E

→ (Student also & who is Employee also or both.)

Note 1

Roll no	name
1	A
2	B
3	C

numeric

name	Emp Id.
E	2
A	1

alphabet

X

By not same domain.
(ye, default order pick that!)

So, here, not Union operaⁿ applies.
(NULL)

49. Division Operaⁿ in R. A. :- →

→ Divisⁿ operator is actually a Derived operator.
(we derived them, from normal operators).

$\div$ (X, $\neg$, π , σ)

→ we use these

Query: Retrieve Sid of Students who enrolled in every course.

2) Every (TA) all test of Divis' method.

Sid	Cid
S ₁	C ₁
S ₂	C ₁
S ₁	C ₂
S ₃	C ₂

'Enrolled'

Cid
C ₁
C ₂

'Course'

Note of Divis' Method: →

$A(x, y) / B(y) = \text{it results 'x' values}$
for that there should be tuple $\langle x, y \rangle$
for every y value of Relⁿ B.

Here,

$$\pi_{sid}(\sigma_{cid=C_1}(Enrolled)) = S_1$$

ie,
S₁ enrolled in every course (C₁ & C₂).

How it happens?

Let, all students are enrolled in every course.

(for this, we do the cross product)

$$(\pi_{cid}(Enrolled)) \times (\pi_{cid}(Course))$$

S ₁	x	C ₁
S ₂		C ₂
S ₃		

S ₁	C ₁	S ₁	C ₁
S ₁	C ₂	S ₂	C ₁
S ₂	C ₁	S ₁	C ₂
S ₂	C ₂	S ₃	C ₂
S ₃	C ₁		
S ₃	C ₂		

(Enrolled)

Now,

(We subtract actual Scenario i.e. (Actual Table) from this cross product. we get S₂, S₃ (who don't enrolled in all course)

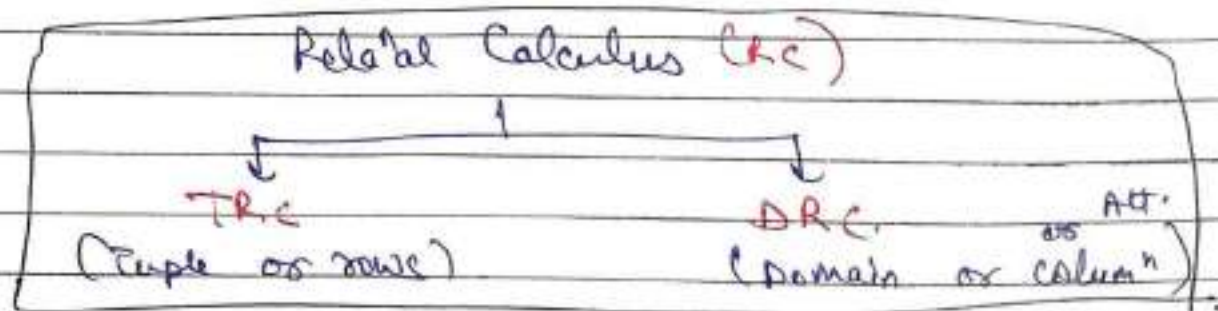
* Now, final! $\rightarrow$ (Query).

$$\pi_{sid}(\text{Enrolled}) - (\pi_{sid}((\pi_{sid}(\text{Enrolled})) \times \pi_{cid}(\text{course})) - \text{Enrolled})$$

S₁
S₂ - S₂
S₃ - S₃
S₃

$\Rightarrow$ S₁ ✓

So, Tuple Calculus in DBMS $\rightarrow$



$\Rightarrow$ Tuple Relational Calculus is a non-procedural query lang. (only tells what to do & not that how to do). unlike Relational algebra.

2. In Tuple Calculus, a query is expressed as

$$\{t \mid P(t)\}$$

Where, t = Resulting tuples,
 $P(t)$ = known as predicate & these are the conditions that are used to fetch t . Thus, it generates set of all tuples t , such that predicate $P(t)$ is true for t .

* Operations! →

→ $P(t)$ may have various conditions logically combined with OR ($\vee$), AND ($\wedge$), NOT ($\neg$).

→ Atomic func's! - (We use one variable here, ex - (Supply Ids. from a supply table) → only 1 condition)

→ It also uses quantifiers!

• $\exists t \in r_1 (Q(t))$ = "there exists" a tuple in t in r_1 or such that predicate $Q(t)$ is true.

• $\forall t \in r_1 (Q(t))$ = $Q(t)$ is true "for all" tuples in r_1 or.

Unsafe expression $\rightarrow$ (U.E.)
Supplier (Table).

• $\{ s.name \mid \neg \text{Supplier}(s) \}$
(not sign)

Here, it means
that, all the supplier name who are not
in the Supplier Table.

So, here, ans is ∞ . (So, ∞ loop it that will not stop)

Unsafe Express

(These are not in the Relational algebra)

• Both TRC & RA have same expressive powers.

unsafe exp. $\rightarrow$ (U.E.)
not stop
in TRC.

(the query which we can
write in RA, can also
write in TRC & even
in DRC.)

But, SQL has more powers than these.
SQL has more extra func

Examples: $\rightarrow$

Q-1: Write a query in SQL to display
Name of Suppliers?

Q-2: Write a query in SQL to display
Name of Parts whose color is Red?

Q-3: Write a query in SQL to display SID of suppliers whose name is 'Varun' & address is 'Chandigarh'.

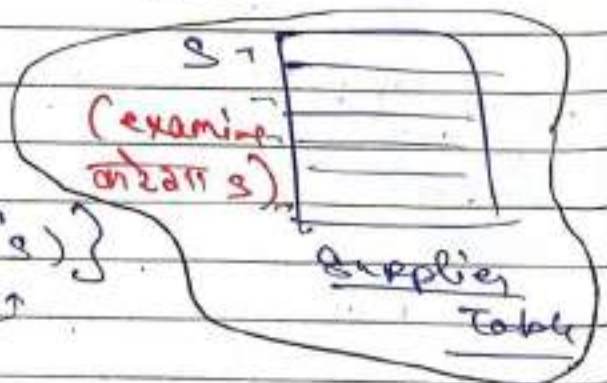
Q-4: Write a query in SQL to display SID of suppliers who supplied some parts?

Q-5: Write a query in SQL to display Sname of suppliers who supplied some parts?

Q-6: Write a query in SQL to display Sname of suppliers who supplied some red color parts?

(1)

$\{ S.Sname \mid Suppliers(S) \}$



(2)

$\{ p.Pname \mid Parts(p) \wedge p.color = 'Red' \}$

(3) $\{ S.SID \mid Suppliers(S) \wedge S.name = 'Varun' \wedge S.add = 'Chandigarh' \}$

(4) Here, it extract ans. from the Catalog Table
 $\{ C.SID \mid Catalog(C) \}$

Sid to Catalog Table to Sid (Supplier Table).
But, Sname only in Supplier Table.

(103)

(5) Here, we need 2 Table $\swarrow$ Supplier
Catalog,

2
final ans. from Supplier Table.
(There exists 3.)

$\{ S.Sname \mid Supplier(S) \wedge \exists c (Catalog(c)) \wedge S.Sid = c.Sid \}$

Here,

S ~~is~~ $\rightarrow$ free variable
 $c \rightarrow$ bounded variable.

(निम्नलिखित साधन से
3 या 4 नमूने)

(6) Here, we need 3 Table $\swarrow$ Supplier
Catalog.
parts.

But,

ans from Supplier Table.

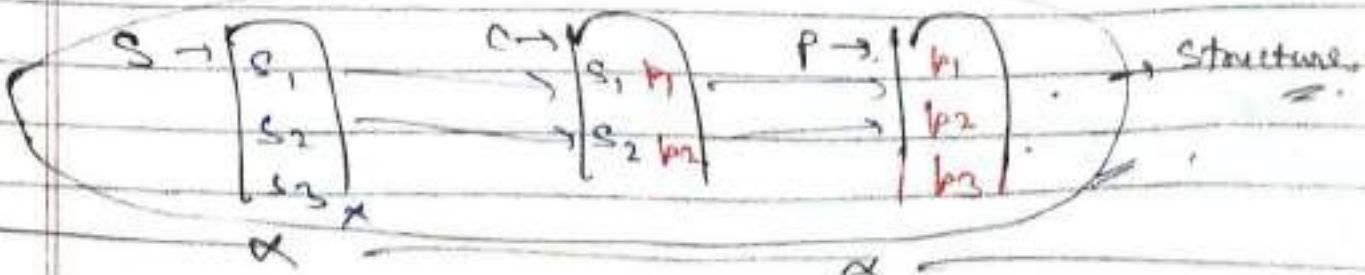
$\{ S.Sname \mid Supplier(S) \wedge \exists c (Catalog(c)) \wedge \exists p (parts(p)) \wedge S.Sid = c.Sid \wedge c.pid = p.pid \wedge p.color = 'red' \}$

S variable for Supplier Table.

c " " " Catalog

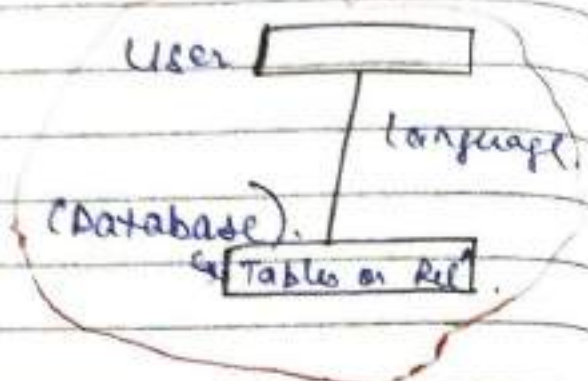
p " " " parts

S $\rightarrow$ free variable
c, p $\rightarrow$ bounded ".



ST- Intro to SQL: → (Structured Query language)

(1970)



→ EF codd gave theory of store & fetching of data. Relational Model:

By the help of Relational algebra & TR: (Tuple Relational Calculus)

→ IBM converts this concept of database of EF codd into the Structure Query language (SQL).

→ Before, SEQUEL → Simple English Query language

then,

SQL → Structure Query lang.

→ then, Oracle & Microsoft, etc. comes into this field of databases.

→ Now, No SQL is comes into market.

10% data → Structure
90% data → Unstructure (Spant, mogads)

*) Properties:-

General purpose! → applicability on multiple places. Ex. C/C++, but
Domain specific! → only use in any particular field. Ex! -
SQL is in only Relational Database.
(When we have to store & fetch data from the Relations (Tables), they only use SQL). and except this, there is no general use of SQL.

- 1.) SQL is a domain-specific language.
- 2.) SQL is a declarative language.

→ declarative lang → What to do.

↳ procedural lang → what to do
Δ how to do

↳ (There is procedure that how to do)

Later on

PL SQL → procedural language.

3) DDL, DML, DCL, TCL

These are diff. commands in SQL to create, insert, fetch, control data. etc.

4.) Keys & constraints. (Ex - P.K, A.K, C.Key).

↓
(primary key (P.K.) constraint,
f.p.
Not, NULL, default)

aggregate → समग्र, family, शरीर गुण, संग्रह करना.
Date: _____
Page: _____

Exist / Not Exist

5.) operators (like, between, In, Not in,
Conditional).

we use these operators intelligently to
use Query.

[Ex:-] IRCTC :- we query on this app
by APIs (Application programming interface).

Train no., seat no.,
IRCTC based on structured Data.

(Train की Info. Tables के form में ही
show होती है).

6.) clauses (distinct, order by, group by,
from having).
also use "majorly" use this in query

7.) aggregate func's :- (max, average, count,
min, sum, 5)

8.) Joins & Nested Query :-

9.) PL SQL (Triggers, func, cursor, procedure)
same as in C++.

⑩ What to do :-

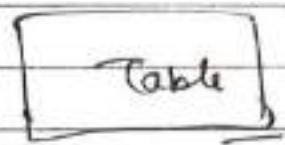
In libraries of Oracle or like SQL
Server, it is already defined that what
select do, & what other commands 'do'.
so, we don't need to give procedure that
how to do.

Q2) All Types of SQL Commands :-

→ DDL Commands :- (Data Definition Language).

→ Deals with Schema (Structure / Table).

- Create
- Alter (change Table Specifications).
- Drop (Remove Table)
- Truncate (Remove Data)
- Rename



* DML Commands :- (Data Manipulation Language).

→ If we have to change anything in data, then we use DML. (Manipulation).

- Select
- Insert
- Update
- Delete



* DCL Commands :- (Data Control Language).

→ who has control over the data.
→ who is authorised on that data.

- Grant (give permission)
- Revoke (take back that permission)

* TCL Commands :- (Transaction control lang.)

→ Mainly used in Transactions.

Ex: Shopping, Bill payment - online

- Commit (If transacⁿ is commit (done), then save ^{data})
- Roll back. (If transacⁿ fails).
- Save point. (Ex: we can save after 20-30% of transacⁿ).

Ex: In Downloading & Buffering,
we have save points. Ex: If
downloads fails on 90%, then it starts
from '0' again as from 90%.)
This is save point.

Notation To execute these commands, we
can use Oracle, My SQL, Server, etc.

Constraints : → we mostly use these
constraints in SQL & DML.

- Primary key (for uniqueness)
- Foreign key ("reference")
- check
- Unique
- Default
- Not Null (mandatory *)

→ Constraints are rules which we made
for store the data.

S3 Create Table in SQL with
execution : →

→ In SQL Commands, very first command is Create Table.

Now, we use Oracle Syntax. (In other like my SQL, SQL server, there is only little diff.)

```
Create table <table-name>
(
  Column1 name datatype,
  Column2 name datatype,
  Column3 name datatype
);
desc table-name;
```

if no space
Ex! abc-xyz.

if describe

Ex!

```
Create table emp
(
  id int,
  name varchar2(20),
  Salary number(10)
);
desc emp;
```

Ex: (2^{bits} fixed)

→ fixed type of datatype

variable — (can be increased)

54. Alter Command (DDL) in SQL :-

Use → (बनी हुई चीज में अगर कुछ change करना है तो।)

Ex:

Student	
int	name varchar(20)

→ already diff

- Add or Remove column/s.
- modify datatype
- modify datatype length
- Add constraints
- Remove constraints
- Rename colⁿ/table

Idname	Address

← student

④ Alter table student Add address varchar(30);

+ multiple colⁿs.

⇒ Modify datatype:-

Ex:- from int to varchar

* Create table emp

```
(
  id int,
  name varchar(10)
);
```

```
alter table emp add address varchar(10);
```

```
desc emp1;
```

```
alter table emp drop column address;
```

```
" " " modify id varchar(30);
```

```
" " " rename column id to reg-no;
```

```
" " " rename to emp1;
```

```
alter table emp1 add primary key (reg-no);
```

(SS) D/B Alter & Update : →

(*) Alter! only change in structure (DDL) & not in data.

Ex!

Emp			
ID	name	Salary	Email
1	A	10k	!
2	B	20k	!
3	C	30k	!

} can only change in table's structure.

→ ID → varchar

→ name varchar (10)

↓
20

→ ID → Eid.

→ Emp → Emp-detail

coln
(name change)
(table name)

(*) Update! → can only change in data (DML) & not in structure.

~	~	~
~	~	~
~	~	~

can only change in data

```
update Emp
Set Salary = Salary * 2;
```

→ for all student

```
update Emp
Set Salary = Salary * 2
Where id = 1;
```

→ for only student of id = 1.

Q6) D/B Delete, Drop & Truncate!

Q1) Delete! -
(DML).

Syntax! ->

Delete from table name

Ex1) Delete from Student, + All rows deleted

Desc: Student;

Student	
ID	name
1	A
2	B
3	C

X

X

X

Note: But here, also flexibility, we can apply condiⁿ for delete some particular rows.

Ex2) Delete from Student
Where id = 1;

=> It is a slower process! - bcz, it also creates log.

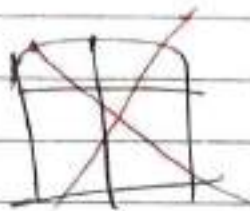
Log! -> Entire file completely delete होने से पहले, Computer जो temp file generate करे, to restore data, उसी log check करे।

Ex3) on delete, our data comes into recycle bin. (It is log file).

Rollback;
 (possible, but it creates log).

Q. Drop! " ~~delete~~ Delete the complete table at once.
 (DDL)

Drop table student;
Desc student;



→ no object found.
 (all delete).

Rollback; ✗ (not possible).

Q. Truncate! +
 (DDL)

Truncate student;
Desc student;

All rows delete
 at once.

Id	name

→ faster process.

→ Rollback; ✗ (but, no log).

Q7. Constraints in SQL! →

→ Constraints means (conditions) restriction.

Ex: In gmail! (2 account ids can't be same)

anurag@gmail.com
 anurag23@gmail.com

✗ (not present)
 ✓ (present) ✓

1. Condiⁿ → that new mail-id must be unique.

2. Condiⁿ → length of password must be 6 digit, Capital letter etc.

Note:- We apply conditions on Attributes (column).

and the data which fulfill these condiⁿ, only comes into the column.

1.) Unique → Ex:- Mobile no (no duplicate can exist in that m.no. colⁿ).

2.) Not Null →

(We can't skip that colⁿ).

i.e., * - mandatory. (value deni hai q3311)

* 3.) Primary key = Unique + Not Null
Ex:- Reg. no.

4.) Check →

age int
-10 x

check (age > 18)

i.e.,
→ 10 x
→ 20 ✓

(q33n particular domain m. 2) (1)
Uses Not Null (1)

Domain - fixed.

Note

5.1. Foreign key :

6.1. Default :

[Salary int default 10K]

if we don't fill anything, they default Salary col^m take value 10K

we can use multiple constraint in a col^m also.

58 SQL Queries & Sub-Queries? →
(from Beginning to end).

Q1:

Emp.	S-id	S-name	Dept	Salary
1	Ram	HR	10K	
2	Anita	MRKT	20K	
3	Ravi	HR	30K	
4	Nith	MRKT	40K	
5	Harun	IT	50K	

(i) Write a SQL Query to display maximum Salary from Emp table.

(ii) Write a SQL Query to display Employee name, who is taking max^m Salary.

[Note] Aggregate func !~ for max, count, average,

< > → not equal to, =, ≠

SM
Date: _____
Page: _____

c) Select max (Salary) from Emp;

Output → 50000

cii) use here, take help of Nested Query or Sub-Query (Query inside a Query).

Select E-name from Emp where
Salary = (Select max (Salary) from Emp);

Outer Query.

Inner Query

Output → Name

→ Inner Query execute first 1 time

40k

50k

Outer	Inner
10k	50k
20k	50k
30k	50k
40k	50k
50k	50k

59

ciii) Write a SQL query to display 2nd highest Salary from Emp table?

civ) → Write a SQL query to display Employee name who is taking 2nd highest Salary?

cvi)

Logic: Highest Salary - remove 1
Rest Salary - find highest

It is 2nd highest in table

max ← 40k

(50k)

Inner

50k - x

Ques) Query:-

Select max (Salary) from emp where
Salary <> (Select max (Salary) from emp);

90k

✓

Sol.

Ques) { = , when we have to compare only
with 1 value.
in , when we have to compare with
multiple values.

Ex:-)

2 = 2

{ 2 = 1, 3, 2, 4, 5 (Wrong way)
2 in (1, 3, 2, 4, 5) (True)

Query:- ✓ only add name in Ques)

Select E-name from emp where
Salary = (Select max (Salary) from emp where
Salary <> (Select max (Salary) from emp));

Output addition.

α

α

(60) Ques) Write a Query to display all the
dept names along with no. of Emps.
working in that ?

Output:-

HR	2
MARKT.	2
IT.	1

Ex:-) In a university, find
how many students are
there in diff. branches.

AI	101
CSE	100

Note-1 Here we use a special funcⁿ.

[group by] clause

(forms grp of dept).

HR	2
HR	2
MRKT	2
MRKT	2
IT	1

Note-2 Condⁿ of group by → जहाँ अर्थ. की मी. में
एक group by की मी. में use कर रहे हैं,
सिर्फ वही select — के साथ लिख सकते हैं

If,
we have to write other attr. in select
other than the attr. of group by funcⁿ,
then we have to use aggregate func^s.

• Query:-

Select dept from emp group by dept;

→ Output →

HR
HR
MRKT
MRKT
IT

→ Same.

Note-3 - Count (dept) or Count (*).

→ Query:-

→ aggregate func.

Select dept, Count (*) from emp group by dept;

Output →

HR - 2
MRKT - 2
IT - 1

61. (ii) Write a Query to display all the dept names where no. of Emps are less than 2.

HR 2
MKT 2
IT 1

Note! - 'where' clause works on complete Table. but now we group by this table. Hence, group by & where are independent. not help each other.
Hence,
we use 'having'.

Query:-

Select dept from Emp Group by dept
having Count (*) < 2;

Output → IT.

Q ⇒. If they ask of Employee name in this Query, then → (Answer → output)

Query →

Select E-name from Emp where dept In
(Select dept from Emp group by dept
having Count (*) < 2);

↳ Answer.

HR In
MKT
IT

Q2

Ques) Write a Query to display highest Salary department wise and name of emp who is taking that Salary.

Note! A SQL query always starts from 'from'
 Select from (table name)

Note! Select max (Y) from group by X;
 Here, It is Right, bco we use aggregate func 'max' with Y
 i.e. max(Y) & if we don't use max, then we can only use X, bco group by X. (Emp)

Query:-

Select Ename from Emp where Salary >= (Select max (Salary) from Emp group by dept);

Output -> Ravi, Nitin, Harun.

Steps →

	Steps	Outer	Inner
HR	HR - 30,000	30k	30k 40k 50k
HR		40k	
MRKT	MRKT - 40,000	40k	
MRKT	IT - 50,000	50k	
IT		50k	

(Note: In the original image, the values 30k, 40k, and 50k in the 'Outer' column are crossed out with an 'X' for the first three rows, and the 'Inner' column is circled in red.)

* means All attributes.

Simple in Query.

63. Use of IN and Not IN

1 = (1, 2, 3, 4, 5)

X (wrong way).

1 = 2

false. (but, Right way).

[Emp]

<u>E_id</u>	E_name	Address
1	Ravi	Chd
2	Varun	Delhi
3	Nitin	Pune
4	Robin	Bangalore
5	Ammy	Chd.

[Project]

<u>P_id</u>	P_name	Loca ⁿ
P ₁	IoT	Bangalore
P ₂	Big Data	Delhi
P ₃	Retail	Mumbai
P ₄	Android	Hyderabad

Q:- Detail of Emp whose address is either Delhi or Chd or Pune;

Query:-

Select * from Emp where Address = 'Delhi';

Output:- [2 | Varun | Delhi]

⇒ But, we have 3 cities. So,

Query:-

Select * from Emp where Address In ('Delhi', 'Pune', 'Chd');

Output:-

1	Ravi	Chd
2	Varun	Delhi
3	Nitin	Pune
5	Ammy	Chd.

Chd ✓
Delhi ✓
Pune ✓
Bangalore ✗
Chd. ✓

last query → Query (Query)

Now, NOT IN :- (means not included)

Query :-

Select * from Emp whose
Address Not In ('Delhi', 'Pune', 'Chd');

Output :-

4	Robin	Bangalore
---	-------	-----------

Chd x

Delhi x

Pune x

Bangalore ✓ ch

Chd x

64. Use of IN & Not IN in Subquery :-
(Same 2 tables as before).

Query :- find the name of Emps who
are working on a project?

→ Suggestion :- first make Dummy Tables

→ Here, we need both 2 tables

Both tables have common - EID
So, we compare by taking EID.

→ Nested → Bottom up → नीचे से ऊपर

Meaning - पहले ऊपर वाली query & then नीचे वाली

Inner Query in Nested Query - Runs 1 time.

(SM)

Rules

first

(123)

* Query:-

Select Ename from Emp where Eid
In (Select Distinct (eid) from Project);

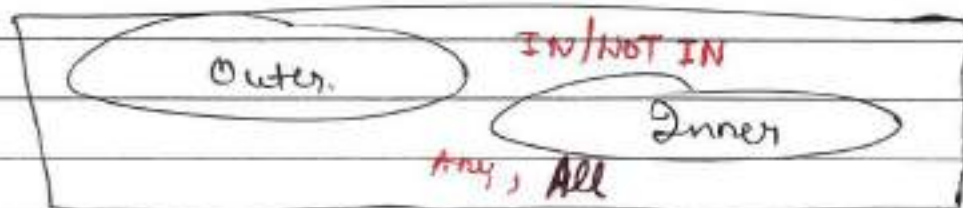
Output:-
Ravi
Mithu
Robin
Ammy

Inner Query

$\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}$ $\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$

Q5. EXIST & NOT EXIST Subqueries :-

→ we use these in Correlated Nested Query.



→ In nested query - we use IN/NOT IN

In correlated nested query - we use EXIST/NOT EXIST

Query → find the detail of Emp who is
working on at least one Project?
(Same 2 Tables)

Not'n In nested Query → Inner Query executes first,
then compare its output with Outer Query.
In Correlated Nested Query → the one row of Outer
Query is compared with all rows of Inner Query.
ie, Top to Down Approach.

Null → means value is not available.
Empty (SM Sign)

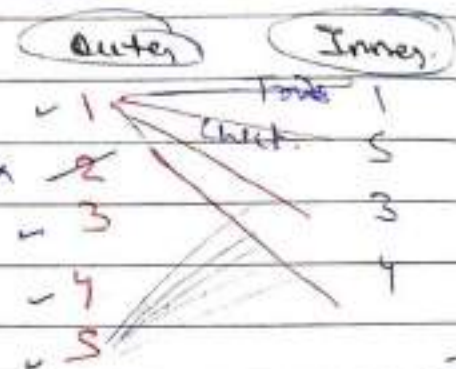
Q Query: 7

Select * from Emp where Eid
Exists (Select Eid from Project where
 Emp. Eid = project. Eid);

Not! Exist / not Exist gives True or False.

Output:

	Emp	Dept
1	Ravi	Chd
3	Nitin	Pure
4	Potkin	Bang.
5	Ammy	Chd.



66. Aggregate Funcs in SQL: →

max, min, Count, Avg, Sum.

(Total no. of values in a Table).

Emp

Eid	Enam	Dept	Salary
1	Ram	HR	10k
2	Amit	MKT	20k
3	Ravi	HR	30k
4	Nitin	MKT	30k
5	Harun	IT	50k
6	Sandy	Testing	Null.

Query for max. Salary: 7.

Select max(Salary) from Emp;

Output → 50k

at

→ If 30k repeats 2 times, then it also gives output 2 times.

4 Same for min

→ Select min (Salary) from Emp;
Output → 10k.

* COUNT:-

Count (*) means no. of rows in the Table.

→ Select Count (*) from Emp;
Output → 6. 6 rows.

→ Select Count (Salary) from Emp;
Output → 5 (bc, one value is Null)

* Distinct:-

→ Select Distinct (Count (Salary)) from Emp;
Output → 4. (bc, 30k is 2 times).

② SUM:-

Select Sum (Salary) from Emp;
Output → 140k. (Sum of all salary)

→ Select Distinct (Sum (Salary)) from Emp;
Output → 110k. (30k repeats)

④ AVG :-

$$\begin{aligned} \text{Avg (Salary)} &= \frac{\text{Sum (Salary)}}{\text{Count (Salary)}} \\ &= \frac{140k}{5} \\ &= 28k \end{aligned}$$

→ Select Avg (Salary) from Emp;
[Output: → 28k.]

→ Select Distinct (Avg (Salary)) from Emp;
[Output: → 28,500.]

$$\begin{aligned} \text{Distinct (Avg (Salary))} &= \frac{\text{Distinct (Sum (Salary))}}{\text{Distinct (Count (Salary))}} \\ &= \frac{110k}{4} \\ &= 27,500 \end{aligned}$$

⑥ Correlated Subquery in SQL :- (Synchronized Query).

→ It is a subquery that uses values from Outer Query.

→ Top to Down Approach :- (Outer to Inner)

→ for one row of Outer Table it compare with every row of Inner Table.



→ Returns True/False

Emp

Eid	name	address
1	A	Delhi
2	B	pune
3	A	Chd.
4	B	Delhi
5	C	pune
6	D	Mumbai
7	E	Hyd.

Dept

Att.

D-id	D-name	E-id
D ₁	HR	1
D ₂	IT	2
D ₃	MKT	3
D ₄	Testing	4

Query: Find all Employee detail who work in a department.

→ True Eid of all Employee Detail & (G.O.T.)

→ Query:-

Select * from Emp where
exists (Select * from dept where
dept eid = Emp eid)

Output:-

Eid	name	Address.
1	A	Delhi
2	B	pune
3	A	Chd
4	B	Delhi

Emp	Dept	Att.
1	1	2, 3, 5
2	1	3, 5
3	1	2, 3, 5
4	1	2, 3, 5

False

68

D/B Joins, Nested Subquery & Correlated Subquery:-

- 1 Nested → Bottom up (अगर में अगर)
- 1 Correlated → Top down Approach (अगर में अगर)
- 2 Joins → Cross product + Condi

Emp

<u>E-id</u>	name
1	A
2	B
3	C
4	D
5	E

Dept.

<u>Dept no.</u>	name	<u>E-id</u>
D ₁	IT	1
D ₂	HR	2
D ₃	MARKT	3

Q1: Detail of all emp who works in any dept.

(Here, we need both 2 Tables)

→ Nested Subquery →

Select * from emp where E-id in
(Select e-id from dept);

Output →

1	A
2	B
3	C

→ Correlated Subquery →

Select * from emp where exists
(Select id from dept where
emp.e-id = dept.e-id);

E-id is
Common
in both
Table

Output:

1	A
2	B
3	C

James

1	= 1	True
2	= 1	
3	= 1	

James

1	< C
2	= C
3	= S

False

1. If 1st Table $\rightarrow$ m rows
2. 2nd Table $\rightarrow$ n rows

then, Total Comparison $\rightarrow m \times n$

JOINS ! $\rightarrow$

Cross product + Condi

It creates a new Table with $(m \times n)$ rows.

then,

check Condi $\rightarrow$ emp.eid = dept.eid

Note

Joins is faster than Correlated Subquery bcz it made a table of rows $(m \times n)$ at once. While in Correlated, we again & again compare one by one row of Outer Table with all Rows of Inner table. So, it takes time. Δ But, Joins take more space (bcz big Table of $(m \times n)$ rows).

69.

Find N^{th} Highest Salary using SQL $\rightarrow$

Emp.	
Id	Salary
1	10k
2	20k
3	20k
4	30k
5	40k
6	50k

1 Highest Salary! →

Select max (Salary) from Emp;

Output → 30K

2 2nd Highest Salary! →

(Nested Query)

Select Max (Salary) from Emp where
Salary Not In (Select max (Salary) from
Emp);

Output → 40K

Now

How to recognise Correlated Subquery?

जैसे जैसे outer Query की कोई नई नई
value होता Inner use करता है।

* How to find Nth Highest Salary! →

Query:-

[Best method - learn it]

select id, Salary from Emp e₁ where
N-1 = (Select Count (Distinct Salary) from
Emp e₂ where
e₂. Salary > e₁. Salary);

here

we make 2 slice of Emp i.e. e₁ & e₂

(Dummies)

ii, Correlated Subquery

7. Ex. on 1st case :-

E ₁	
id	Salary
1	10k
2	20k
3	20k
4	30k
5	40k
6	50k

E ₂	
id	Salary
1	10k > 10k (Salary)
2	20k > 10k count = 1
3	20k > 10k count = 1 (dis)
4	30k > 10k count = 2
5	40k > 10k c = 3
6	50k > 10k c = 4

⇒ why we make 2 (E₁ & E₂) :-
bc, employee use use 2 times. (E₁ & E₂)

→ If we don't create E₁ & E₂, then
E₂. Salary > E₁. Salary

→ It means, employee Salary > employee Salary
E₁ & E₂ → No Mean, X

SLV	10k	> 20k	f
<u>X</u>	20k	> 20k	f
2nd	20k	> 20k	f
row	30k	> 20k	count = 1
	40k	> 20k	count = 2
	50k	> 20k	count = 3

count = 3

Ex. let's we have to find 4th highest, then
N = 3, 4 = [3]

Select id, Salary

3 =

When we get 3 as count from this inner query, then, our output comes.

And, we get, Count = 3
 from the 2nd row. bcr,
 from 1st Row we get Count = 4.
 Hence, Ans is data of 2nd Row

Output:

ID	Salary
2	20k

20k is our 4th highest Salary.

(
 50k - 1st
 40k - 2nd
 30k - 3rd
 20k - 4th
)

Ex. 11 for 3rd highest Salary!
 $N=3$, $N-1 = \underline{2}$

Hence,

for which Row we get Count = 2,
 that row will be our output.

10k	> 30	F	, count = 1 , count = 2
20k	> 30	F	
20k	> 30	F	
30k	> 30	F	
40k	> 30	T	
50k	> 30	T	

4th Row is our output

Output:

4, 30k

30k is our 3rd highest Salary.

3 Imp Questions on SQL basic concepts

Q1. You need to display last name of employees who have 'A' as 2nd character in their names. Which SQL statement display the required result?

- a) Select last-name from emp where last-name like ' _ A %' ✓
- b) " last name = ' _ A %' ✗
- c) " name like ' _ A %' ✗
- d) " like ' _ A %' ✗

Note: Questions like, 2nd letter same, Salary be of only S nos, like that, based on 'Like' command.

% → Any value.
_ → fixed a place for a value.

Ex: i) % A % Anything A A B A (not all are A)

ii) 2nd letter → _ A % (one pos is fixed)

iii) 4th character must be A → _ _ _ A %

iv) 2nd last letter must be A → % _ A %

2. A Command to remove relⁿ from SQL database → (Table).

- A) Delete table < table name >
 B) Drop table < ————— > 3. Ans
 C) Erase table < ————— >
 D) Alter table < ————— >

2) DDL Commands deal with Schema (Table structure).

create, drop, alter $\rightarrow$ DDL Commands.

→ Alter table → to change anything in table

! Delete table, Erase table. ✗ (Even not a command)

3. In the following, scheme R is R (a, b)

Q1: Select * from R

Q 2: (Select * from R) Intersect (Select * from R)

Q3: Select distinct * from R

- a) Q_1, Q_2, Q_3 produce same result
b) only Q_1, Q_2 ————
c) only Q_2, Q_3 ————
d) Q_1, Q_2, Q_3 produce diff. result.

2.1.1

T
1
2
3

1	1
2	2
3	3
3	4

$$\begin{array}{r} 921 \\ 1 \\ 2 \\ 3 \\ \hline 3 \end{array}$$

Dummy Table
Aorich Dekh lo

→ 1, 2, 3

02203

Is there
a table of
contents

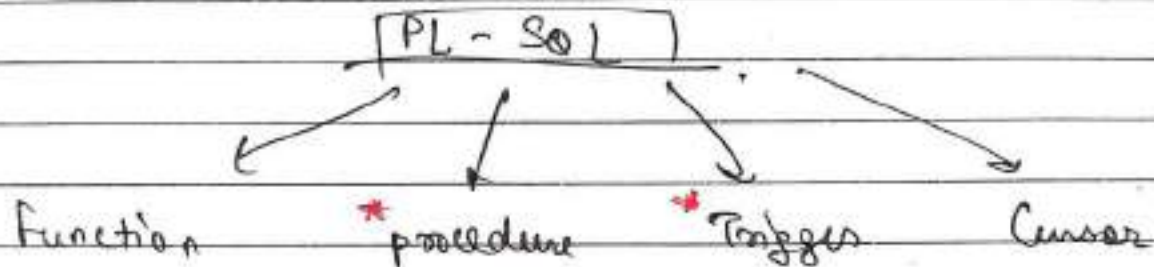
Q1.

PL-SQL

(procedural - SQL)

SQL → Declarative in nature (only 'What to do').

PL-SQL → procedural (1.) What to do ✓
(2.) How to do. ✓
(programming language like C++).



In SQL → (write a query)

In PL-SQL → (write a program, or write a PL-SQL Code)

(program write next Q1).

Q2 Block Code has 3 parts: →

```

declare
a int
b int
c int
begin
a := 10;
b := 20;
c := a+b;
end;
  
```

declare	a int;
Executable Code	begin
Exception handling (error).	end

means
(if manually, we have to raise any errors). Like $\frac{x}{0}$, we define it in Excepⁿ handling.

CPU always performs in RAM. CPU - fast.
CPU never works on hard disk. Hard Disk - slow.

72. TRANSACTION CONCURRENCY :->

Transac: It is a set of operations used to perform a logical unit of work.

(जैसे मैं मैं change करे रहे, Database में डिले, जैसे मैं मैं Database में Read करे रहे, that is also a part of transac.)

Ex-1

1) When we withdraw our money from ATM, then we have to perform a ^{set} of operaⁿ.
These set of operaⁿs called as Transaction.

[Work - Withdraw Money.]

A transaction generally represent change in database.

Database Transactions has 2 operations -> Read & Write. (Commit) - we also use this.

[Read is the access of Database.
Write is change in database, we made.]

2) When we read or access any data from HDD (hard drives), then it comes to RAM where we perform operaⁿs.

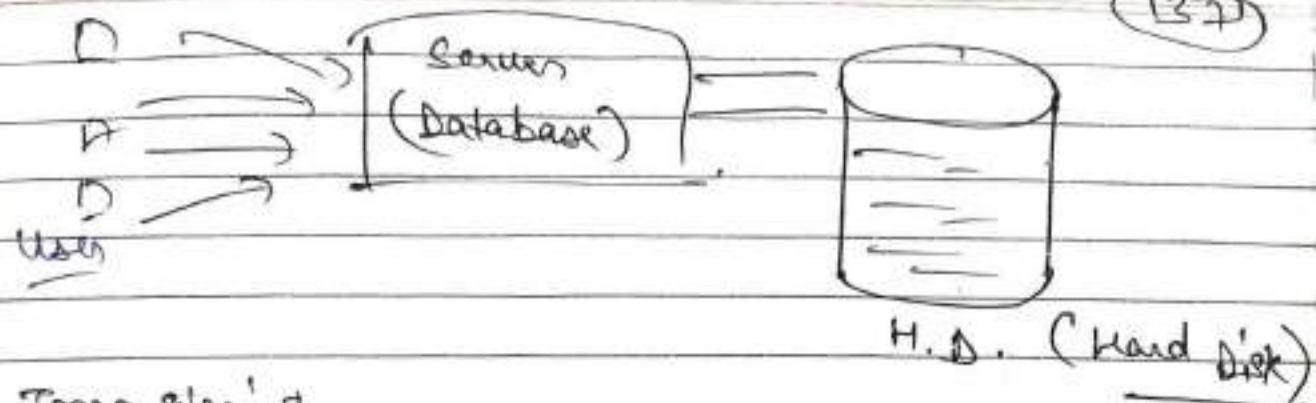
Commit -> Whatever change we made, save that permanent in Hard Disk.
(Update data in HDD)

In C++ for assignment, =
In PLSQL, :=

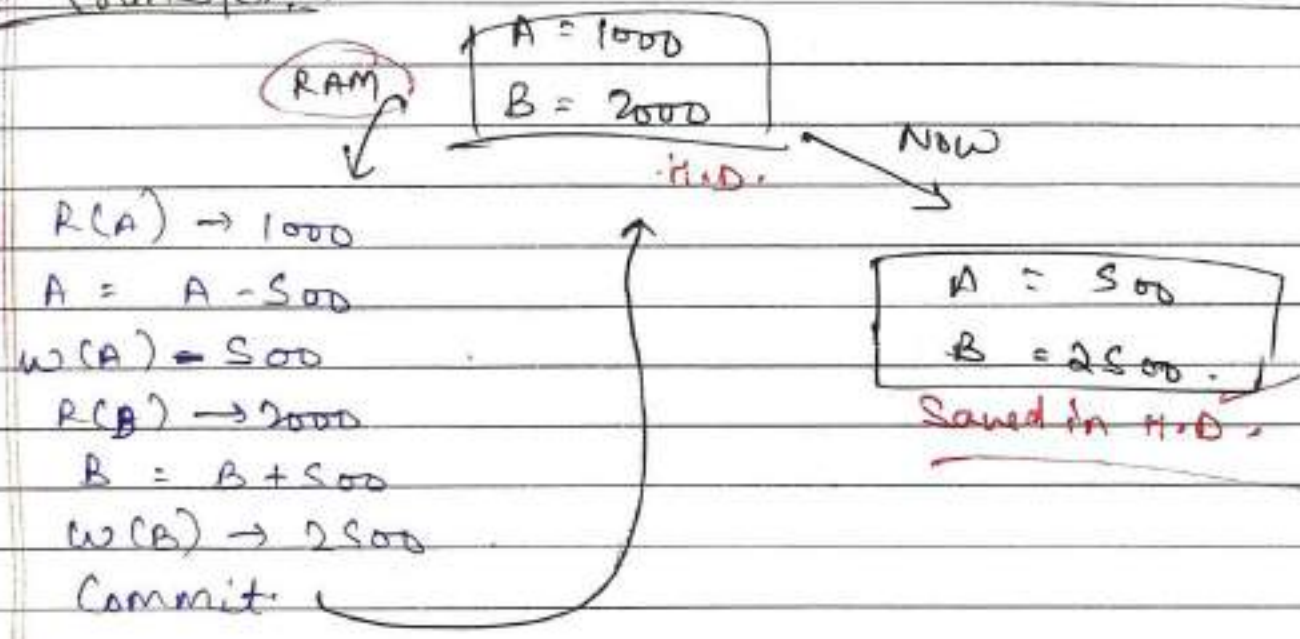
sim Data Type

C++, ==
PLSQL, =

(137)



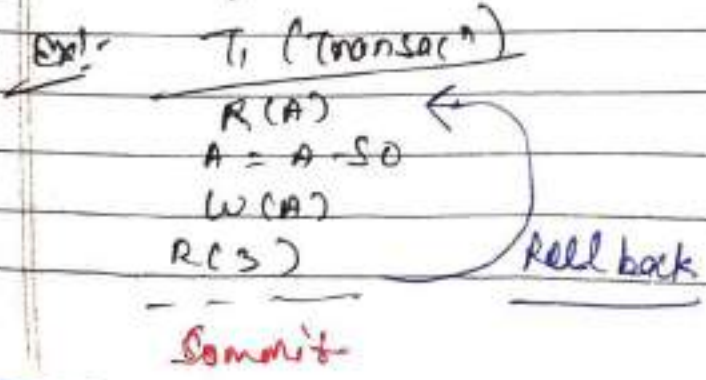
* Transaction :-



(73) ACID properties of a Transaction :-

- A $\rightarrow$ Atomicity
 - C $\rightarrow$ Consistency
 - I $\rightarrow$ Isolation
 - D $\rightarrow$ Durability
- } At back End

* Atomicity $\rightarrow$ Either all or None.



(If we commit at 4th step and if fail at 5th, then Roll Back)

[या तो सगळी operation execute होती, commit होत नाही. वरना पुनः से Roll back होत नाही]

Note:- A failed Transaction cannot be resumed.
A failed Transaction will always restart.

Consistency :-

Before trans. start And,
after the trans. completed,

Sum of money should be same.

Ex:-

$\begin{cases} A = 2000 \\ B = 3000 \end{cases}$
5000

A $\xrightarrow{1000}$ B

T₁

R(A) 2000
A = A - 1000
W(A) 1000
R(B) 3000
B = B + 1000
W(B) 4000
Commit.

$\begin{cases} A = 1000 \\ B = 4000 \end{cases}$
5000

Sum is same

Isolation :-

T ₁	T ₂	T ₃
R(A)		
	R(A)	
		R(A)

parallel Transac's.

→ We try to convert a parallel schedule into a serial schedule. (Conceptually)

CPU speed is in MIPS (million instructions per second).

Q Hard Disk $\rightarrow$ 10/20 Instrⁿ per second.

Ans CPU never compatible with HD for execution.

(139)

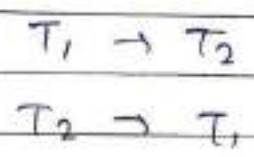
$\Rightarrow$ Yes,

Serial Schedule is always consistent

* Parallel



* Schedule



* Durability ! $\rightarrow$

Whatever changes we made, they must be permanent. i.e.

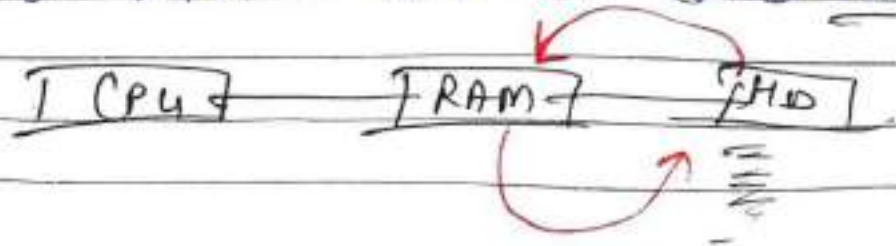
(update for lifetime until we again update the data).

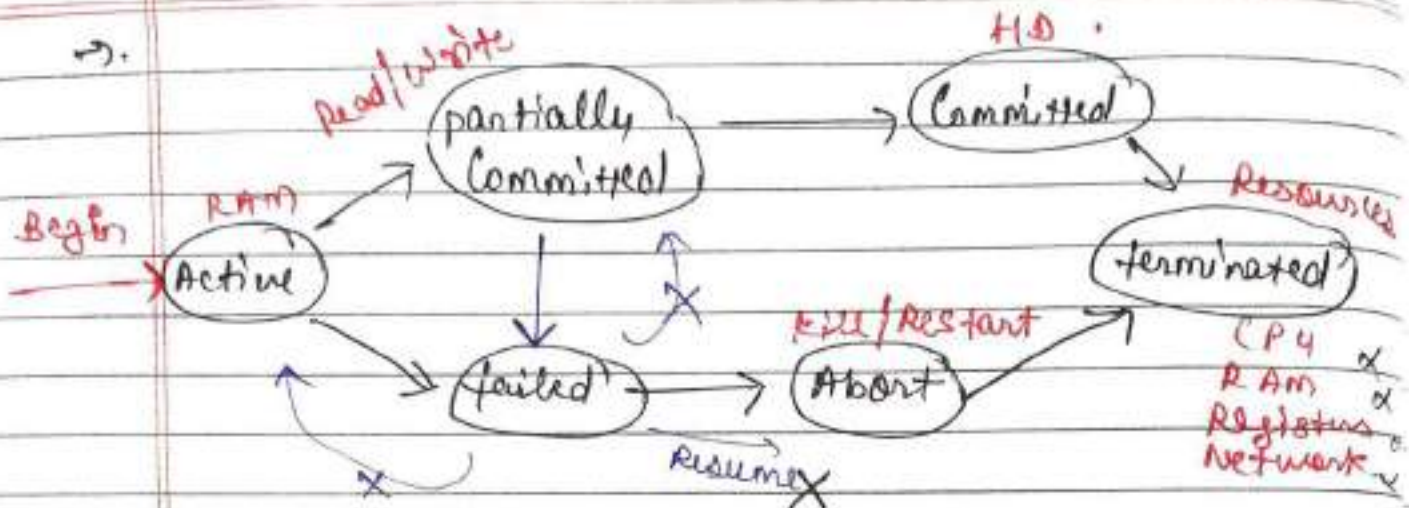
$\Rightarrow$ That's why, we save data in HD for durability.

(74) Transaction States ! $\rightarrow$

$\Rightarrow$ At now, Transacⁿ is in inactive state. And as we start executing it, it comes into Active state.

* In terms of a.s. ! $\rightarrow$ When we write a program in C++ & save it. Now, the program is in Hard Disk in idle state. But, when we start executing or compiling, it comes into RAM that we called Active STATE.





→ partially Committed! →

All ops are done except Commit.
ie. If N operation.
then

$(N-1)$ is done.

All ops are stored in local memory / shared memory until now.

→ Committed! -

changes are now saved to hard disk.

→ terminated! →

Here, we deallocate our resources.
ie. free all resources.

Resources are Limited

Now, they move to anyone else.

→ failed! → power failure, switch damage, etc.

(failed either from Active or partially committed state)

→ Abort! → Rollback the oper's & Restart the oper's.

(25) SCHEDULE! → (Serial vs parallel Schedule)

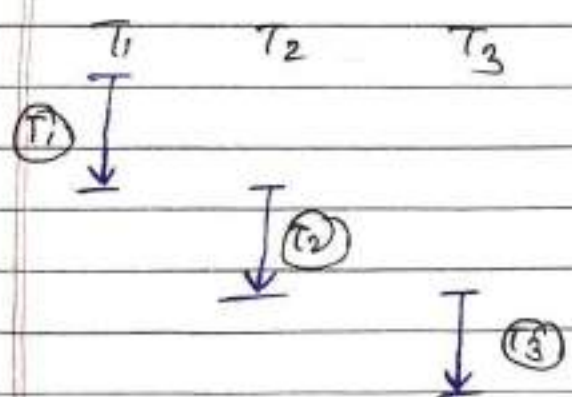
→ Schedule! → It is chronological execution sequence of multiple transactions.

T_1 T_2 T_3 ————— T_n

↳ What is the sequence that these transac. are getting executed, that's called Schedule.

Serial Schedule! → Until a transacⁿ gets completed, no other trans. can interfere.

All transac's executed by a ~~serial~~ serial sequence.



Advantage →
→ Consistent (bcoz, no one can interfere).

Disadvantage →
(When T_1 gets executed, T_2 & T_3 are in waiting.)
→ Performance degrade.

Parallel Schedule! →

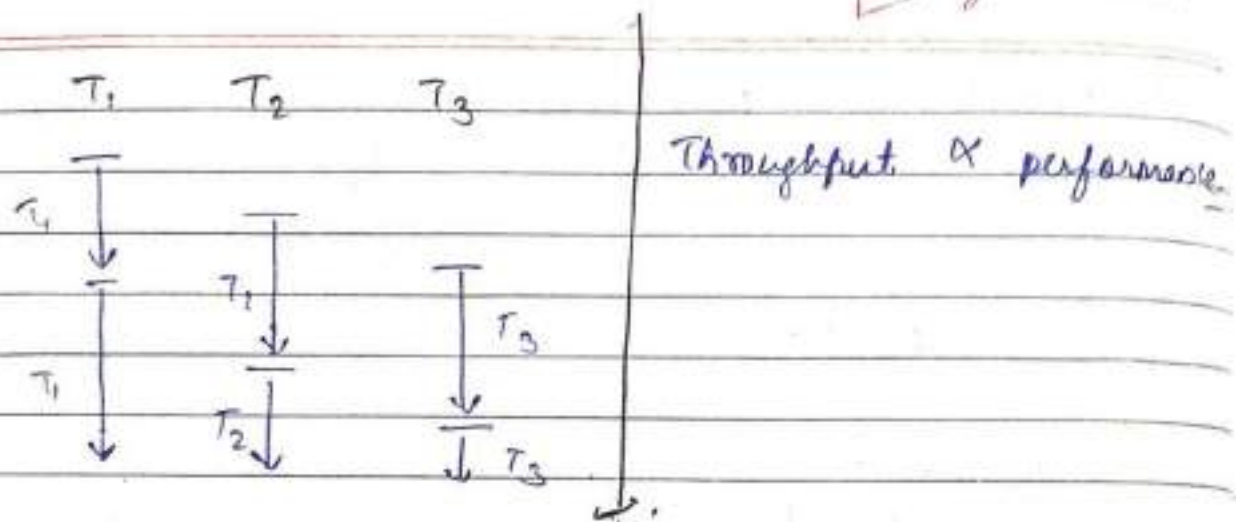
→ We can switch to multiple transac's at a time. i.e.,

→ Multiple transacⁿ can execute at a same time.

Ex!? Banking Online System! → Multiple users can use at a time.

Throughput $\rightarrow$ No. of transac^{ns} executed / time

SM
Date: _____
Page: _____



$\rightarrow$ Advantage: $\rightarrow$

$\rightarrow$ performance Increased i.e., (Throughput is high).

$\rightarrow$ Disadvantage: $\rightarrow$

$\rightarrow$ problem may occur & (Inconsistent)

*) Nowadays,

(Parallel Schedule is more performed.)

$\hookrightarrow$ better good performance in less time.

X ————— X

7b) Types of problems in Concurrency $\rightarrow$

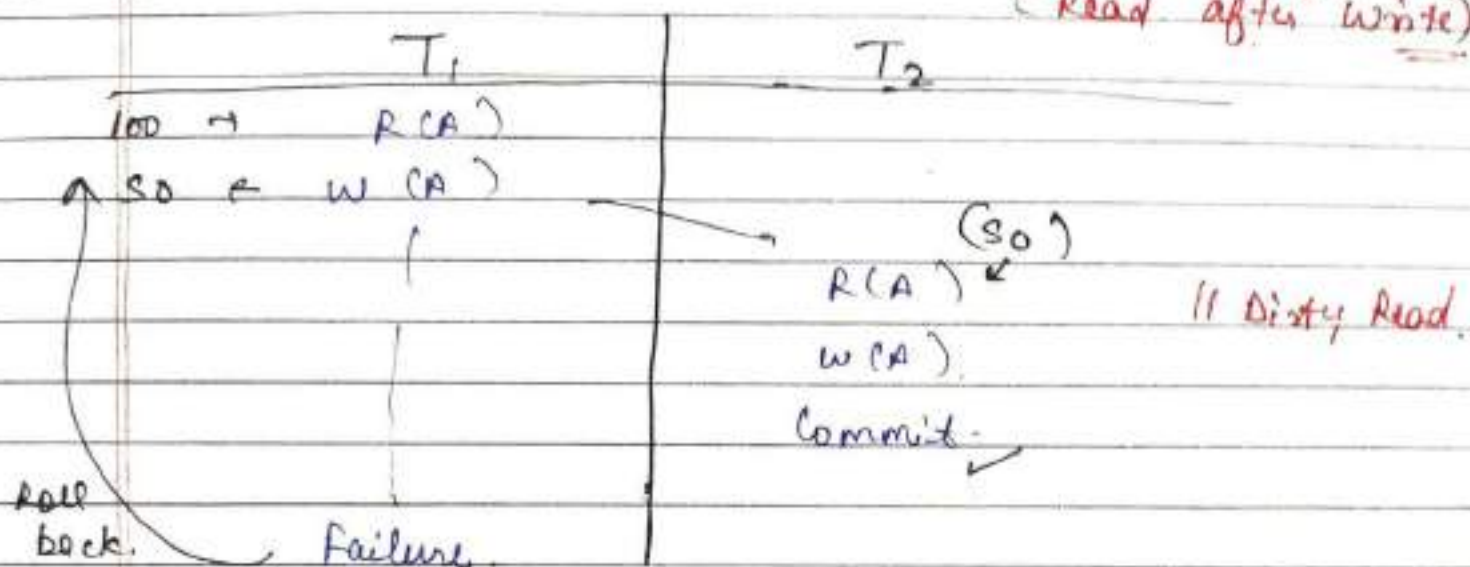
$\rightarrow$ Concurrency, means when multiple trans. executed at a same time i.e., Parallel schedule.

$\rightarrow$ (Mean, No problem is in Serial Schedule. There are problems only in Parallel Schedule.)

- 1) Dirty Read
- 2) Incorrect Summary

- 3.) Lost update
- 4.) Unrepeatable Read
- 5.) phantom Read.

1.) Dirty Read! $\rightarrow$ or Uncommitted Read or RAW.
(Read after write)



Hence, when T_1 gets failed. Then, how T_2 can use the A-50. So, Dirty Read.

2.) Incorrect Summary problem! $\rightarrow$ (7.)
It occurs mostly when a transacⁿ start & T_2 comes & start performing its aggregate funcⁿs.
(Sum, average)
then,
we get incorrect value of sum, average etc.

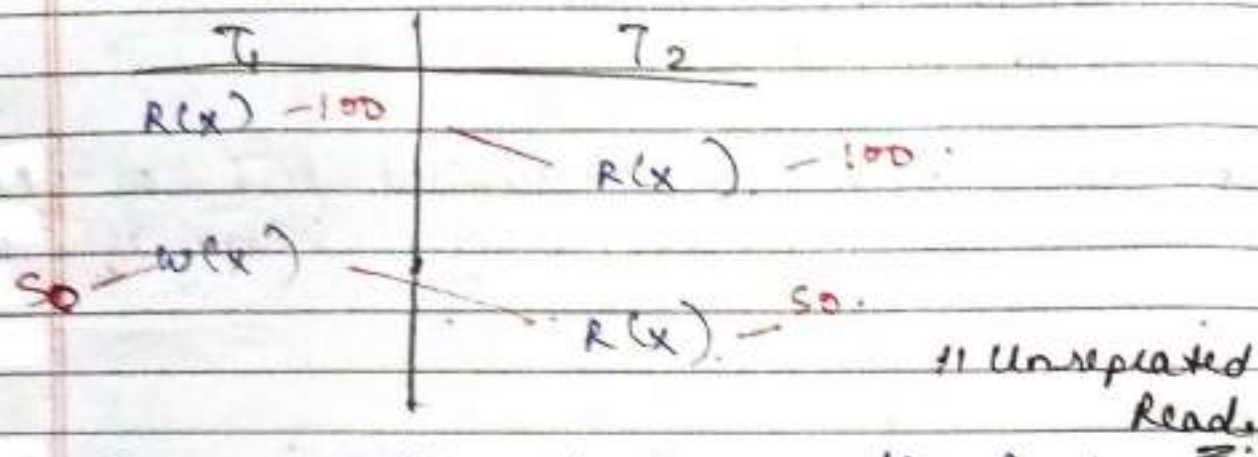
3.) Lost update! $\rightarrow$

$\Rightarrow$ जो पहले (T_1) ने changes कीये। उस changes को (T_2) ने ओवर update कर दिया। & पुराने वाले changes हमारे lost हो गये। i.e. update किया हुआ lost हो गया।

Ex: T_1 (id-10) / T_2 (id-1)
Slices / Slices

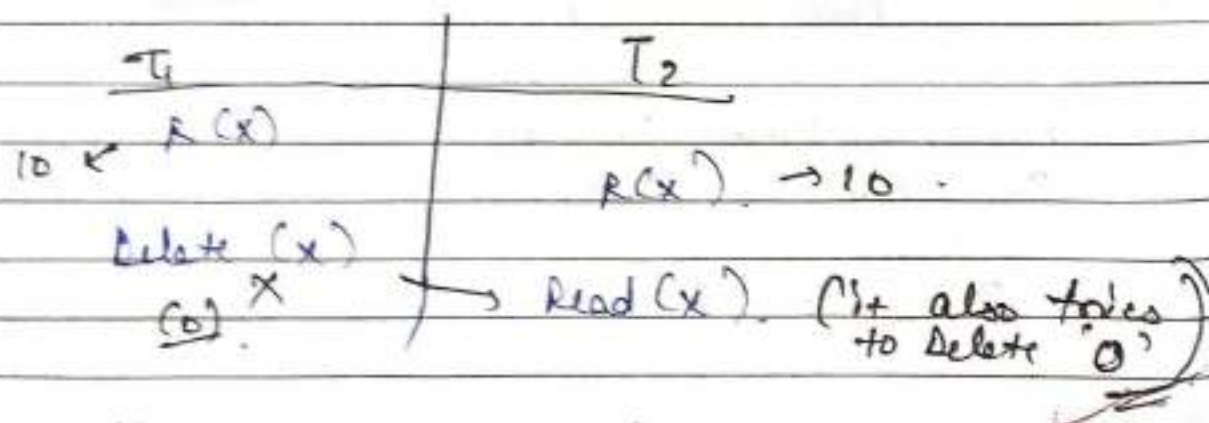
$\Rightarrow$ & we finally get 5 likes on the same product instead of 10 likes.

4.) Unrepeatable Read ! →



→ T₂ got diff values on diff. Read.
(100 & 50).
So, it is also a problem.

5.) phantom Read ! →



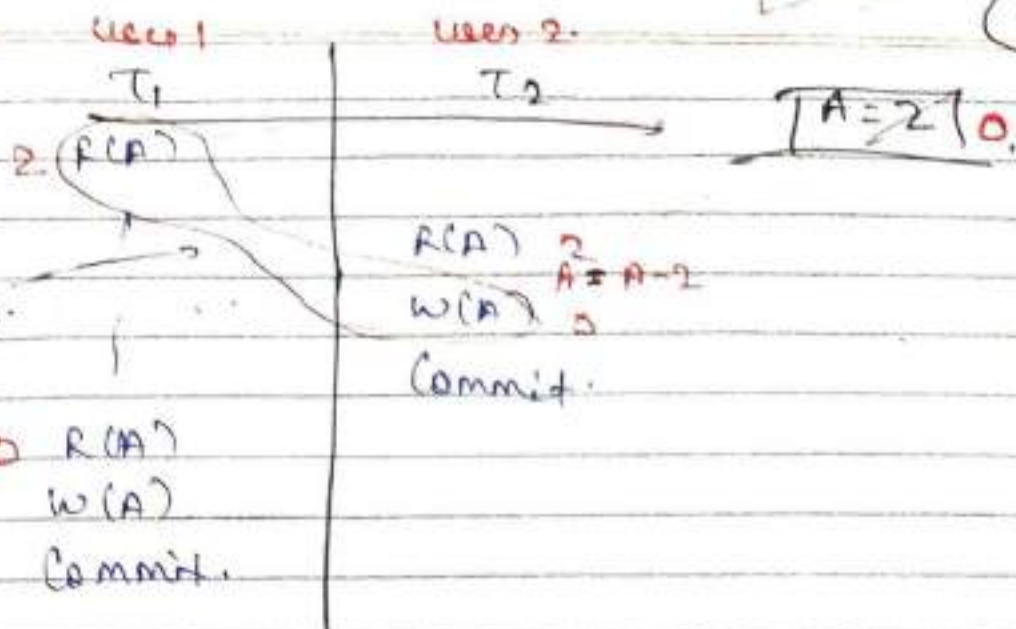
7.7 Read-Write Conflict (or) Unrepeatable Read Problem ! →

4 cases are there →

Same Data.	
R(A)	R(A)
R(A)	W(A)
W(A)	R(A)
W(A)	W(A)

Problem

Ex- (1)



Ex - TRCTC

Let User 2 reserve both the seats. Then,
 $A = A - 2 = 0$

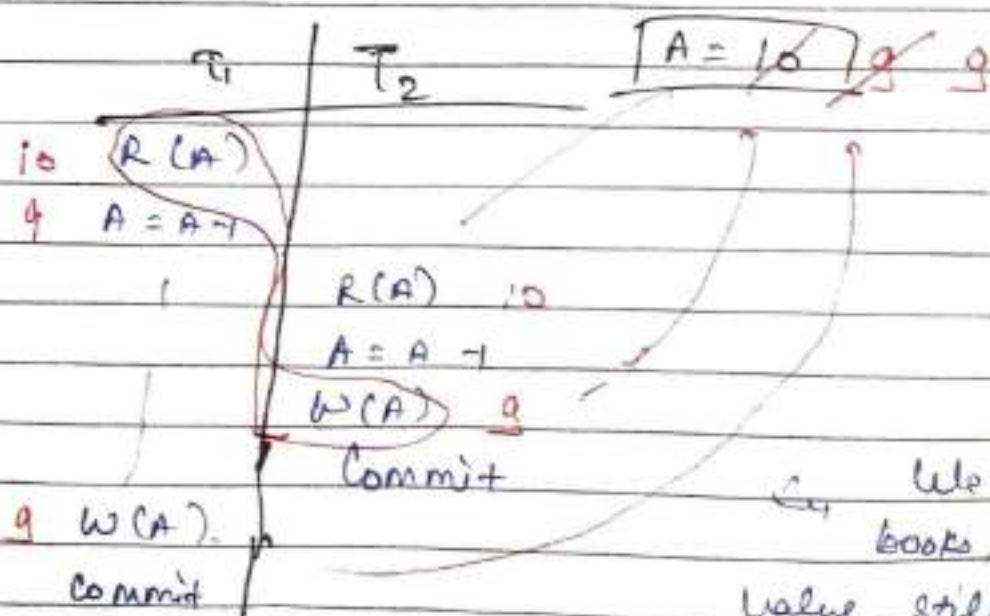
User 1 remained in thinking to reserve or not. & when he sees again,
 Now,

Seats becomes '0'

So,

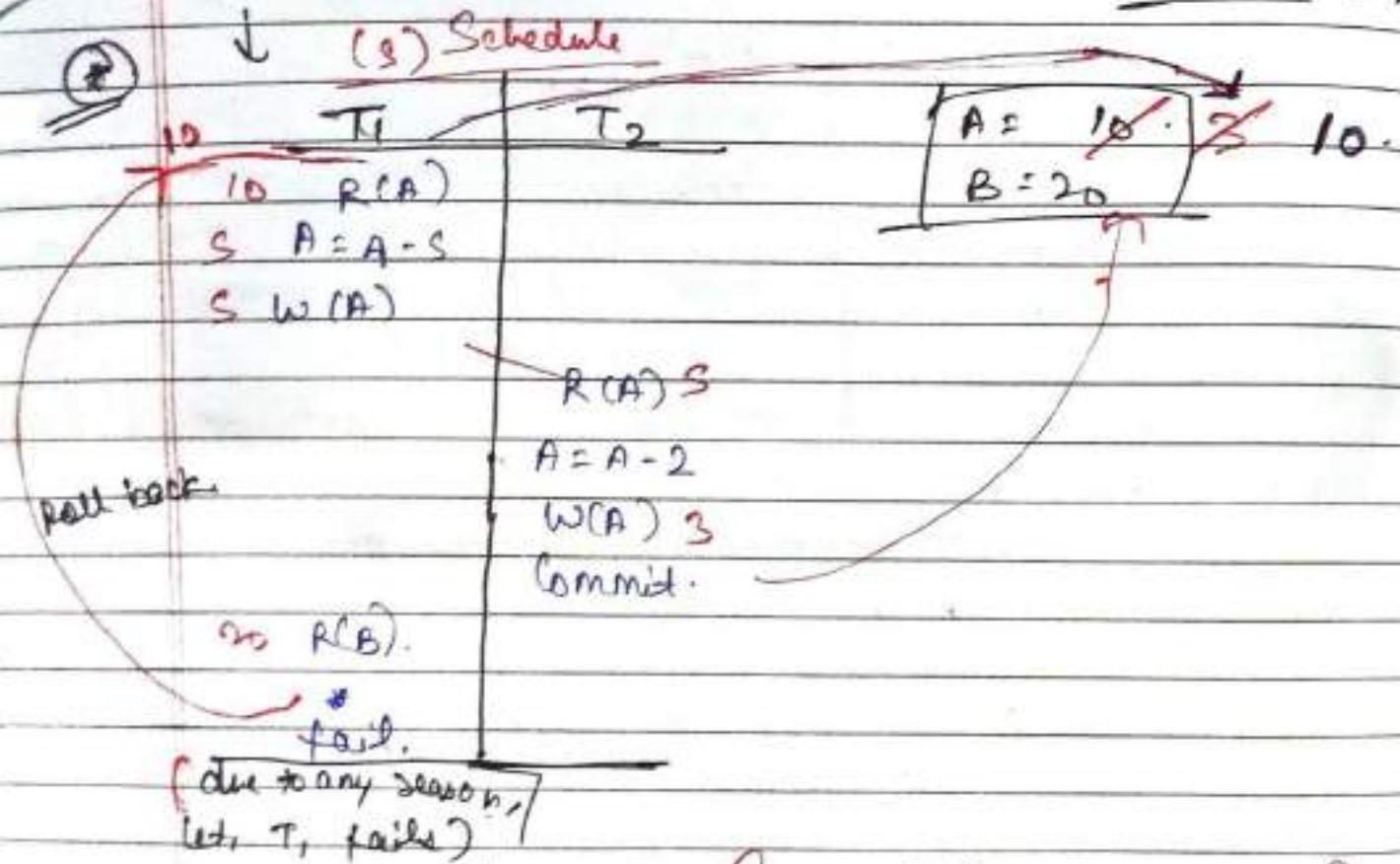
Now, User 1 have to be Roll back. (Abort)

Ex- (2)



78. Irrecoverable Vs Recoverable

Schedules in Transacⁿs :-



Now, (T₁ Rolls back due to Atomicity property.)

(Either all or None)

On Roll back, everything happens in T₁ is gone.

Again Now, [A is 10]

But, here T₂ also done something & that is lost now.

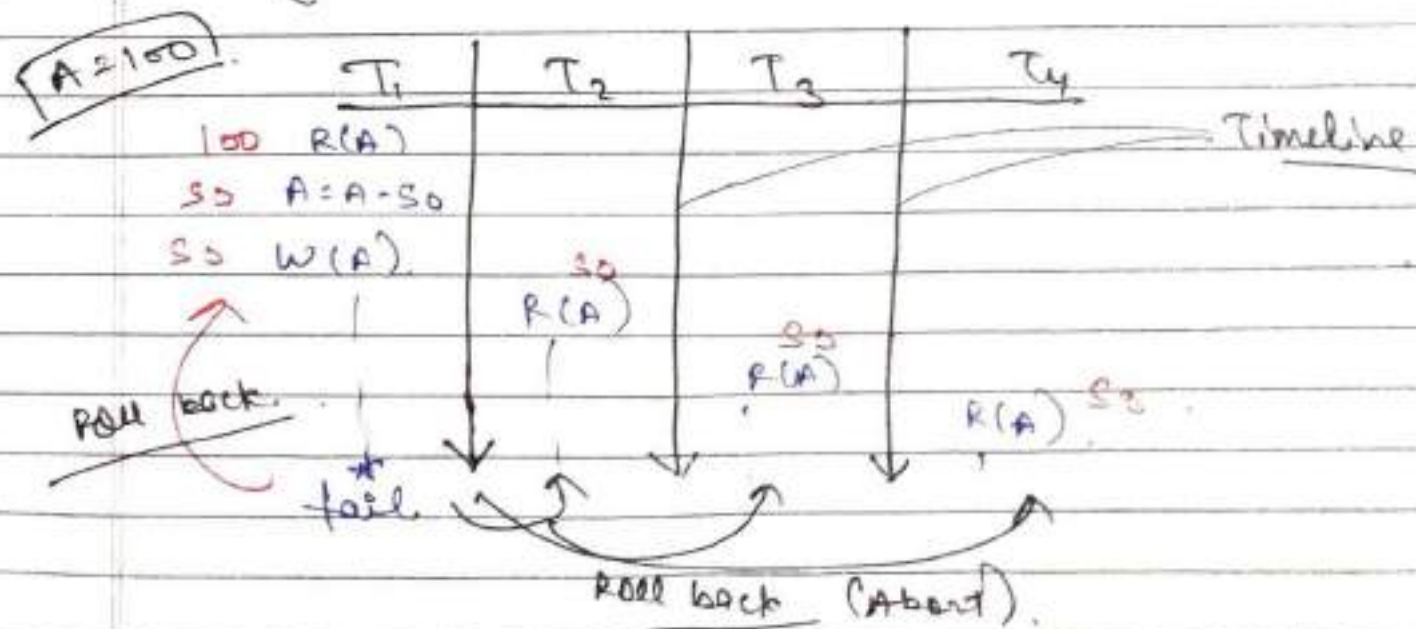
∴ T₂ Change is lost. we can't recover it now. ∴ It is Irrecoverable Schedule.

79. Cascading (vs) Cascadeless Schedules

→ In Recoverability, these 5 are Important:

- 1.) Recoverable
- 2.) 2pc-Recoverable
- 3.) Cascadeless
- 4.) Cascading
- 5.) Strict Recoverable.

⇒ Cascading! **means** Due to occurrence of one event, multiple events are automatically occurring.



→ Here, let T₁ fails due to any reason. Then, it roll backs automatically due to Atomicity & again (A is 100). But, now T₂, T₃, T₄ was (A → 50). So, they are working on wrong data. So, now, we also forcefully Roll back (Abort) the T₂, T₃ & T₄.

So, This is cascading.

(If T_1 fails, then we also have to roll back T_2 , T_3 & T_4)

→ Here, CPU utilisation gone waste by (T_2 , T_3 & T_4).

performance is bad / poor.

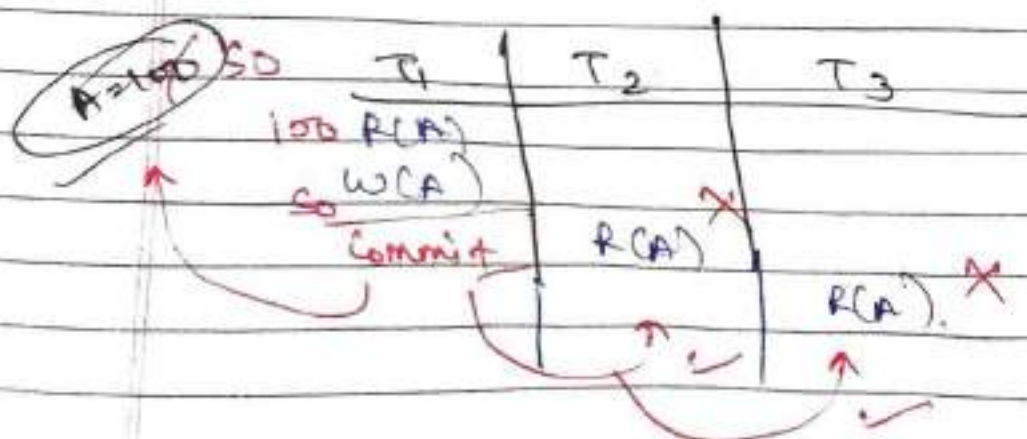
Ex: If a intelligent student (S_1) removes a answer. Then, other 3 students (S_2 , S_3 & S_4) also cuts that answer. Bcoz it is wrong. So, their work has gone waste. Cascading.

* Cascadeless : →

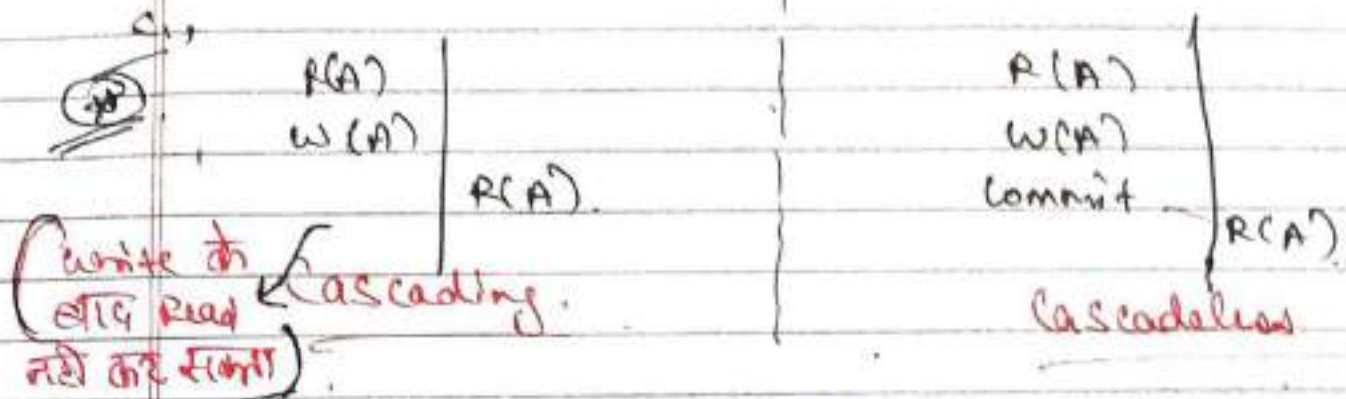
→ How to remove the problem of cascading?

→ Solⁿ: T_2 & T_3 can't Read the (A) value from T_1 until A value gets committed or roll back in T_1 .

(Commit → डेटा च ओके तो ओई रिकॉर्ड ही नहीं हटाएँ)

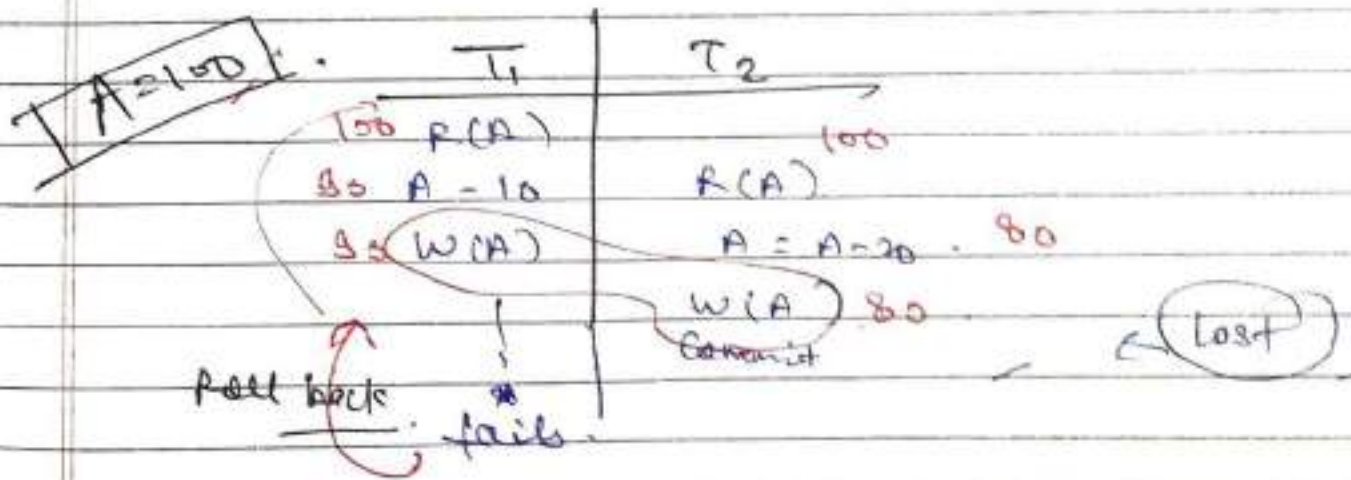


- i. Don't allow Read in T_2 & T_3 .
 It automatically becomes Cascadeless.



But, there is still W-W (Write-Write) problem in Cascadeless, b/c

(Read allow नहीं है, write तो कर सकता है)



Now, when T_1 fails. (A becomes 100 again)

The work of T_2 automatically gets lost.
 i.e. Write-Write problem (or) lost updation problem.

b/c T_2 value of W(A) $\rightarrow$ 80 तो नहीं च Reflect होता ही नहीं।

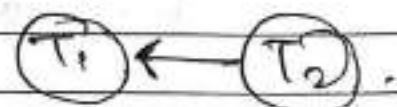
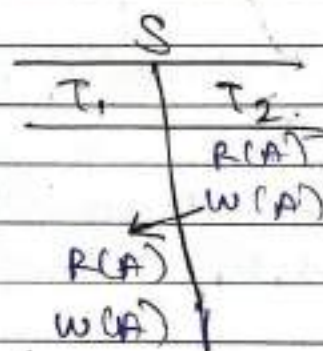
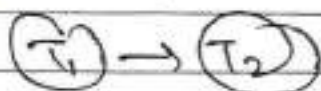
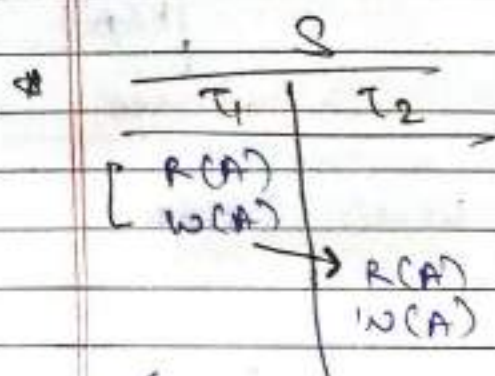
Strict Serializable tells that we also can't write along with Read.

80. SERIALIZABILITY: →

Serializability means that a schedule has ability to become a serializable.

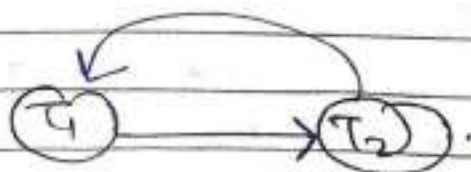
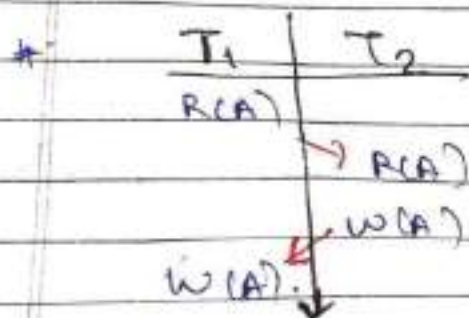
Mean,

Schedule - collection of transacⁿ ($T_1, T_2, \dots, T_n$)



By seeing these, we can tell that they are already in serial schedule. (Serial Schedule तो ये already है, Serial कहें नहीं कर सकते।)

→ We want, Parallel Schedule →



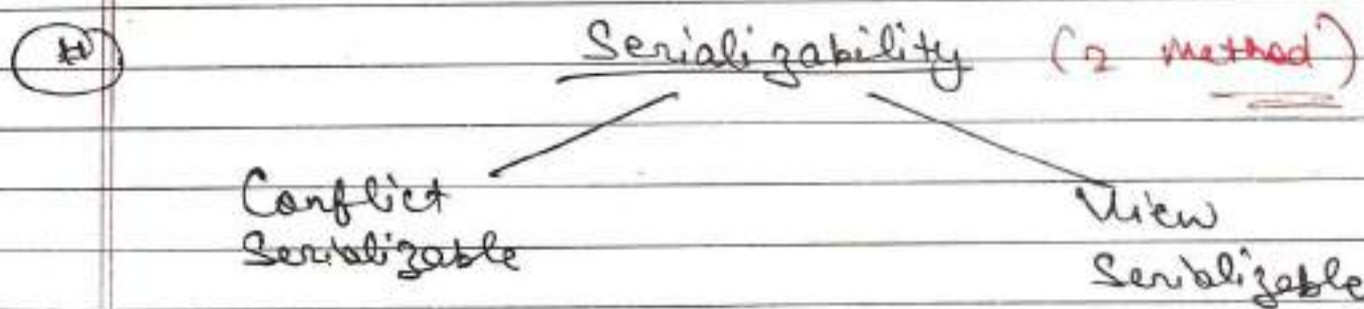
Convert it to Serial Schedule →

2 ways:

1) $T_1 \rightarrow T_2$
(OR)

2) $T_2 \rightarrow T_1$

2) To check Serializable? Check that if there exists an Serial Schedule Equivalent to Parallel Schedule or not. This concept is known as Serializability.



→ We check that a parallel schedule can convert to a Serial Schedule or not. (done of 11 schedule) Serializable

iv) Let, a schedule has 3 transacⁿs.

parallel schedule

	T ₁	T ₂	T ₃
		R(A)	
			R(A)
			W(A)
		W(A)	
	R(B)		
	W(B)		
		W(B)	

Serial Schedules

16 ways

1. $T_1 \rightarrow T_2 \rightarrow T_3$
2. $T_1 \rightarrow T_3 \rightarrow T_2$
3. $T_2 \rightarrow T_3 \rightarrow T_1$
4. $T_2 \rightarrow T_1 \rightarrow T_3$
5. $T_3 \rightarrow T_1 \rightarrow T_2$
6. $T_3 \rightarrow T_2 \rightarrow T_1$

⇒ If we are able to convert that parallel schedule in any of the 6 forms of serial schedule, then we can say that it is a serializable.

⇒ Why we get 6 forms! →

for we have 3 values (T_1, T_2 & T_3).

So, 3! = 6 ways:

Ex 1) Conflict Equivalent Schedules! →

$R(A) \quad R(A)$ } Non-conflict pair.

$R(A) \quad W(A)$
 $W(A) \quad R(A)$
 $W(A) \quad W(A)$ } Conflict pairs

Ex 2) $R(B) \quad R(A)$
 $W(B) \quad R(A)$
 $R(B) \quad W(A)$
 $W(A) \quad W(B)$ } Non-conflict pair.

Ex 3) How to Convert : →

If we have adjacent non-conflict pair, then swap their pair.

Ex 1) $T_1 \quad T_2$
 $R(B) \quad R(A)$ → $T_1 \quad T_2$
 $R(A) \quad R(B)$

Q: To check Conflict Equivalent: $\rightarrow$

$S \equiv S'$ $\rightarrow$ check

S	
T ₁	T ₂
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	

S'	
T ₁	T ₂
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

Sol: So, In S, we have adjacent non-conflict pair, so swap them.

S	
T ₁	T ₂
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	

$\Rightarrow$

S	
T ₁	T ₂
R(A)	
W(A)	
	R(A)
R(B)	
	W(A)

Again.

S	
T ₁	T ₂
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

= S' Hence

$S \equiv S'$

Now
Serial
Schedule $\leftarrow$

Hence,

S & S' are Conflict Equivalent Schedule.

Ex!

T_1	T_2
R(A)	
W(A)	
	R(A)
W(A)	
R(A)	

↓
2 adjacent pairs, But

they are conflict pairs. ∴, no change in positions.

Note: $S \xrightarrow{\text{CE}} S' \rightarrow \text{Serializable}$
 ↓
 conflict equivalent
 (Serial Schedule)

(1-81) videos end here

INDEX

Anurag Singhal DBMS STD: SEC: ROLL NO:

S. No.	Date	Title	Page No.	Topic / Page No.
82.		Conflict Serializability; precedence Graphs	1-4	
83.		View — " —	4-6.	
84.		Concurrency Control protocols	7-8.	
85.		Drawbacks in Shared / Exclusive	9-11	
86.		phase locking (2PL) protocol	12-15.	
87.		Drawbacks in 2PL protocol	15-17.	
88.		Strict 2PL, Rigorous 2PL & Conservative 2PL ^{Schedul}	17-21.	
89.		Basic Timestamp Ordering Protocol	21-24	
90.		Some Ques ⁿ on — " — " — " —	24-25.	
91.		INDEXING	26-28.	
92.		Ques ⁿ on I/O Cost in Indexing	29-31.	
93.		Numerical on — " — " — " —	31-35.	
94.		Types of INDEXES	35	
95.		Primary INDEX	36-37.	
96.		CLUSTERED Index	37-38.	
97.		Secondary Index (Multilevel Indexing)	39-42.	
98.		Intro to B-tree & its Structure	42-45.	
99.		Insertion in B-Tree	45-47.	
100.		How to find order of B-Tree	48-49.	
101.		Diff B-Tree & B+ Tree	49-53.	
102.		Ques ⁿ on order of B+ Tree	53-54.	
103.		Immediate Database Modification	55-57.	
104.		Q ⁿ on DBMS Basic Concepts & Data Modelling	57-59.	
105.		Imp Ques ⁿ on Advance DBMS	60-63.	
106.		Deferred Database Modification	63-66.	
107.		LIKE Command in SQL	66-67.	
108.		Basic PL-SQL programing with execution	68-69.	

109. PL-SQL (while, for loop).

110. Single & multipow func's in SQL

111. character func's in SQL

112. View in Database.

* SQL cheatsheet

69-70.

70-71.

72-73.

74-77.

78-81.

Q2

Conflict Serializability, Precedence Graph

Ans:

T_1	T_2	T_3
$R(x)$		$R(y)$ $R(x)$
	$R(y)$ $R(z)$	$w(y)$
	$w(z)$	
$R(z)$ $w(x)$ $w(z)$		

↓ — Timeline.

First, we have to make precedence graph → (mean, just graph with edges & vertices).

Vertex → No. of Transac's (T_1, T_2 & T_3).

Now

* Edges

check the conflict pairs in other transactions and draw edges.

like * T_1 → (check in T_2 & T_3)

* Conflict pairs!

$R-W$
$W-R$
$W-W$

of same variable.
lets (x).

* Start!

Now, Start with first opⁿ. i.e.,

$T_1 \rightarrow R(x)$

→ check the conflict pair of $R(x)$ in other Transac's i.e. $(T_2 \& T_3)$.

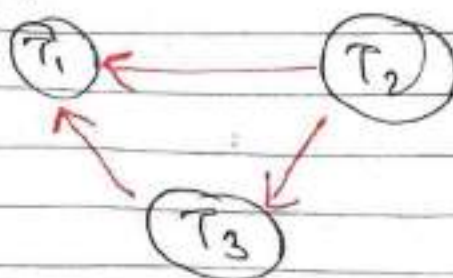
→ Conflict pair of $R(x) \rightarrow w(x)$

now, Similarly,
Conflict pair of $w(x) \rightarrow R(x), w(x)$.

Now, do check for every oper's in all transac's & after checking list these oper's. & if you find any conflict pair, then draw edge acc. to vertex (Transac').

T_1	T_2	T_3	C.P. we find $\rightarrow$
$R(x)$		$R(y)$	$R(x) - w(x) (T_3 \rightarrow T_1)$
		$R(x)$	$R(y) - w(y) (T_2 \rightarrow T_3)$
	$R(y)$		$R(z) - w(z) (T_2 \rightarrow T_1)$
	$R(z)$		$w(z) - R(z) (T_2 \rightarrow T_1)$
	$w(z)$	$w(y)$	$w(z) - w(z) (T_2 \rightarrow T_1)$
$R(z)$			
$w(x)$			
$w(z)$			

Precedence Graph:



Now, Come on Graph! -
 check that if there is any loop/cycle in the graph.

How to check cycle? → अगर हम एक किसी भी Node से चलें, तो क्या हम वही Node दोबारा आ सकते हैं? If yes → loop/cycle exists.

(It is not must that the cycle must covers all the nodes. May be possible, it comes just after visiting one node). → also a cycle.

(# In our graph, there is no loop/cycle.)

No loop/cycle → Conflict Serializable Schedule
 If loop/cycle exists → Not C.S.S.

Conflict Serializable → Serializable → Consistent.
 (Serial Schedule)

* Now, how to make Serializable! →

$$\begin{array}{l}
 T_1 \rightarrow T_2 \rightarrow T_3 \\
 T_1 \rightarrow T_3 \rightarrow T_2 \\
 T_2 \rightarrow T_1 \rightarrow T_3 \\
 \boxed{T_2 \rightarrow T_3 \rightarrow T_1} \\
 T_3 \rightarrow T_1 \rightarrow T_2 \\
 T_3 \rightarrow T_2 \rightarrow T_1
 \end{array}$$

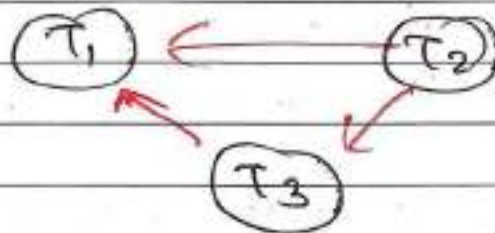
6 Cases → Now, which Case?

Again, see the graph & check

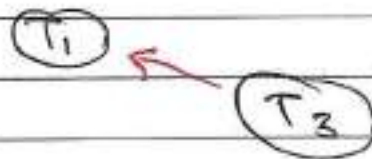
Indegree = 0

Indegree = 0, means a vertex with '0' indegree.

Mean, जहाँ पर कोई भी Edge (arrow) नहीं आ रहा है।
vertex



T_2 has, Indegree = 0,
 Now, remove T_2 .



∴

$T_2 \rightarrow T_3 \rightarrow T_1$ is

T_1	T_2	T_3
	0	
0		0

x

x

83. View Serializability $\Rightarrow$

Q!- check whether schedule is conflict Serializable or not?

→ first make precedence graph.

Loop $\rightarrow$ No Answerable. then, view serializable Answers.



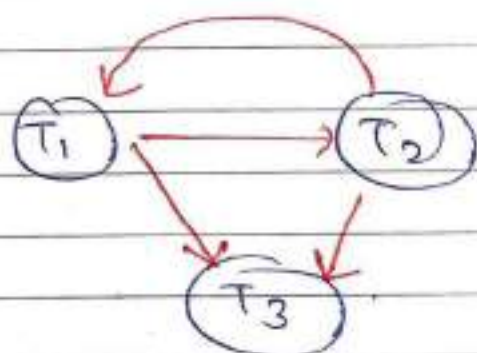
Date _____

Page No. _____

(8)

S

T_1	T_2	T_3
$R(A)$		
	$w(A)$	
$w(A)$		
		$w(A)$



Here, we get a loop.
So,

It is Non Conflict Serializable.

Here, we can't tell that it is serializable or not.

Now,

To check that, we use View Serializability.

Now, we arrange this table,

T_1	T_2	T_3	T_1	T_2	T_3
$R(A)$			$R(A)$		
	$w(A)$		$w(A)$		
$w(A)$				$w(A)$	
		$w(A)$			$w(A)$

we change the position

Now this is a serial schedule, $T_1 \rightarrow T_2 \rightarrow T_3$.

But,

we have to check whether they match each other or not.

Let $A = 100$



T_1	T_2	T_3
100 $R(A)$	$A = A - 40$ $W(A)$ - 60	
$W(A)$ $A = A - 40$ (20)		$W(A)$ $A = 20$ (0)

T_1	T_2	T_3
100 $R(A)$	60 $W(A)$ (20)	
	(20) $W(A)$ - 20	$W(A)$ (0)

Now, apply this same here. $\Downarrow$

Here, finally
A value - 0

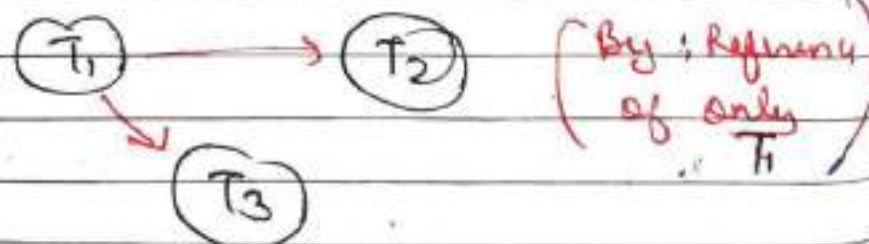
Here, also
finally A is '0'.

Hence, Both are Equivalent.

They are View Equivalent ($\equiv$).

[Note:-] here, finally output is given by A of T_3 . So, if adjust the posⁿ of A of T_1 & T_2 . So, there is no problem.

Now,



is

Hence, $T_1 \rightarrow T_2 \rightarrow T_3$ (By Table).

[Note:-] Conflict Serializable tell that it is not Serializable (no ans.). But, View Serializable Checks it & tell that it is Serializable.



84.

(parallel schedule) Concurrency Control Protocols! → (C.C.P.)

(Shared-Exclusive Locking Protocol)

→ C.C.P. is that how to make them serializable, or how to make them recoverable. Schedules are concurrent, but how we can make them serializable & recoverable, this comes under Concurrency Control Protocol (C.C.P.).

→ We achieve this, By using Locking Protocols.

① 'Shared-Exclusive locking'! →

We use 2 locks here,

→ Shared lock (S) → if trans. locked data item in shared mode, then allowed to Read only.

U → means unlock.

T_1
S(A)
R(A)
U(A)

→ Shared lock.

→ only Read

→ unlock.

→ Exclusive lock (X) → if trans. locked data item in exclusive mode then allowed to Read & write both.

T_2
X(A)
R(A)
W(A)
U(A)

→ Exclusive lock.

→ unlock.

* Compatibility Table! →

Request

↓

		S	X
grant	S	Yes	No
	X	No	No



Exple:-

S has only R(A)

X has both R(A), W(A)

So

⇒ S - S (No conflict) b/c R(A) - R(A)

⇒ S - X (Conflict)

R - R

R(A)	R(A)
	W(A)

⇒ X - S (Conflict)

R(A)	
W(A)	R(A)

⇒ X - X

R(A)	
W(A)	R(A)
	W(A)

→ Conflict

* Problems in S/X Locking! →

1.) May not sufficient to produce only serializable schedule.

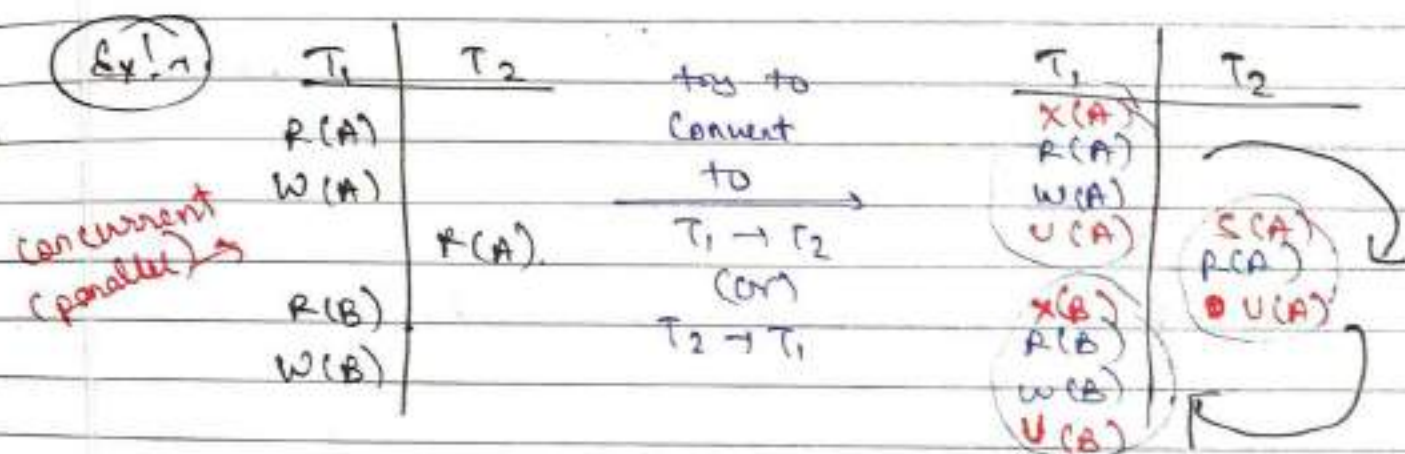
2.) May not free from Ir-recoverability.

3.) May not free from dead lock.

4.) May not free from starvation.

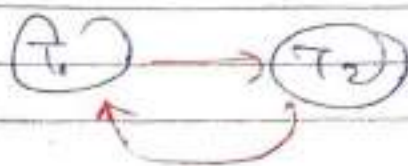
85. Drawbacks in Shared / Exclusive S/X locking protocol →

- Locking gives us serializable Schedule.
→ Consistent.
- 1.) May not sufficient to produce only serializable schedule.
→ Not all correct, not all.



Note: 1) Until, we do unlock, $v(A)$ in T_1 , we don't be able to put shared lock $S(A)$ on T_2 . b/c. by Compatibility Table.
 $(X-S) \rightarrow No$, so first we unlock $X(A)$.
 &
 then, use $S(A)$ on T_2 data.

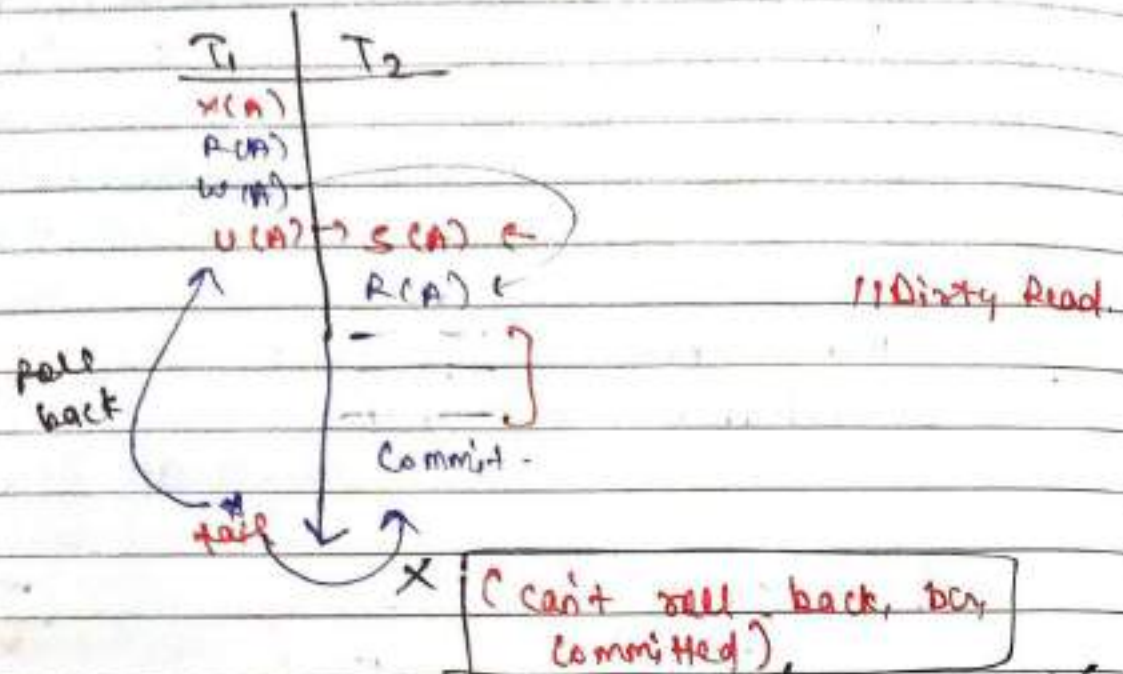
- 2) Here, we don't get Serializable Schedule even after applying SIX locking. As u can see in Table.





2.) May not free from 2nd recoverability.

Ex:-



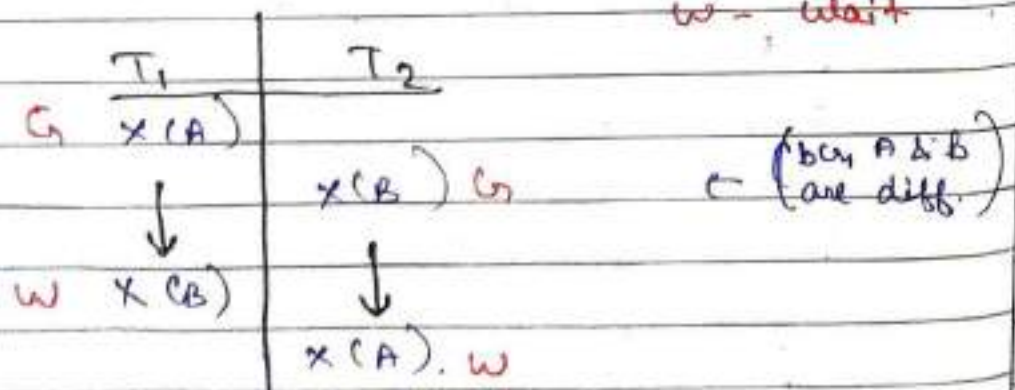
∴ It can't recover here

3.) May not free from dead lock.

Dead Lock! → When 2 person wait for resources, & they both are waiting in an infinite loop, so, when we wait infinitely, it is called Dead Lock State.

G - Grant
W - Wait

Ex:-



2) Here, both are in waiting bcz, They are not unlock after use. So,

→ T_1 also waits that when T_2 unlocks, he can use on X(B).

Same

→ T_2 also waits that when T_1 unlocks, he can use on X(A).

So, Both are waiting in an Infinite loop.

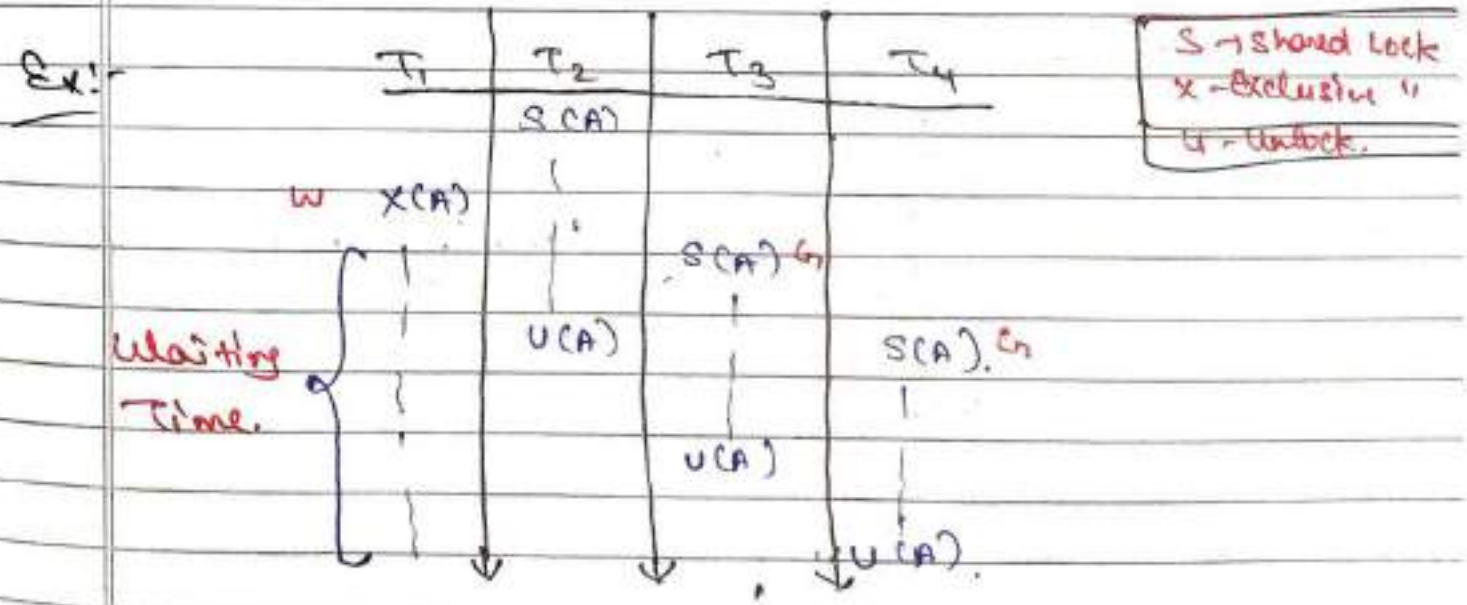
4.] May not free from starvation.

In deadlock, waiting upto Infinite time.

But,

In starvation, waiting not upto the Infinite time.

→ Shared to Shared & Shared to Shared by Compatibility Table. Without unlock.



So, here, T_1 waits for X(A) until T_4 unlocks.

So, here starvation also. (key Shared to Shared Exclusive, T_1 ni unlock k grant nahi deta)

86.

Phase Locking (2PL) protocol in

Transaction Concurrency Control :-

2) 2PL (2 phase locking) is just the extension of simple shared/exclusive locking.

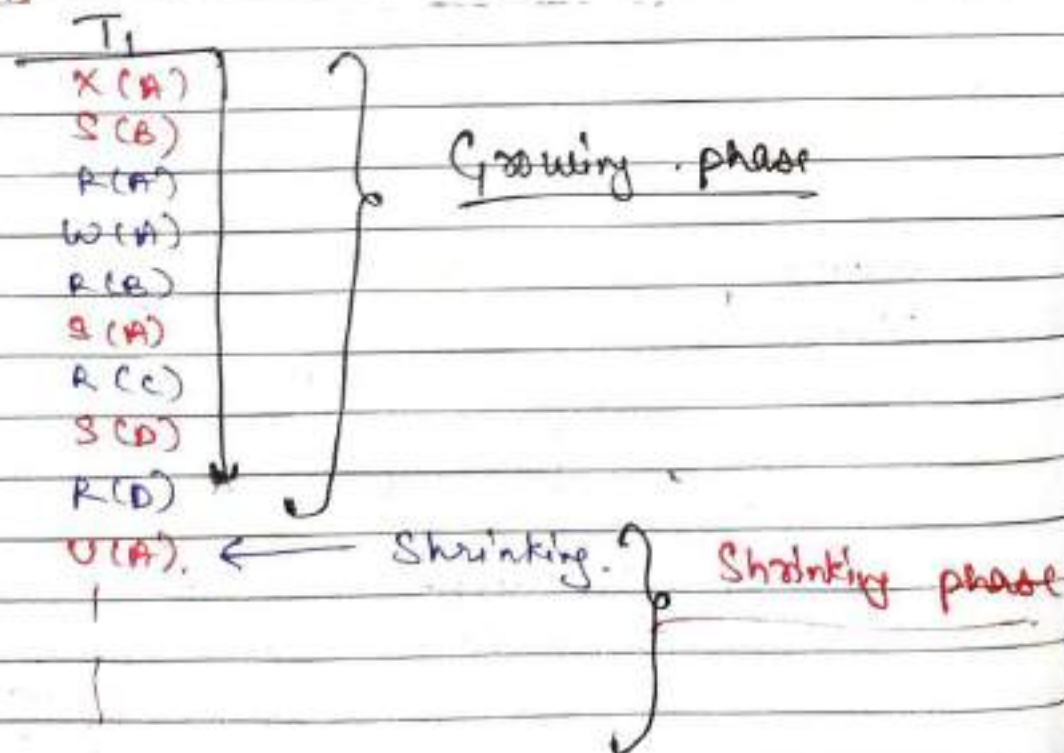
We just do modifications in them.

(#) 2-Phase locking (2PL) :-

→ Growing phase :- locks are acquired
↳ no locks are released.

→ Shrinking phase :- locks are released.
↳ no locks are acquired.

[S/X]



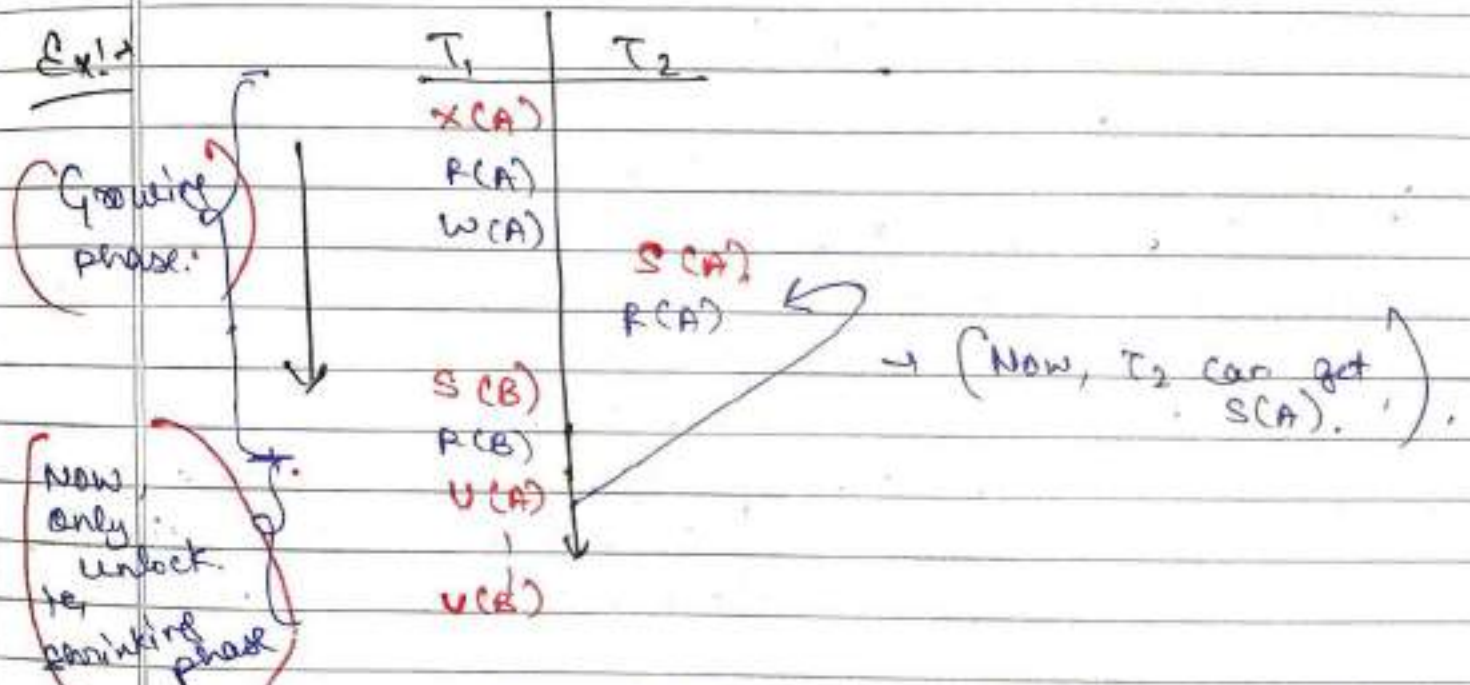
Note! When growing phase starts, then only apply locks.

[and]

When we first unlock, i.e., shrinking phase starts, then we only unlock. (and not able to apply any lock).

→ We achieve Serializability by this, that we don't achieve in simple shared/exclusive protocol. So, we made (2-PL) protocol for this.

Ex! →



Hence, we first starts T_1 , & if T_2 comes in b/w then we don't entertain him. First, we complete T_1 & then goes to T_2 .

$T_1 \rightarrow T_2$

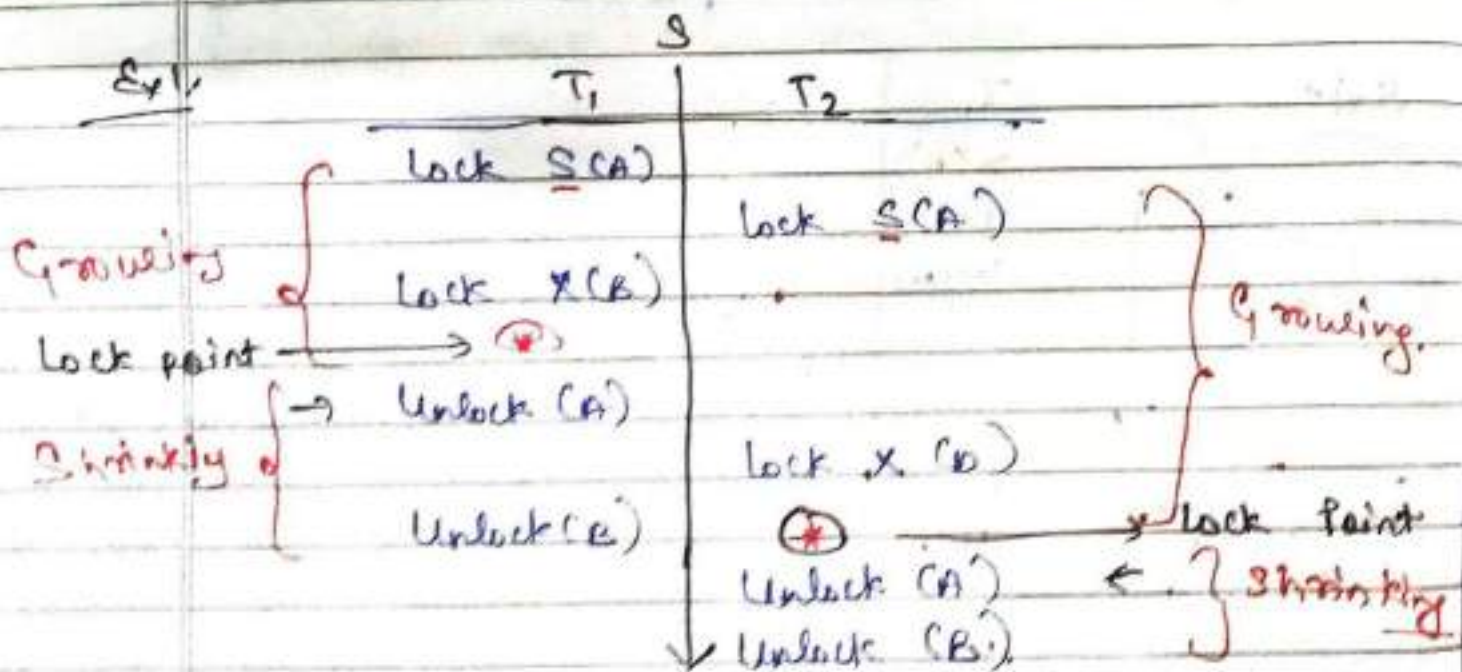
Serializability achieved.

Consistency automatically achieved.

Note! The transaction which follows (2-PL), then it is always serializable.

Ex! 4th & 3rd stored in 3rd stored in 1st means,

While T_1 is in growing phase by using S(A) (Shared Lock), then at same time T_2 also starts its growing phase by using S(A). b/c, by Compatibility Table.



Now,

Serializability Schedule, How formed?

i.e. $T_1 \rightarrow T_2$ (OR)
 $T_2 \rightarrow T_1$

So, This is done by (Lock Point).

Commit or Roll back. नहीं कर सकते।

→ Lock Point! → where trans. is taking the last lock. (or)
where trans. is unlock first time.

→ जिसका Lock Point पहले आ जाएगा। वो trans. पहले आ जाएगा।

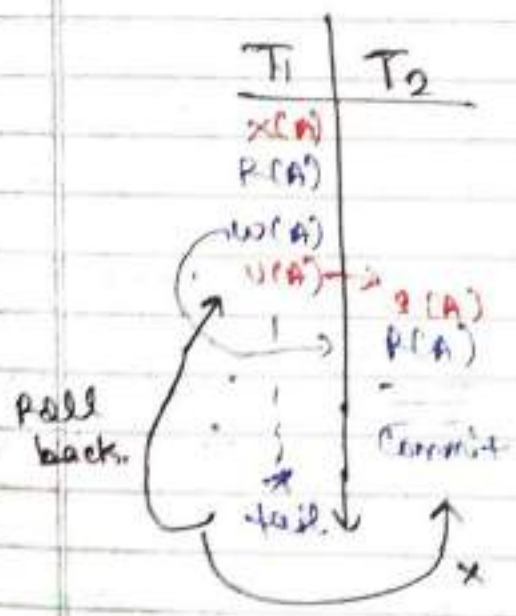
ie, $T_1 \rightarrow T_2$

(87) Drawbacks in (2-PL) protocol! →

+ Advantage! Always Ensures Serializability.

2) Drawbacks! →

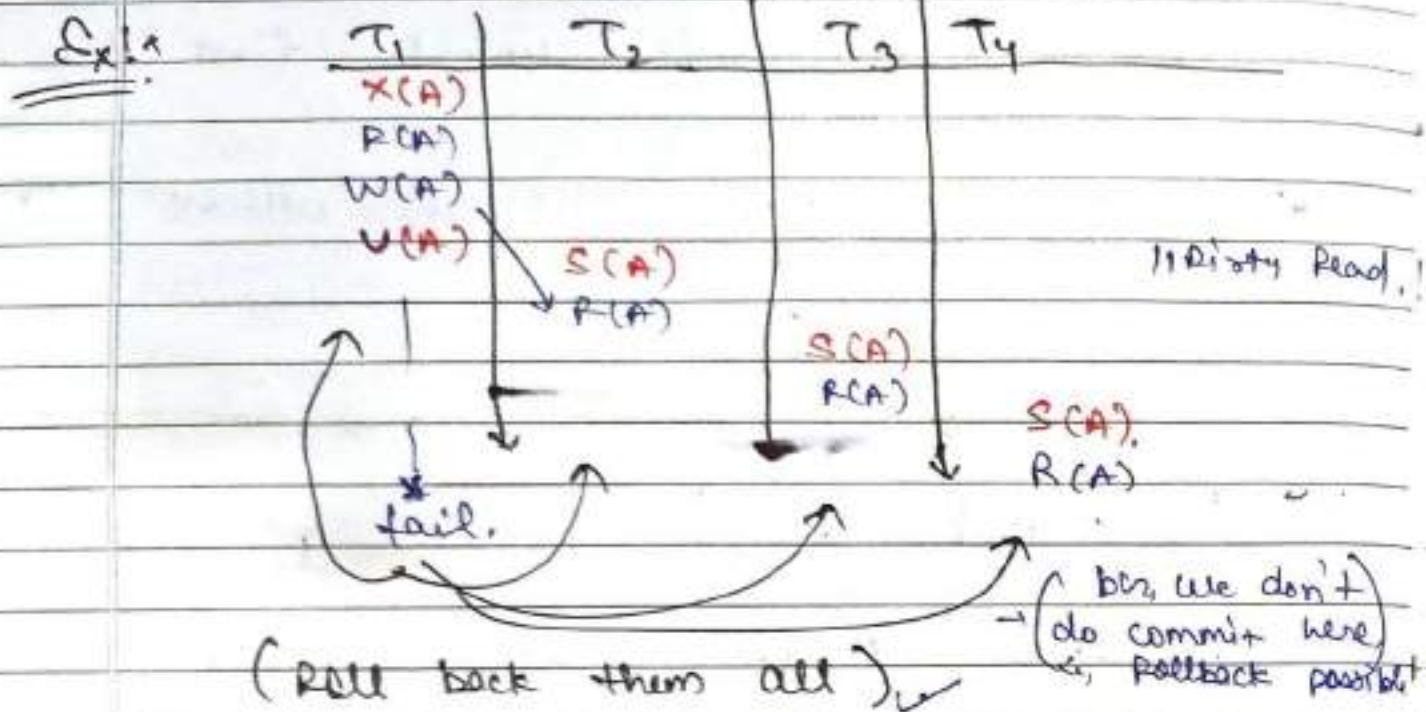
(1.) May not free from Ir-recoverability.



→ (we can't roll back it, bcoz it is already committed)

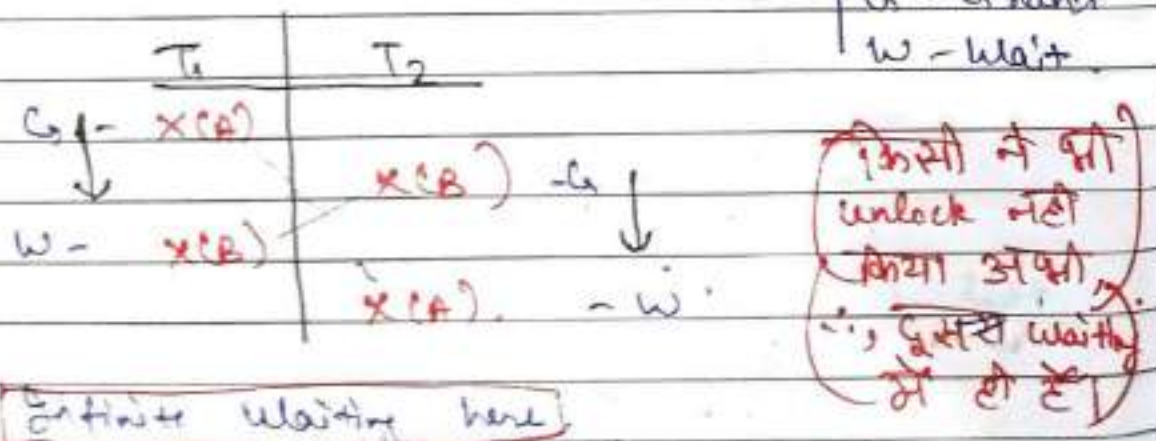
Hence,
It is Ir. of Ir-recoverable Schedule.
We can't recover it (Roll back it).
(normal).

(2.) Not free from Cascading rollback →



Bad performance. & Cascading Rollback is possible here.

(3.) Not free from Deadlocks.



(T₁ & T₂ both waiting here to complete their transac's.)

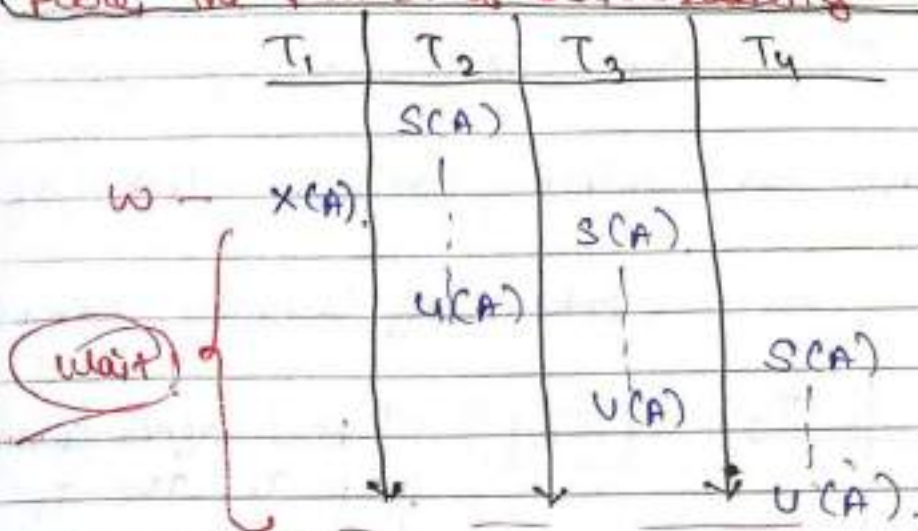
(4.) Not free from starvation.

(wait for limited time)

In 2PL

Here, the problem of serializability

removes.



88. Strict 2PL, Rigorous 2PL & Conservative 2PL Schedule :-

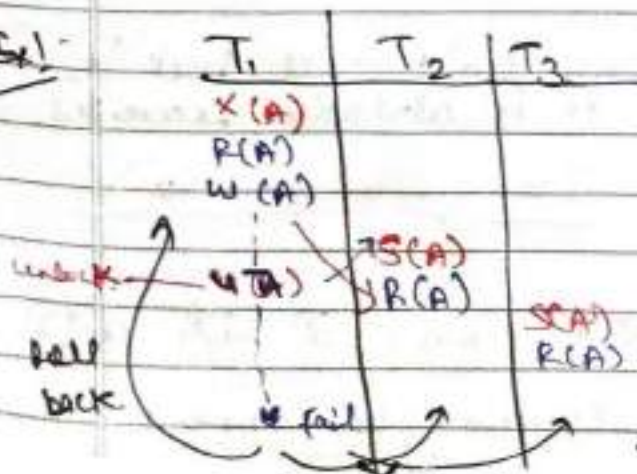
→ These are the extension (advance) of 2PL's basic.

Strict 2PL :-

It should satisfy the basic 2PL and all exclusive locks should hold until commit/Abort.

Rigorous 2PL :- It should satisfy the basic 2PL and all shared, exclusive locks should hold until commit/Abort.

Ex:-

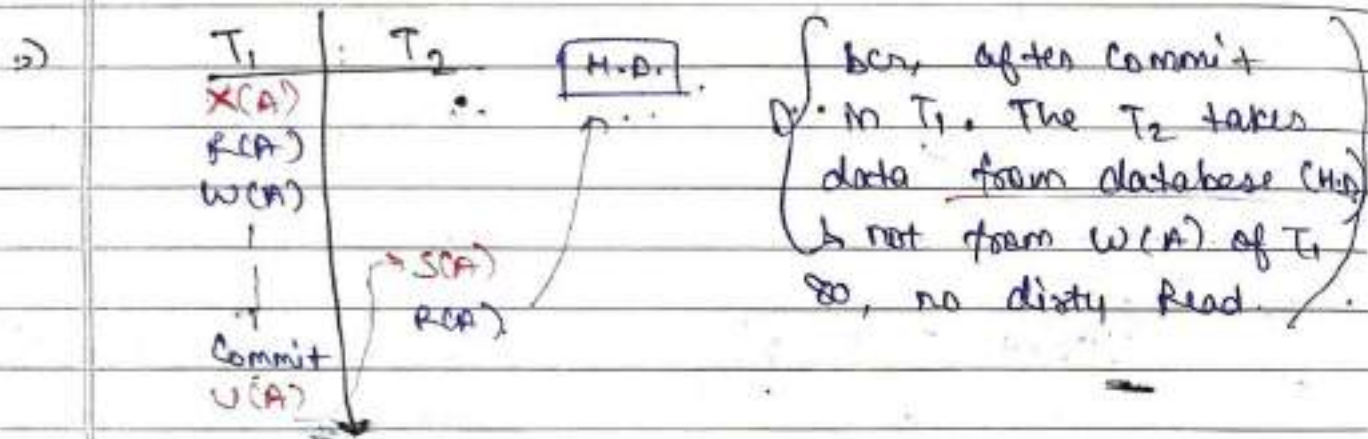


ie. (Cascading roll back)

→ (also have to roll back T₂ & T₃)

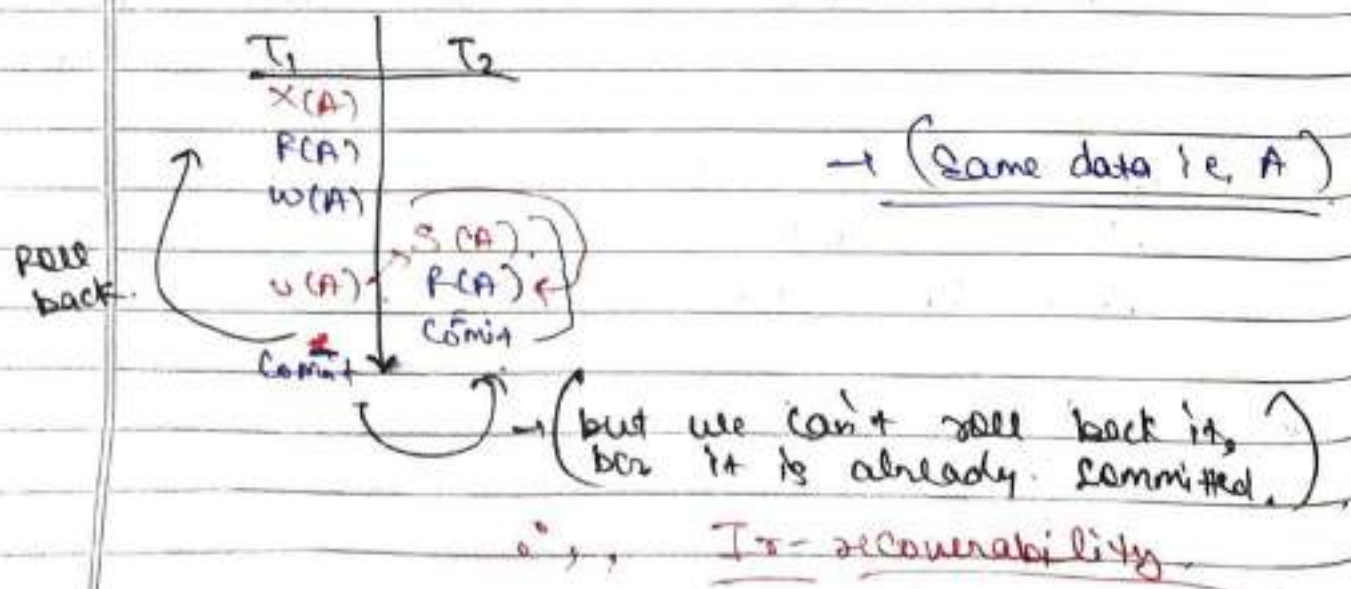
→ We have to unlock Exclusive lock after Commit.

then, this problem of cascading sermons.



Hence, Here, Cascading Rollback is Removed,
 & It will always produce Cascadeless.

⇒ Now, δ Ir-recoverability! $\rightarrow$



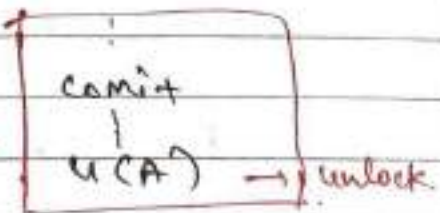
→ Hence, इस इंसको पहले unlock ही नहीं करो।
Use unlock it after commit is done.



2) Hence, T_2 को S/x कुछ भी नहीं मिलेगा, जब तक T_1 commit नहीं हो जाता।

और अगर commit होने के बाद S/x मिल गया है (T_2 को), then No problem.

3) Hence, Irrecoverability remains here.



फिर वो Database (H.D.) से ही read करेगा।

→ It produce Strict Recoverable Schedule.

Note:

2 problems remains here →

1.) Cascadeless

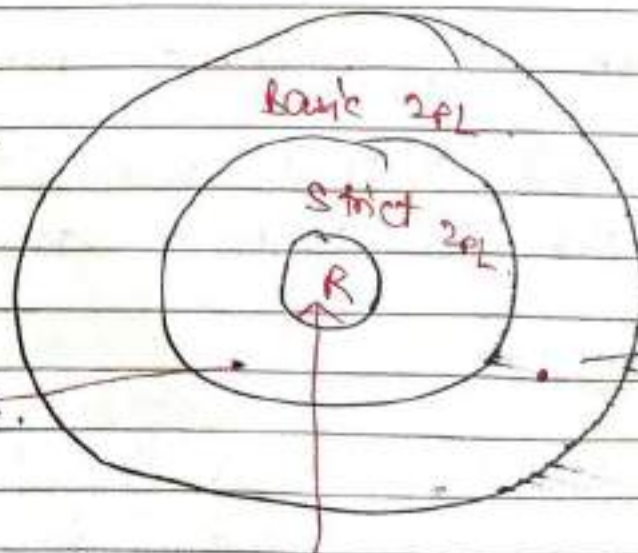
2.) Strict Recoverable

→ Rigorous 2PL (R), इस और शक्ति हो गया। AC, इसमें S/x दोनों आ गए।

(इसमें हम shared lock को भी release नहीं कर सकते।)

④

In both 2PL & Strict 2PL. But not in Rigorous 2PL.



In Basic 2PL only

But, problem of deadlock & starvation still are there.

1)

T_1	T_2
$X(A)$	$X(B)$
$W - X(B)$	$X(A) - W$

→ Still, Deadlock.

→ Infinite waiting.

2)

T_1	T_2	T_3
$W - X(A)$	$S(A)$	
waiting	C	$S(A)$
		C

3) finite waiting

→ Still, starvation.

Conservative 2PL →

practically, it is not possible. In this, कि कोई भी trans. start होने से पहले ही साहे lock लेले। ($A, B, C \rightarrow$ सब ये लेले।)

3) इससे T_2, T_3 को एक भी lock नहीं मिलेगा।

∴ problem of Deadlock remains here.

• (T_1 को पहले ही सारी resource दे दे, ताकी T_2 को एक भी resource ना मिले।)

3)

T_1	T_2
$G - X(A)$	$X(B) - W$
$G - X(B)$	$X(A) - W$

G - Grant
W - waiting.

1, T_1 completes here first. So, no infinite waiting here.

(T_1 on comp. line पर T_2 use करे नहीं)

2,

[No Deadlock here.]

89. ^(T.S.) Basic Time stamp Ordering Protocol : →

- unique value assign to every transaction.
- Tells the order (when they enters into system)

Let,

10:00	10:10	10:15	Time
T_1	T_2	T_3	
100	200	300	Time stamp
↓ older (पहले आई)	↓ Younger (बाद में आई)	↓ Youngest	

- Read-Ts (RTS) = last (latest) transaction no. which performed Read successfully
- Write-Ts (WTS) = last (latest) transaction no. which performed write successfully

Ex 12

10	20	30	T.S. (T_i)
T_1	T_2	T_3	
R(A)			
	R(A)		
		R(A)	

$$\Rightarrow RTS(A) = 30$$

→ here, latest read is of T_3 & Time Stamp (T.S.) of T_3 is 30.

Ex:-

10	20	30
T_1	T_2	T_3
$w(A)$		$w(A)$
	$w(A)$	

$$WT(A) = 20$$

Rules :-

In Timestamp, let
 put Trans. T₁ first, let T₂ complete first.

Serializability: older $\rightarrow$ younger.

follows
 conflict
 serializability

Read

1. Trans. T_i issues a Read (A) operaⁿ

a) If $WTS(A) > TS(T_i)$, Rollback T_i

b) otherwise Execute R(A) operaⁿ

Set $RTS(A) = \max\{RTS(A), TS(T_i)\}$

(0, 100, 200, 300) $\rightarrow$ in next video use

Write

2. Trans. T_i issues Write (A) operaⁿ

a) If $RTS(A) > TS(T_i)$ then Rollback T_i

b) If $WTS(A) > TS(T_i)$ then Rollback T_i

c) otherwise execute write (A) operaⁿ

Set $WTS(A) = TS(T_i)$

Understanding these :-



Ex → (old) ¹⁰⁰ T₁ | ²⁰⁰ T₂ (young)

R(A)	W(A)
------	------

T ₁	T ₂
W(A)	R(A)

T ₁	T ₂
W(A)	W(A)

✓ (NP).

(No Conflict).

bcz

(old → young) on (T₁ → T₂). So,

(T₁ पहले Execute हो जाएगी, तो T₂ में कोई
Read नहीं आएगी।)

Ex → (old) ¹⁰⁰ T₁ | ²⁰⁰ T₂ (young)

W(A)	R(A) = 10
------	-----------

Roll Back.

(T₁ को होने
नहीं देना।)

(Not possible)

Case I

(bcz, now ये T₂ में
A = 10, कौन सी value
Read करते रहेंगे हैं। bcz
T₁ ही वो Database में 20
update कर दिया।)

Case II

Ex → T₁ | T₂

20 → R(A)	W(A) = (20)
-----------	-------------

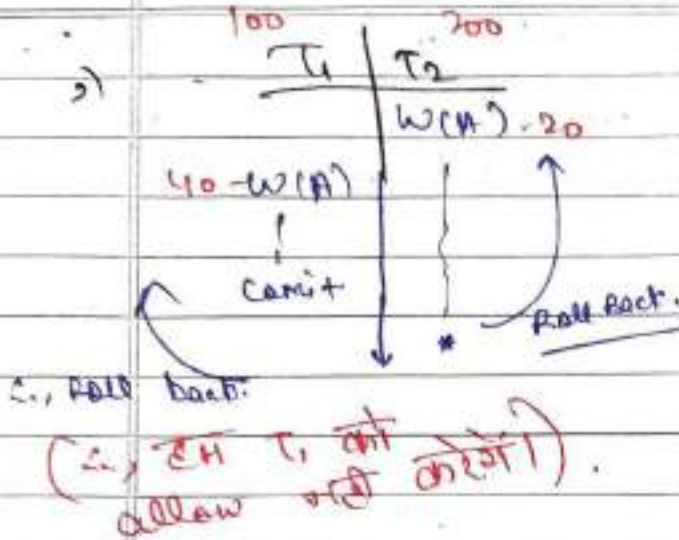
Roll Back.

(हम T₁ को read
नहीं करने देंगे।)

(Hence, ये 20 अब कहीं
पहं है ही नहीं। i.e.,
T₁ गलत Data पे काम
कर गयी।)

|| Dirty Read

Case III



Let, If T_2 fails, then, our updation will be lost.

11 lost updation.

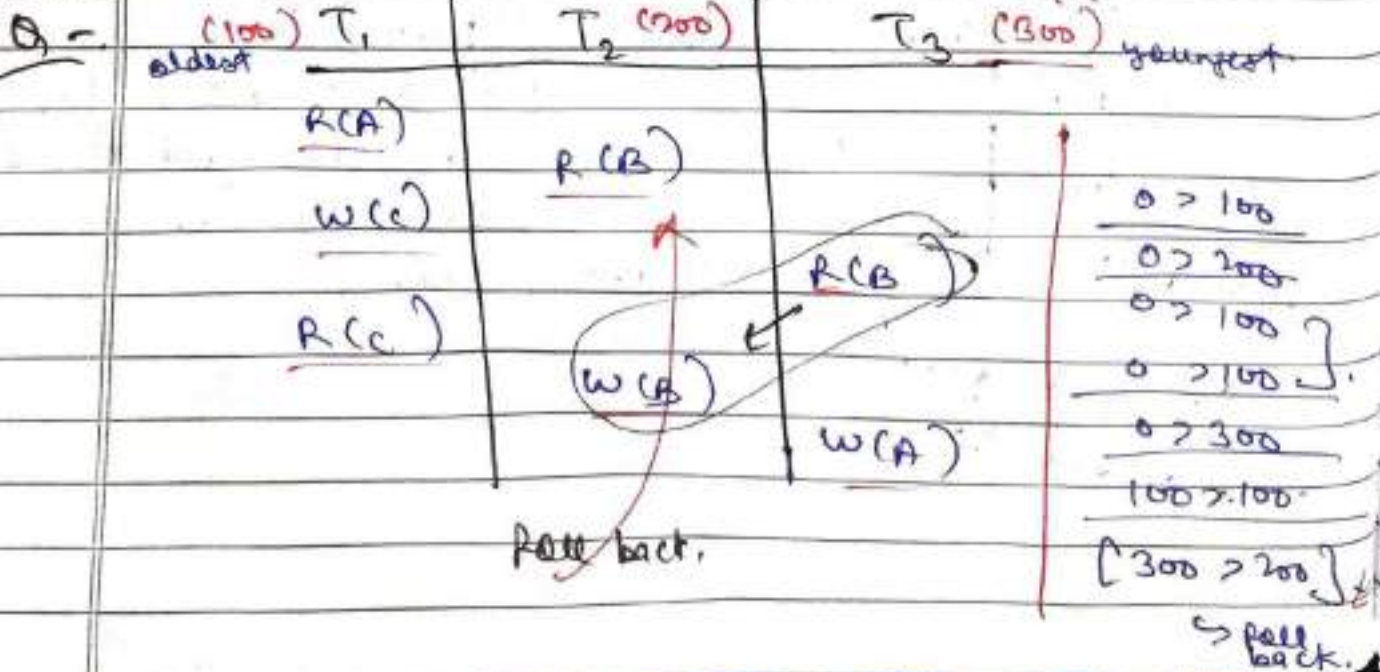
90. Solve Ques on Timestamp Ordering Protocol.

Use same 2 Rules →

1.) 1st Rule is of Read → has only 'cond'
i.e., Only (R-W) conflict.

2.) 2nd Rule is of Write → has 2 cond's,
i.e., (W-R, W-W) → 2 conflicts.

Now



	A	B	C
RTS	0 ¹⁰⁰	0 ^{200 300}	0 ¹⁰⁰
WTS	0 ³⁰⁰	0	0 ¹⁰⁰

Initially, all values are 'zero'.

(Final Table)

1) Check, for every value in the Table & put that value in Rules & then make the Table. (Trans)

for R(A) use 1st Rule of Read
for W(B) use 2nd Rule of Write

1) first check R(A) of T₁, then
R(B) of T₂, then
W(C) of T₁, then
R(B) of T₃, then
R(C) of T₁, then
W(B) of T₂, then
W(A) of T₃

Here, Roll back. ←

i.e. pattern write as in Table

&

then, make the final Table of

(RTS) & (WTS)

[OR]

We also do the Quesⁿ direct, without using these 2 Rules with just the [older → younger] concept & cases (we discussed in prev. video).

Q1. INDEXING :-

- CPU → processor (who process).
- Query comes to CPU. CPU process them.

उदाहरण

Select * from student where Roll no = 1;

- CPU has to execute it, But Data is in the memory.

- In general architecture, we only let, 2 types of Memory →

- 1.) RAM - primary memory
- 2.) H.D. - Secondary memory

- Ram is volatile, Ram works directly with CPU.

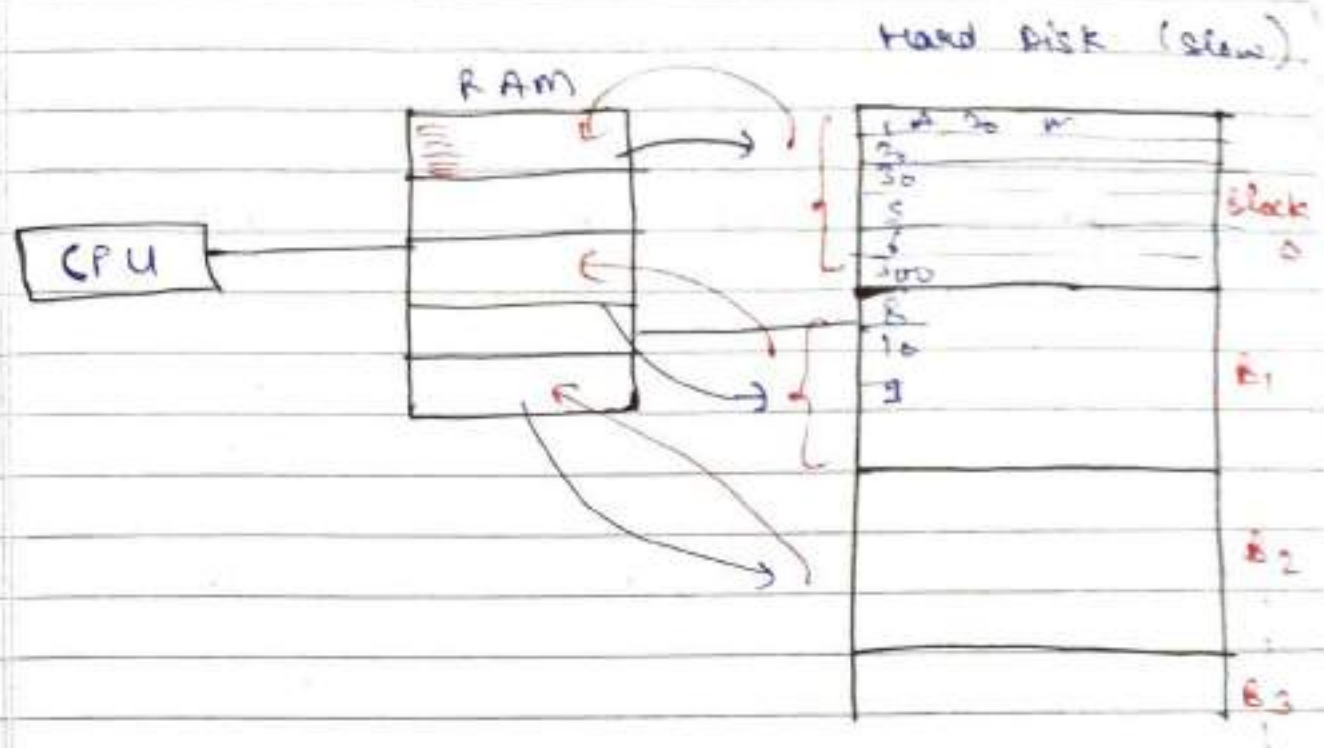
- CPU Speed → In MIPS (Million Inst. per Second)
- H.D. Speed → 10/20 Inst. per second

→ these both are not compatible with each other.

So,

- Ram comes b/w them.

- Hard speed counts in milliseconds
CPU " " " nanoseconds/pico seconds



We divide Hard Disk in blocks.
Logical Blocks (& not physical blocks)

Ex: -

Logical Drives $\rightarrow$ C Drive, D Drive, — etc.

O.S. divides hard drive into fixed sized blocks & then insert data. (Blocks / pages)

Ex: -) If we have record of 10k students
 $\hookrightarrow$ Block size is of 100 Records in Hard Disk.
 So, we need 100 Blocks in H.D. ($100 \times 100 = 10k$)

$\Rightarrow$ Now, data may also be stored in 2 ways
 1.) Sorted (ordered) 2.) Unsorted (unordered)

$\Rightarrow$ Let's

2) Unordered Data: Then, we send 1-by-1 every block of H.D. into RAM. & if we get our data, then ok. Otherwise, RAM send back that block & next blocks come into RAM. & so on.

If we get data → HIT
If we don't get → miss

(#b) So, Here, INDEXING is used.

(EH Ram H H.D. in Block not searching time not find that,)

1) We transfer data from H.D. to Ram. So, there is a transfer cost. i.e.

I/O cost

→ Input/Output.

2) Data not call the 2nd, 3rd etc. I/O cost.

3) If call more blocks, then I/O cost more. & then time also increases of searching.

So, Indexing, that we have to call min^m no. of blocks. i.e. I/O cost is reduced.

Ex: Let, a book has 1000 pages. & we have to search a page.

Worst Case → 1000

(find in last)

Best Case → 1

(find on 1st page).

Average $\rightarrow$ 500 pages.

Q. If we use Index, then the no. of pages we shuffle are decreases.

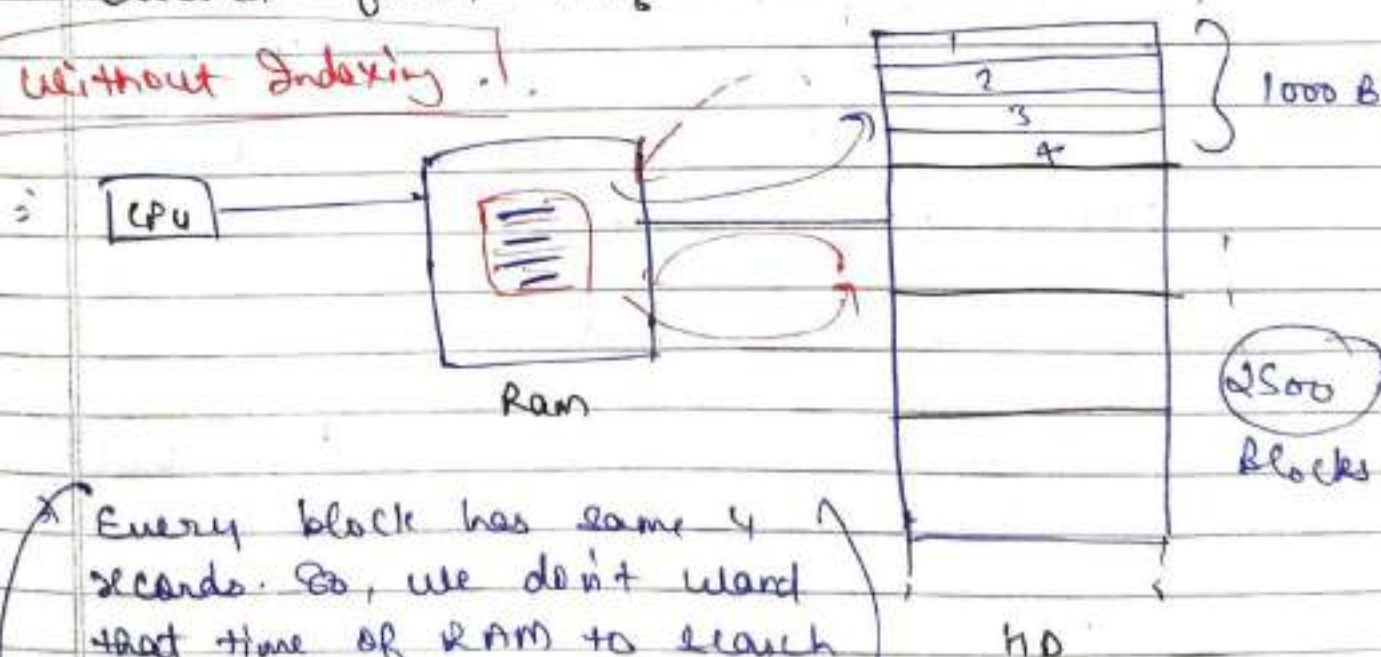
Q. (Let, Index is of 2-3 page, then just see that topic on Index & then directly go to that page).

Q2- Numerical Ex. on I/O Cost in Indexing $\rightarrow$

Q1- Consider a Hard Disk in which Block Size = 1000 Bytes, each record is of size = 250 Bytes. If total no. of records are 10,000 and the data entered in Hard Disk without any order (unordered).

Q. What is avg time Complexity to search a record from HD?

Without Indexing!



Every block has same 4 records. So, we don't want that time of RAM to search in a block bcz, i.e., always 4.

2) No. of records we can put

$$\text{in every block} = \frac{10000}{2500} = 4$$

$$= 4$$

2

$$\Rightarrow \text{No. of blocks required} = \frac{10000}{4} = 2500$$

$$= 2500$$

3) [I/O Cost :-]

Best Case = 1

Worst Case = $2500 = N$

Avg. Case = $\frac{2500}{2} = 1250 = \frac{N}{2}$

[Avg Time Complexity: $O(N)$] ch

2 This is all for unordered data.

Hence, we used Linear Search here.

3) Let, Ordered Data! :-

then, we use Binary Search here,
but it searches on sorted data
(either ascending or descending).

2 then,

Time Complexity: $O(\log_2 N)$ ch



Here, $\Rightarrow \log_2 2500$

$\Rightarrow 12$ approx

Hence,

we need approx 12 blocks in ordered data.

$\Rightarrow$, this both case are without Indexing.

$\Rightarrow$ When we used Indexing, we even need less blocks than these.

I/O cost is low.

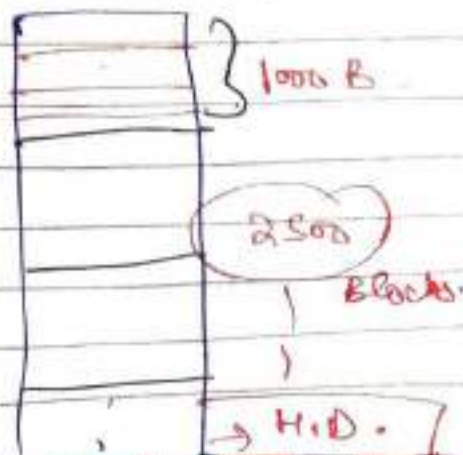
Q3. Numerical on I/O Cost in Indexing $\rightarrow$

Q1. Consider a H.D. in which Block Size = 1000 Bytes, each record is of Size = 250 Bytes. If total no. of records are 10,000 and the data entered in Hard disk without any order (Unordered Data).

What is the avg time Complexity to search a record from Index Table if Index Table entry = 20B (key + pointer).
10B 10B

1. 2500 blocks

2. Index is also stored in the H.D. When we search any data, then Index come into the Ram.



Then, we first search into the Index.

Noting

Block size in Index

=

Block size in H.O.

page

1000 B

page

1000 B

Understanding: In a book, all pages are of same size. Whether it is Index page or anything.

Index Table Entry = 20 B

(

key

+ pointer

)

10 B

10 B

key value → all value that to search are set

(Topic name), Roll no. - etc.

pointer → page No.

Each block in Index Table contains Record

=

Block Size

Index Entry size

=

50

1000

%

= 50

Hence, we can put 50 records in a block of Index.

(i.e., contains only topic name)

→ उद्दिष्ट और विषय।

key	pointer
50 Records	
50 Records	

1000 B



⇒ How we can enter records in an Index! →

1) Dense! used when unordered data.

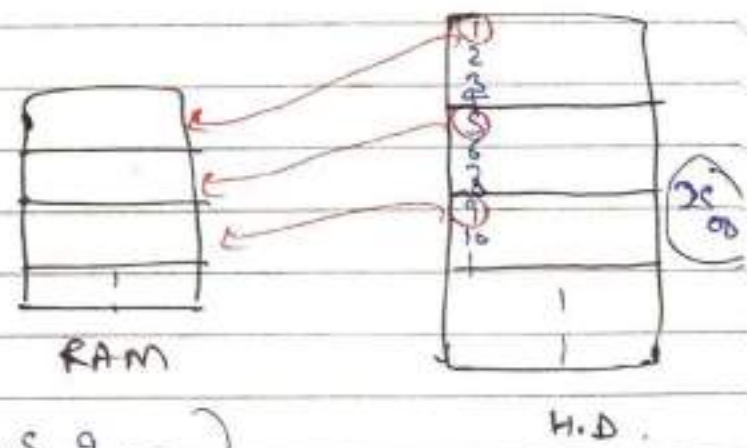
Here, we have to put all the records of H.D. into the Index.

Ex! → Here, all 10k records are to be entered in Index.

2) Sparse! used when data is ordered / sorted.

Here, we just enter 1 record from the block into the Ram.

Like,
we entered ~~the~~
Anchor data (header)
into the Index. (1, 5, 9, ...)



Ex! → So, here → we just have to enter the no. of records in RAM equal to no. of blocks in H.D.

Here, 2500 records are to be entered in Index.

⊛ Now, come to ques¹,
If our data is ordered → then,
it is Sparse →

no. of ~~records~~ in Index = $\frac{2500}{50} \Rightarrow 50$

Blocks



Date _____

Page No. _____

Then, search time $\rightarrow \log_2 50$

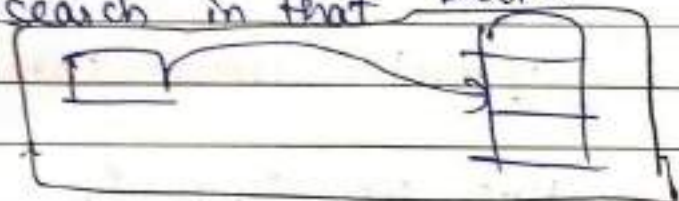
≈ 6 approx

Hence,

We have to search 6 times to find the desired page no.

then that page no. takes us to the desired block in H.D. & then, we just only have to search in that block.

So



Total no. of Search = 6 + 1

≈ 7

✓

6 search in Index.

last (1) search in H.D.

But

$\rightarrow$ Our data is unordered.

Hence, Dense $\rightarrow$

3 No. of blocks in Index = All records

Record in one block of Index

$$= \frac{200}{50}$$

Hence,

$$\approx 200$$

Searching $\rightarrow \log_2 200 + 1$

$\approx 8 + 1$

≈ 9

approx

$\log_2 2^8 + 1$

$\approx 8 + 1 \approx 9$

$2^8 = 256$

- Here, we use $\lceil \log_2 200 \rceil$ also, bcoz
 Index में ही unordered data को order/
 sorted हो जायेगा नही।

we use $\lceil \log_2(n) \rceil$ here.

94. Types of Indexed: →

1. Primary Index
2. Clustered Index
3. Secondary Index

Key → uniqueness.
 Non Key → not unique.

Ex.

Table:

ordered file	Primary Index	Clustered Index	At most 1.
unordered file	Secondary Index	Secondary Index	
	key	Non Key	

1
2
3
4
5
...

1
2
2
3
3
...



We all have
 primary Index in our Books &
 In last of Book - Secondary Index.

If we want I/O cost to be less, then we make Indexes.



Date _____

Page No. _____

Q5. Primary Index! →

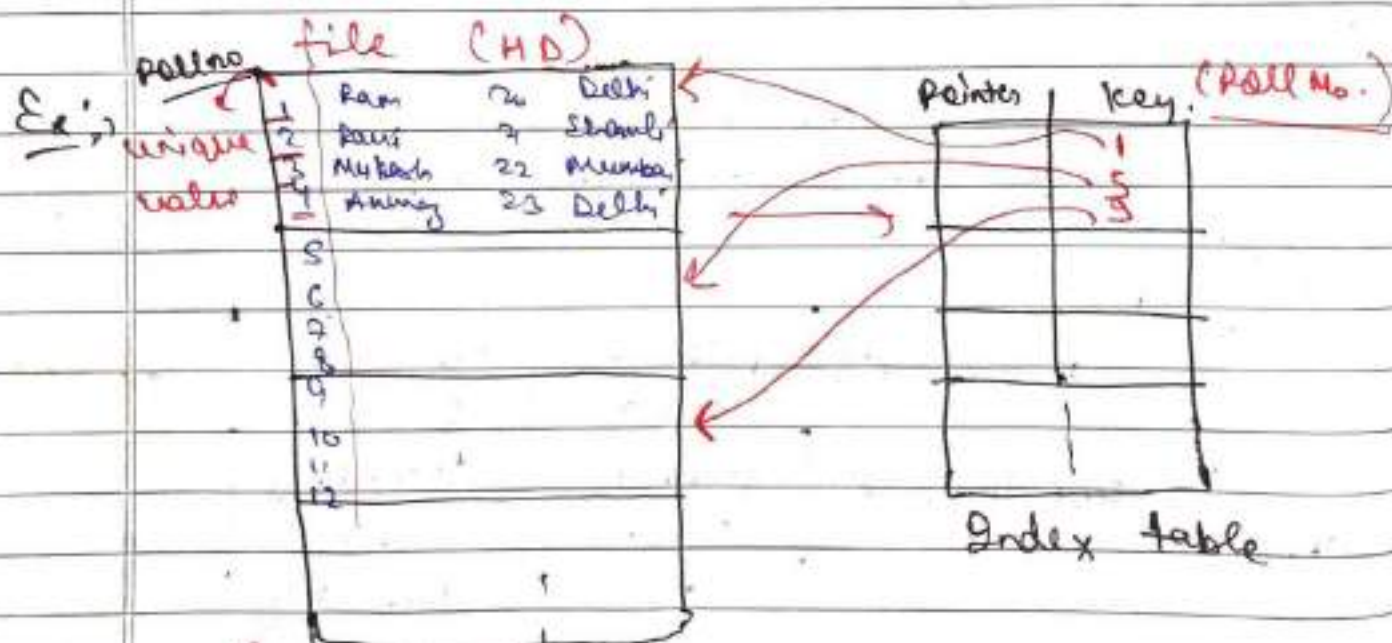
1) In a table, if we make any of att. primary key, then by default -> primary index is created on them automatically.

2) Advantage:-

1.) data is ordered.

2.) & also unique. (key value is present)

Ex: In IRCTC, we just find everything, by Train No.



↓
Data is sorted & unique

So, we use sparse here.

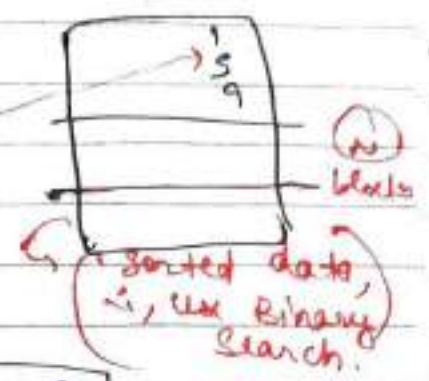
→ No. of Entries in Index Table = No. of blocks in H.D.

and, we call it Anchor (leader).

(1, 5, 9, 13, ...).

Primary Index is sparse.

Ex: 1) we have to find 3,
 then,
 we know it is found in
 block of '1'.



Hence,

$$\text{Total Search Time} = (n \log_2 N + 1)$$

where,

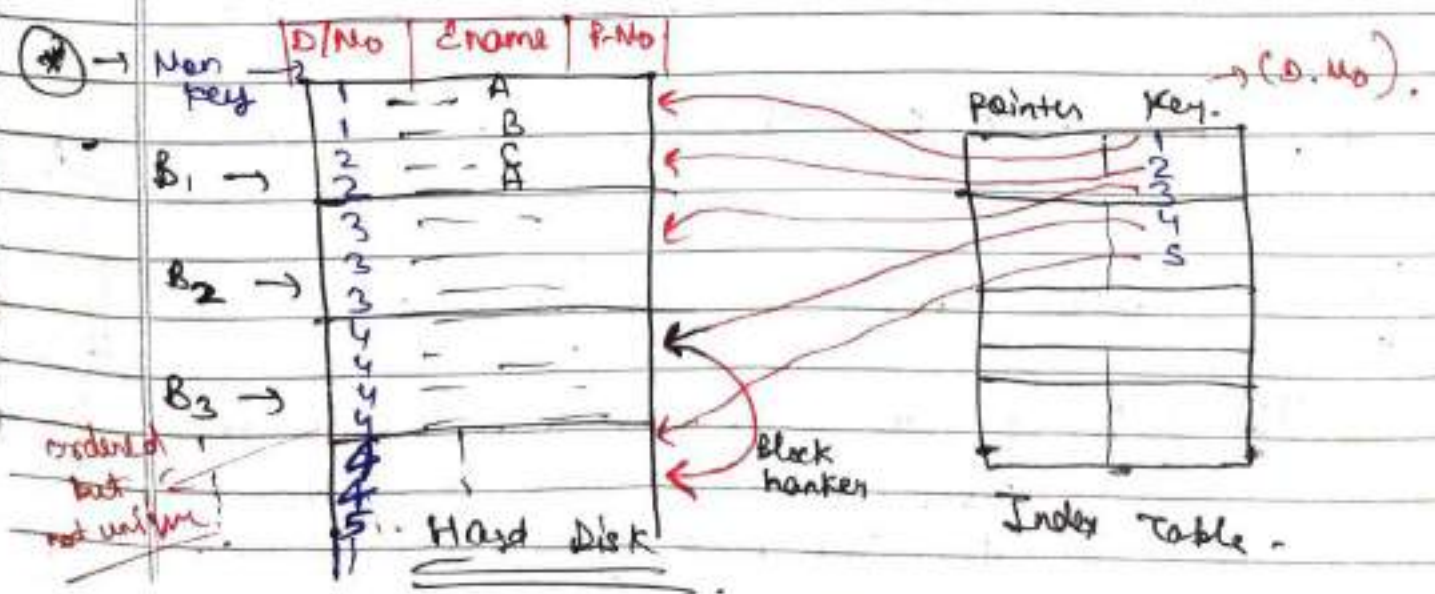
n is the No. of blocks in Index Table.

36) Clustered Index ! →

1) data must be ordered & with Non Key,
 (i.e., not unique).

i.e.,

1) value may be repeated but must be sorted.



- If our data is multiple times (not unique) in H.O. then, it also comes only 1 time in Index.

(In Index, no repetition). ^{ie,} (unique)

- # Here, it is not only in one block, some are also in next block.
So, How we point to them.

So,

Now, we use Block pointers to point to next block for same value.

- Here, Searching criteria slightly increased (↑).

✓ Clustered Index is also sparse, ^{always.} bco we don't need to make pointers for every value. $\therefore$ Repetition - there is only 1 pointer.

- In both $\therefore$ primary Index & cluster Index HT not need, bco no need. & 1 Table has only 1 primary key HT.

So,

At most 1, (both primary & cluster Index)

✓
$$\text{Total Search Time} = (\log_2 N + 1) + 1$$

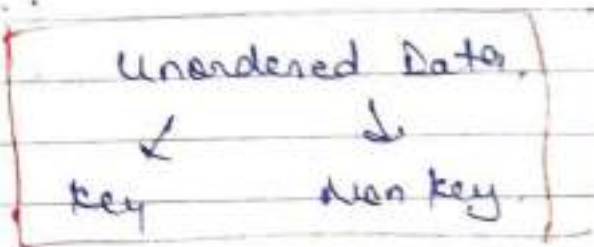
Where, N is No. of blocks in Index Table.

(These extra are $\therefore$ Block pointers. $\therefore$ for, Each block pointer, there is +1).

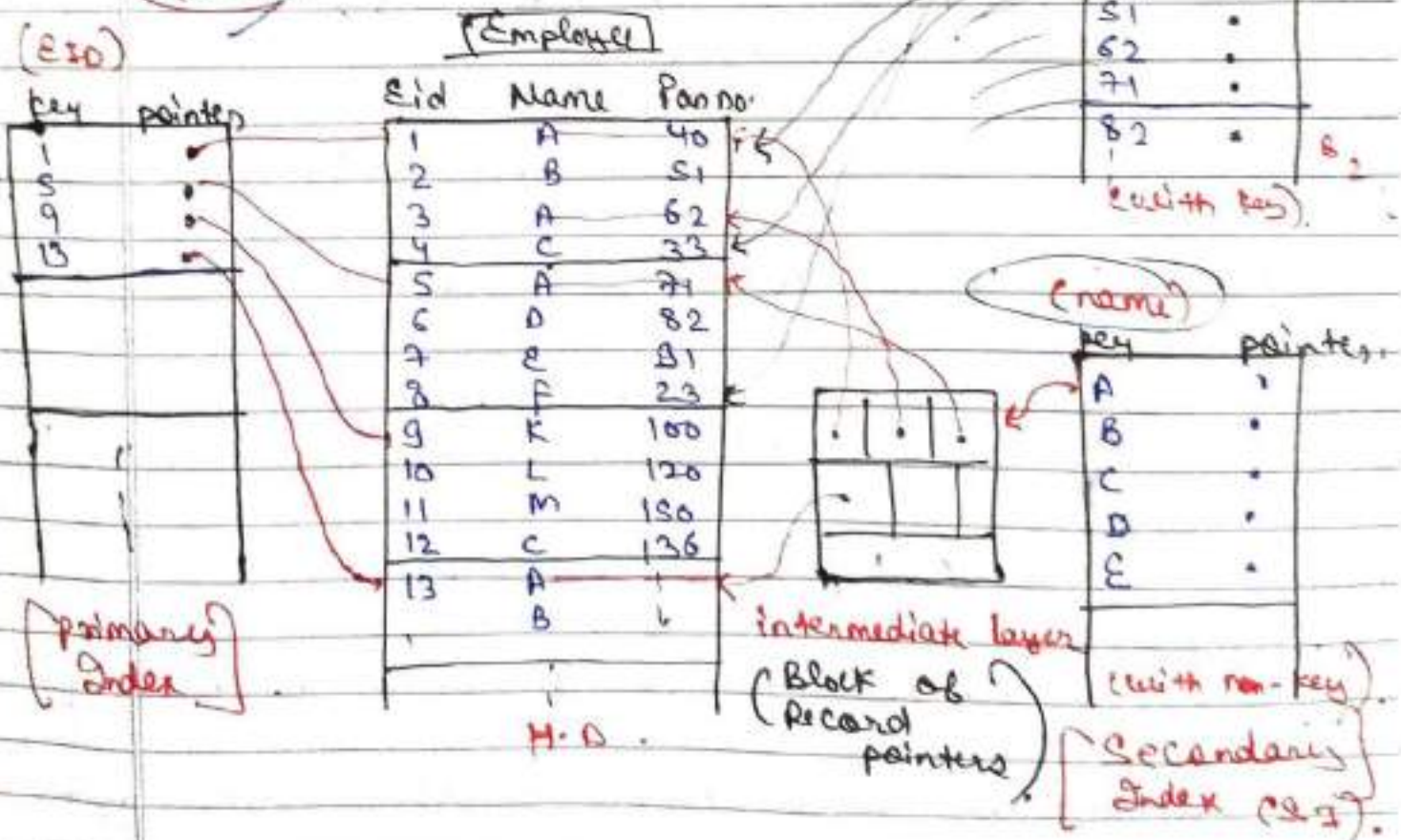
Q7) Secondary Index, (Multilevel Indexing): CS I.T.

→ (already there is an Index in database, we have to make another Index).

→ Where use S.I.?



→ Why another Index?



→ Why Another Index?

1. Let, H.D. has data of Employees table
 & data is already sorted on basis
 of EmpId (primary key). → [Unique + Not Null]

2. bcs most of the query are on EmpId.

3. Find * of Employee where EmpId = ...

then we use primary Index.
 (which is already there, formed on basis
 of EmpId.)

But, sometimes we also use
 name & pan no.

then, primary Index fails, bcs it is
 made on EmpId.

So,

we use Secondary Index here.

4. Case I when we also have key.
 then

we use PAN no.
(unique + unordered)

(As u can see in Diag.)

5. Always use sorted values in Index.

Then, we apply binary search → Time Saving.

* Index is always sorted, + unique.



Date _____
Page No. _____

41-

→ Secondary Index on key, is always DENSE.

here, we don't have any anchors (leaves) here.
like in primary In.

→ These values are not sorted. Hence, we put them sorted in Index. & write all the records of H.D. in Index Table.
So, Dense.

ie,

[No. of records in Index = No. of records in H.D.]

→ Search time: $O(N + 1)$

Where,

N is the no. of blocks in Index Table.

⊗ Case II When we have Non key with unordered data.

It is the worst case.

then,

we use NAME,

↳ (neither ordered, nor key)
(unique).

↳ (Secondary Index in B+).

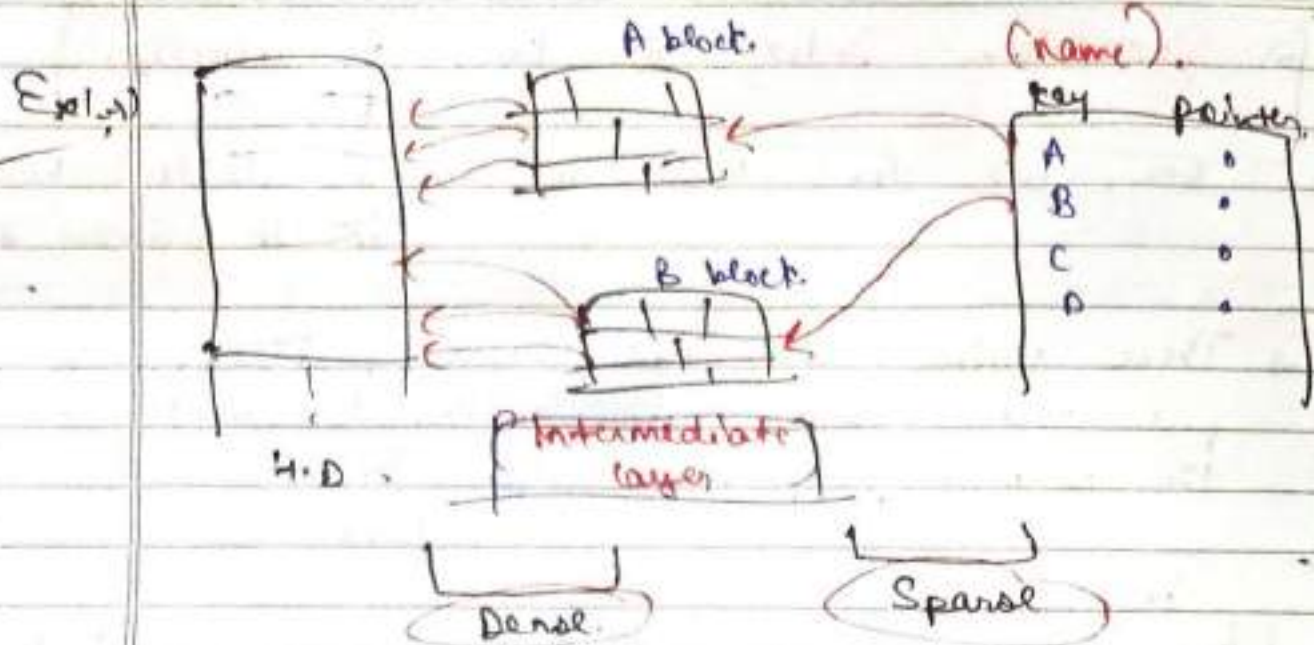
→ In Index, we don't have to take multiple values of A. (Only once).

But, A is many times in Main Attr.

So,

here we made an Intermediate layer.
(It is a block of record pointers).

book to find Secondary Index (S.I.) मिल जाता है।



2) Hence, It is Mix of Dense & Sparse.

So,

3) We can say it DENSE Overall.

4) Time Complexity : $\log_2 N + 1 + 1$ to ...

(And this is only for 'A'.
It may be more than
 $T \log_2 N + 2$.)

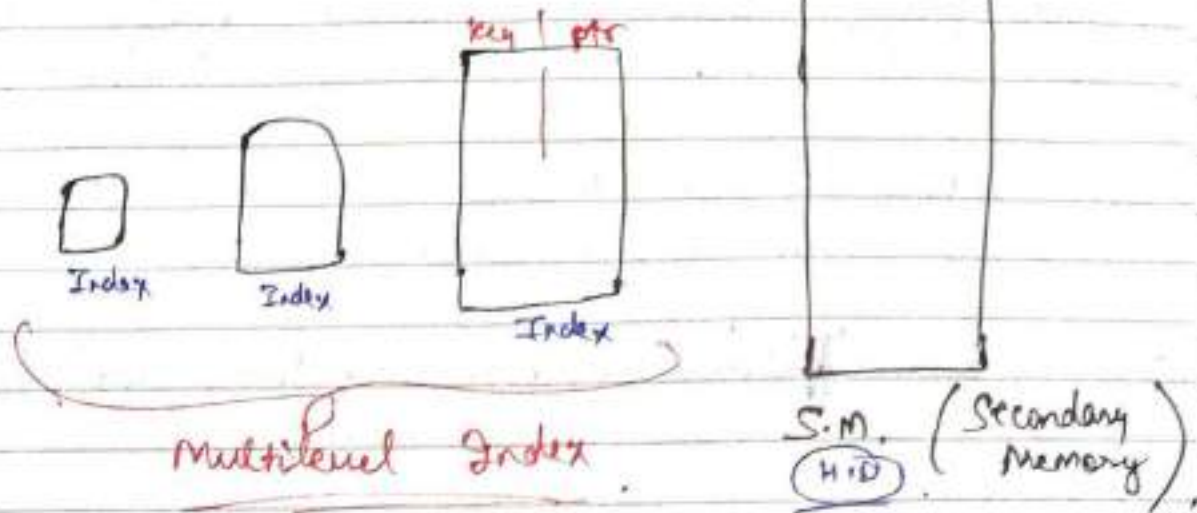
↓
(Time for intermediate layer)

Ⓢ Secondary Index is always dense.

98. Intro to B-Tree & its Structure :-

1) B-Tree (Dynamic Multilevel Index) :-
(Balanced Tree)

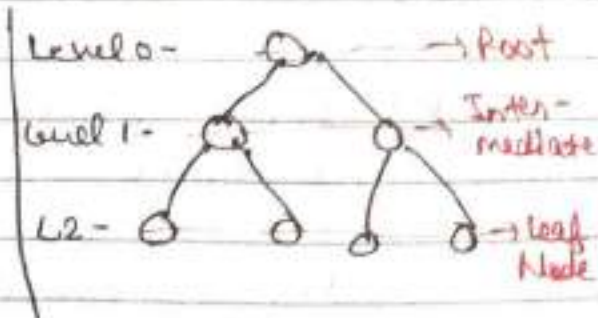
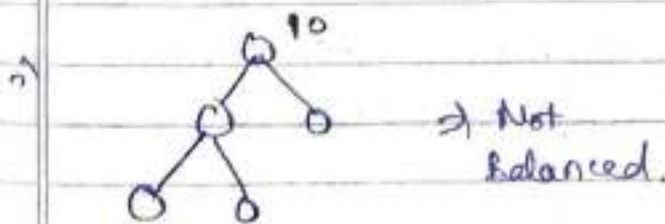
graph $\rightarrow$ cycle
Tree $\rightarrow$ acyclic.



- $\Rightarrow$ In this, it is hard to manage. b/c, If we insert data in S.M. then we have also insert in all the indexes & same for delete.

④ Balanced Tree (B-Tree!)

- $\Rightarrow$ means, all the elements are at the same level of leaf Node.



- $\Rightarrow$ Block pointer (B.P.) or Tree pointer! $\rightarrow$ When node denotes his child. Then, we use Block pointers (B.P.).

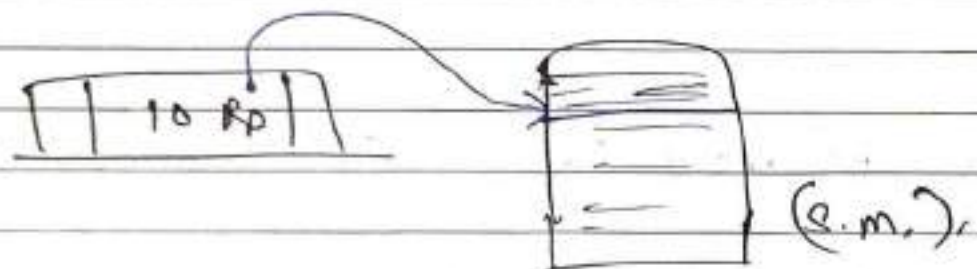
*Note! A node can contain multiple values here.

→ Keys! - that on which basis we have to search. Searching Criteria - key.

(key at EN Nodes in 3rd stage)
(we can insert multiple keys in a node).

→ Data pointer (D.P.) (or) Record pointer (R.P.)! →
These all correspond to keys.

→ This record pointer points to in the Secondary Memory (Hard Disk) where record is present of that key.



(*) Keys = Record pointers.

(*) Block pointer depends on the how many children are there of a Node.

No. of children = No. of B.P.

(*) Order = p (of a B-tree)
= Max. no. of Block pointers.

Order = p = Max. no. of children a node can have.

(*) Keys = $p-1$ ^{max}
 $R_p = p-1$

Min Keys = $\lceil \frac{p}{2} \rceil - 1$

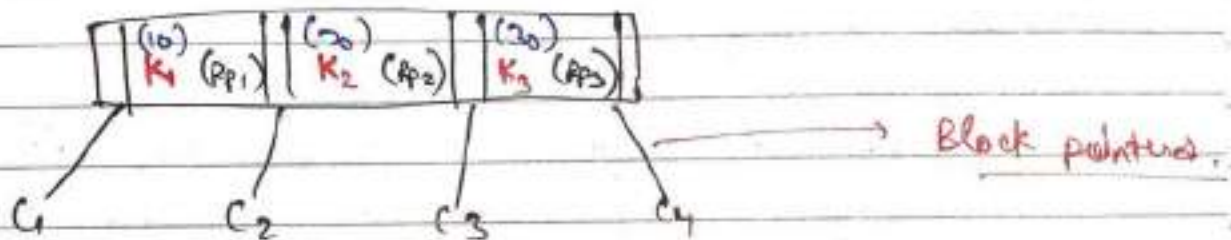
Table:-

		Root	Intermediate Node
Children of	max	p	p
	min	2	$\lceil \frac{p}{2} \rceil$

→ Ceiling value

Ex:-
Let

$p = 4 = \text{order of a Tree.}$



We always insert keys in the sorted Order. (bco, it follows the prop. of Binary Search Tree).

* * *

Q9. Insertion in B-Tree ! →

Q1: Insert the following keys into B-Tree, if order of B-Tree = 4.

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

⇒ Order = 4 = max. no. of children.

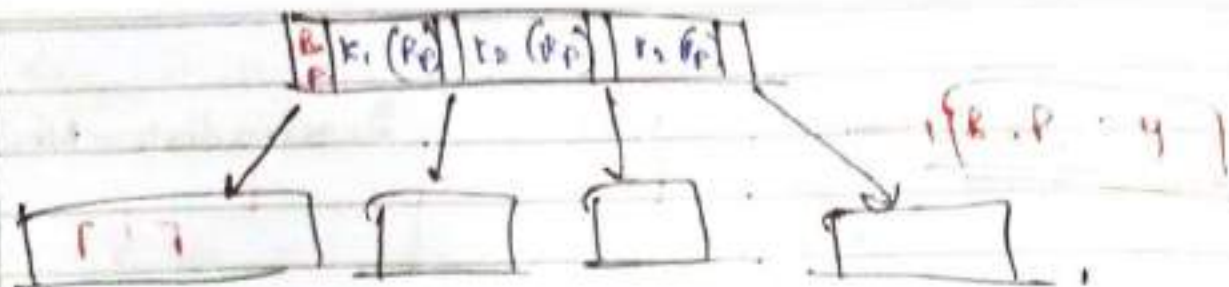


Max keys = $p - 1 = 3$

Min keys = $\lceil \frac{p}{2} \rceil - 1 \Rightarrow 1$



2)



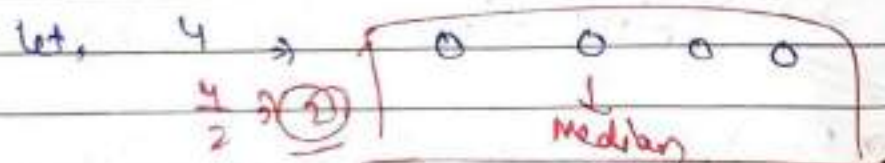
⇒ We follow the prop. of Binary Search Tree in Insertion.

① In Binary Search Tree,

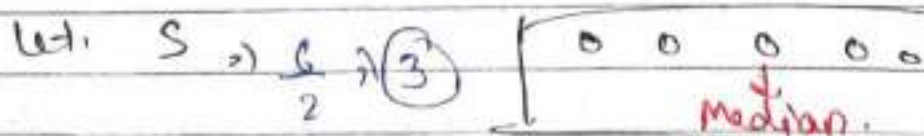
[Root → Left = Small Element
Root → Right = Big Element]

② Overflow sid se, Median chahiye the break chahiye karna,

→ When, n - Even $\Rightarrow \frac{n}{2}$



→ When, n - odd $\Rightarrow \frac{n+1}{2} = \text{Median}$



⇒ and, shift Median to upward (↑), & shift Left & Right Elements below it. All elements 1 & 2, how we break.



Q10,

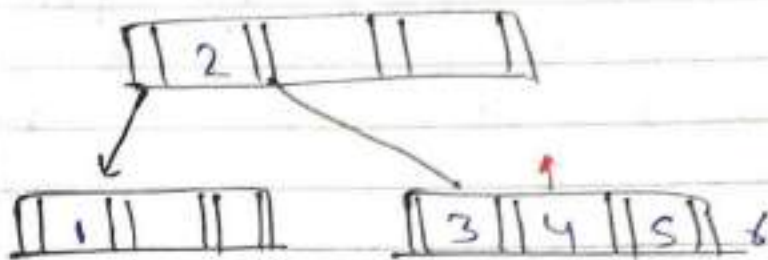
1, 2, 3, 4, 5, 6, 7, 8, 9, 10

a)



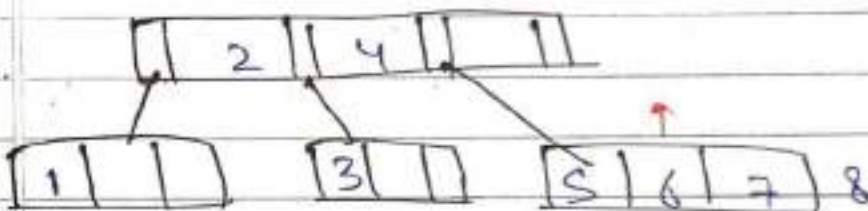
(overflow) $\Rightarrow \frac{5}{2} \Rightarrow \frac{3}{2} \Rightarrow 2$

b)



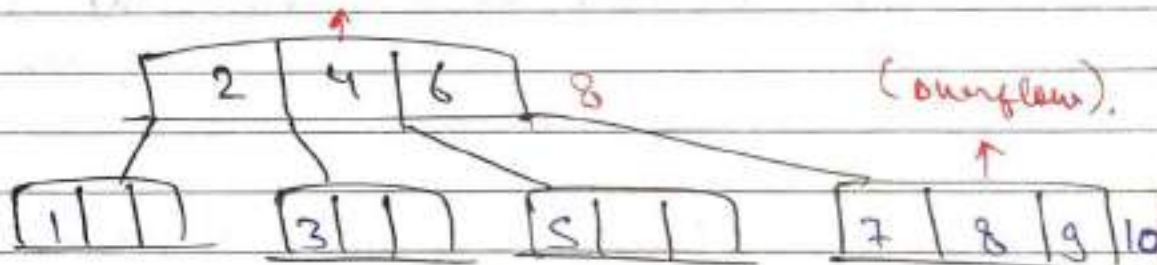
(overflow)

c)



(overflow)

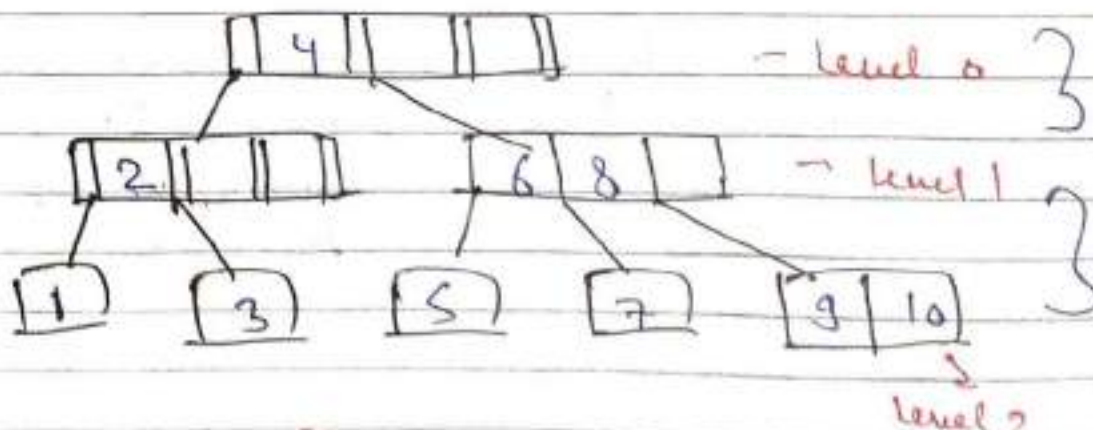
d)



(overflow)

Now,
4 Max
child

e)



Level 0

Level 1

Level 2

Height = 2

f)

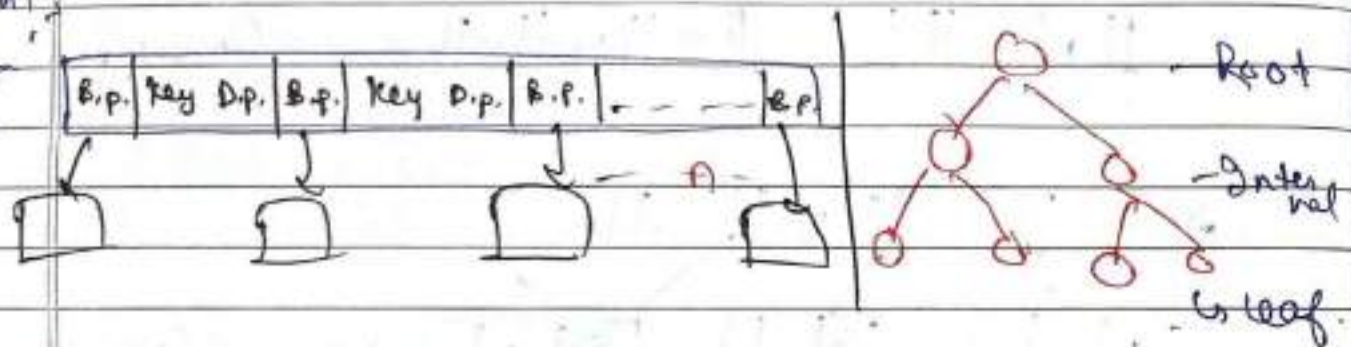
height = 2
levels = 3

↓

100. How to Find Order of B-Tree! $\rightarrow$

Q. Consider a B-tree with key size = 10 bytes, block size 512 bytes, data pointer is of size 8 bytes and block pointer is 5 bytes. Find the order of B-tree.

Solⁿ:

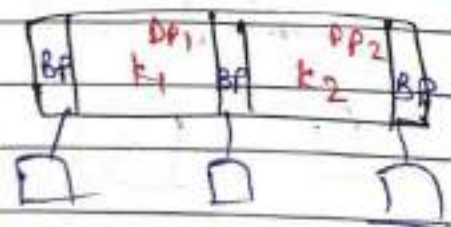


A lot, In a node / Block has 'N' no. of block pointers.
 So,

$$\text{Total Size of B.P.} = n \times (\text{size of 1 B.P.})$$

$$= n \cdot B_p$$

If n B.P.s. (as children a node can have) then, $(n-1)$ keys, & Record pointer.



$$n \times B_p + (n-1) \text{ key size} + (n-1) R_p \leq \text{Block size (or) Node size.}$$

2) $n \times 5 + (n-1)(10+8) \leq 512$

$5n + 18n - 18 \leq 512$

$23n \leq 530$

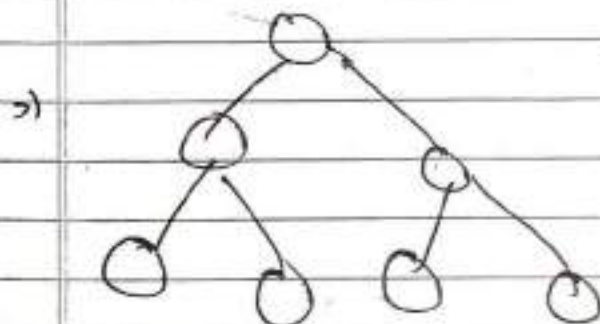
$n \leq \frac{530}{23}$

$n \leq 23.04$

$n=23$ Ans — Max. 23 children.

3) Every b.p. represent a children.
800,

Max. Order = 23.



Max	Min	Children
23	2	
23	$\lceil \frac{23}{2} \rceil \Rightarrow [11.5]$	<u>12</u>

Leaf has no children.

Keys = Order - 1 22 Ans

101 D/B B-Tree & B+ Tree ! $\rightarrow$

* B-Tree :-

1) Data is stored in leaf as well as internal nodes.

2) Searching is slower, deletion complex.

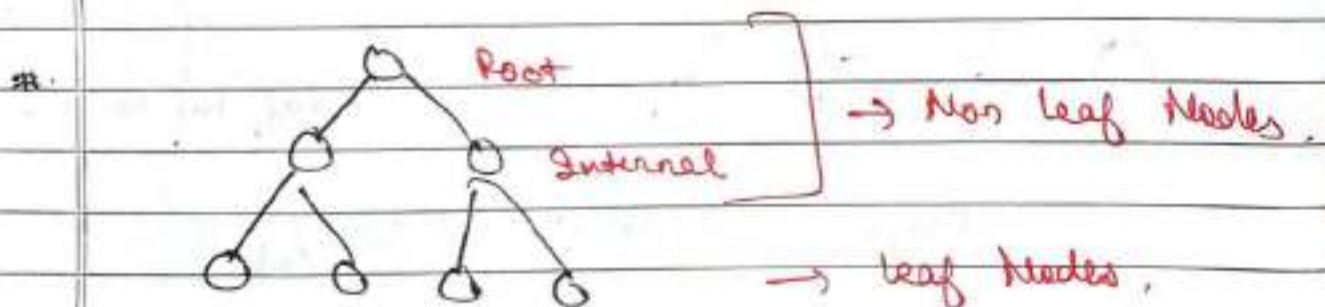
- 3.] No Redundant (Duplicate) search key present.
- 4.] Leaf Nodes not linked together.

② B⁺ Tree :

- 1.] Data is stored only in leaf nodes.
- 2.] Searching is faster, deletion easy.
(Directly from leaf node).
- 3.] Redundant keys may present.
- 4.] Linked together like linked list.

#. We use B & B⁺ Tree, to put Index Record. These trees actually contain Index Record.

Index Record $\left\{ \begin{array}{l} \text{Key} \\ \text{Data pointer (Record pointer)} \end{array} \right.$



In B tree, the structure of every node is same. (Either root, internal or leaf).

Leaf node, has no children. So, when's the role of B.P. Then, they point to NULL.

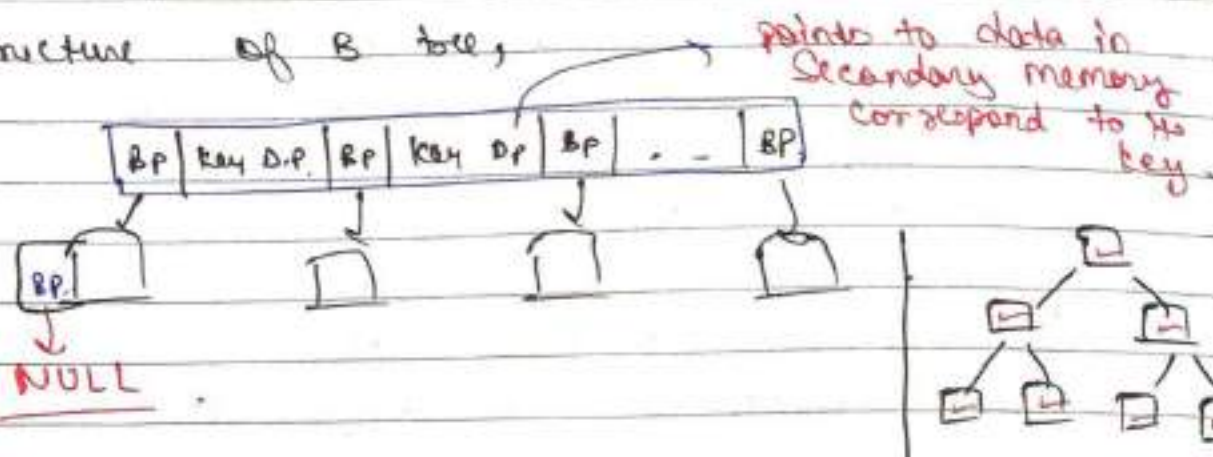
B.P - Block pointer.



Date _____
Page No. _____

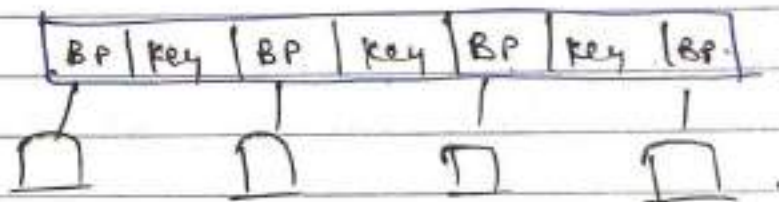
(51)

Structure of B tree,



Structure of B⁺ tree :-

⇒ Non leaf structure → or Internal Node →.



→ There is no Data pointer (D.P.) in Internal Node structure / Non-leaf Node structure.

R.P / D.P - (X)

Hence,

We have more space in Internal node. Hence, we can create more children & put more keys.

So, that's why,

B⁺ tree → Breadthwise longer.

B tree → Depthwise longer.

∴ (i.e., less no. of children breadthwise as compared to the B⁺ tree).

So, depth more.



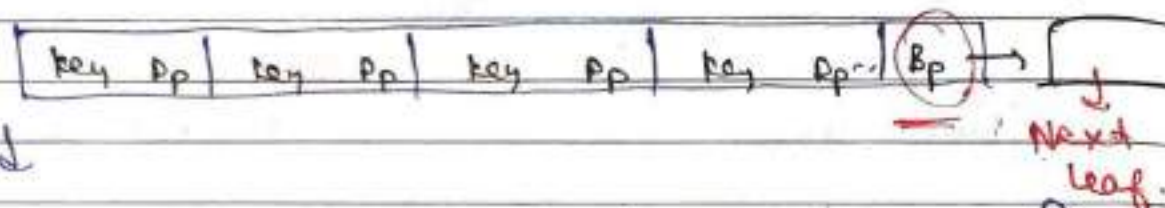


#. So, that's why searching is slower in B tree as compared to B+ tree.

#. B+ tree Structure →

Leaf Node Structure →

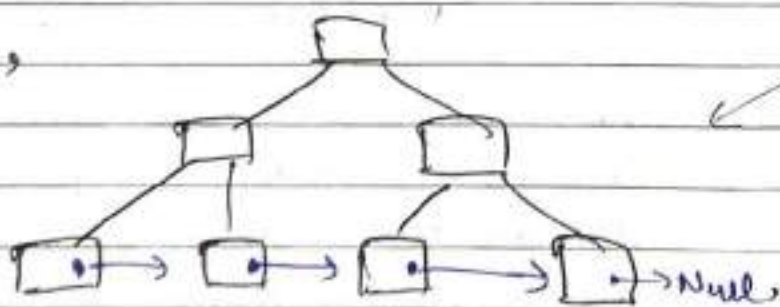
(Structure of leaf & Non leaf Nodes are different.)



⇒ (There are no Block pointers in leaf Node except 1 in last which points to Next leaf.)

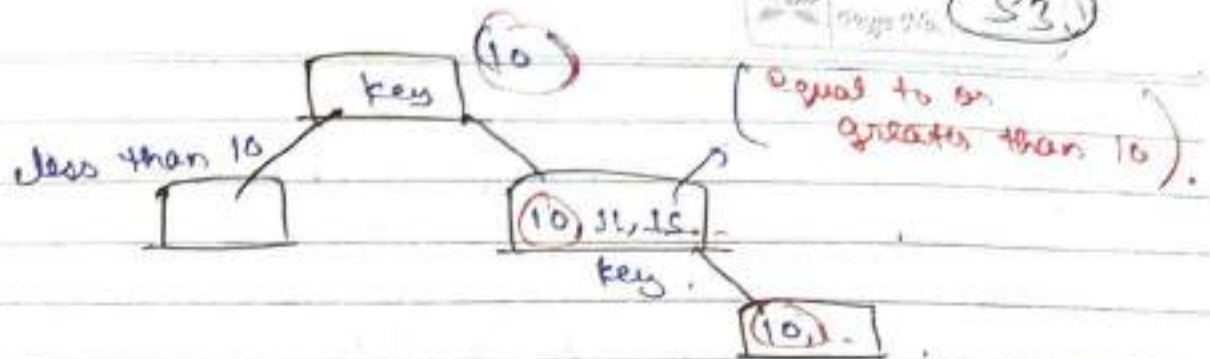
#. Both, B & B+ are balanced, i.e., all leaf nodes are at same level.

#. In B+ structure,



⇒ In this, ^{less} value ~~key~~ to key is in left node & the greater value to key are in right node. (But here, on right side, we also have to put the key ~~value~~ with its greater values).

⇒ i.e.,



Reason: $\therefore$ bcz, we only search in leaf Node, bcz only leaf Node has all keys with Data pointers are present.

Also, we also have to search the D.P. of the key, so, that's why we also carry the key value upto leaf Node.

Searching is that's why faster in B+ tree, bcz, have to search only in leaf Node.

bcz of this, Redundant keys are also present in B+ tree.

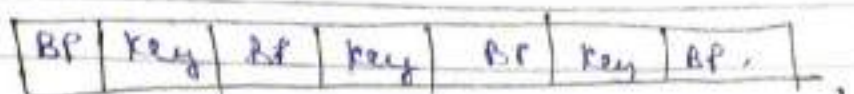
Leaf Nodes are also linked together.

102. Quesⁿ an Order of B+ Tree $\therefore \rightarrow$

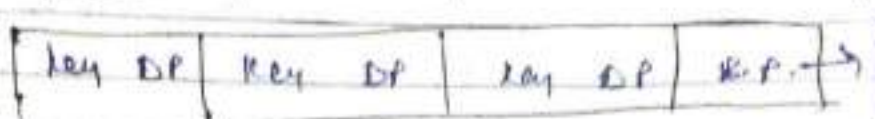
Q! Consider a B+ Tree with Key size = 10 bytes, block size = 512 bytes, data pointer = 8 bytes & block pointer = 5 bytes. What is the order of leaf & non leaf node?

Solⁿ:

Non leaf $\therefore \rightarrow$



Leaf Node $\therefore \rightarrow$





→ Non-leaf! →

$$n \times BP + (n-1) \times \text{key} \leq \text{Block Size}$$

$$\rightarrow 5 \times n + (n-1) \times 10 \leq 512$$

$$5n + 10n - 10 \leq 512$$

$$\rightarrow 15n \leq 522$$

$$n \leq \frac{522}{15} \quad 34.8$$

$$\boxed{n \leq 34.8}$$

$$\boxed{n = 34}$$

$$\rightarrow \text{Order} = 34$$

[Max BP.] or (Max children possible)

→ leaf! →

Let x pairs

$$x (\text{key} + \text{pp}) + BP \leq \text{Block Size}$$

$$x (10 + 8) + 5 \leq 512$$

$$\rightarrow 18x \leq 507$$

$$x \leq \frac{507}{18} \quad \boxed{28.18}$$

$$\boxed{x \leq 28.18}$$

$$\boxed{x = 28} \quad \text{Order} = 28$$

Note: Order of a leaf node in B+ tree is the no. of (key, p.p.) pairs.



103.

Immediate Database Modification! → (Log Based Recovery Methods).

→ Immediate means jitni, jitni CH!

Ex 1)

A = 100	200
B = 200	400

Hard Drive

 T_1

R(A)

 $A = A + 100$

W(A) - 200

R(B)

 $B = B + 200$

W(B) - 400

Commit.

Transaction Log $\langle T_1, \text{Start} \rangle$ $\langle T_1, A, 100, 200 \rangle$
old new $\langle T_1, B, 200, 400 \rangle$
old new $\langle T_1, \text{Commit} \rangle$ Redo

→ जैसे ही Ram के store value (200) हुई, उसी Time में ही Database में भी $(A=200)$ करते। फिर Commit हुई।

→ i.e., At time, when we write in memory (RAM), at same time also update in the H.D. we don't wait for commit.

→ But, when we see in Trans. Log,

Our Recovery Manager sees the log & check whether T_1 has both start & commit or not. & If yes, then it Redo

→ Redo, means saves the latest value in the Database. i.e.

→ (Recovery Manager don't see value in the H.D., it only sees in the Trans. log & check (starts & commit) & then saves in the H.D. database. & if already saved, then Over-write & fixed them.

→ But, If

		<u>Transacⁿ log</u>	
Ex. 1	A = 100 200 B = 200 400 Database	T ₁	< T ₁ , start >
		R(A)	< T ₁ , A, 100, 200 >
		A = A + 100	old new
		W(A) - 200	
		R(B)	< T ₁ , B, 200, 400 >
		B = B + 200	
		W(B) - 400	<u>UNDO</u>
		fail	

→ Here, Recovery Manager don't find commit in Trans. log. so, he UNDO.

→ UNDO, means it saves the old value. But, in database there are updated values now. so, Recovery Manager takes the old values from the Trans. log & saved them in the Database.

- In Immediate, we store both old & new value.
 But,
 In Deferred, we only store new value.
 (i.e. only REDO, not UNDO)

→ that's why

Immediate Database Modification is known as UNDO-REDO strategy.

Ex: → Transac log →

$\langle T_1, \text{start} \rangle$
 $\langle T_1, A, 1000, 2000 \rangle$
 $\langle T_1, B, 5000, 6000 \rangle$
 $\langle T_1, \text{Commit} \rangle$

REDO

$\langle T_2, \text{start} \rangle$
 $\langle T_2, C, 700, 800 \rangle$
 old.

UNDO

104.

Ques on DBMS basic Concepts & Data Modelling.

Q:- Let, $R(a, b, c)$ and $S(d, e, f)$ be 2 Rel's.
 'd' is foreign key of S that refers to primary key of R. Consider 4 operations on 'R' & 'S'.

- i) insert into R
- ii) insert into S
- iii) delete from R
- iv) delete from S

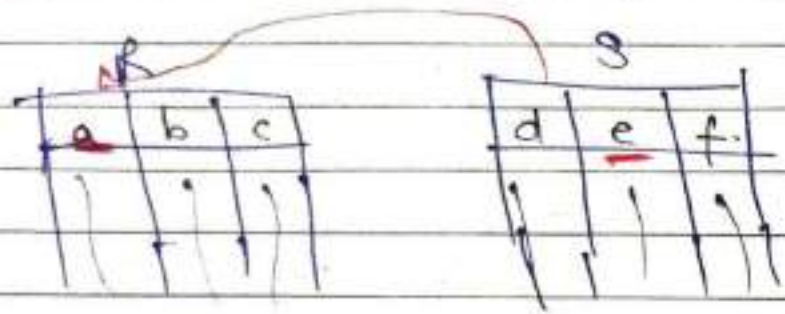


→ Which of these can violate Referential Integrity?

- a) i & ii
- b) i & iii
- ~~c) i & iv~~
- d) i & iv

→ Ans:

Explain:



→ Understand form: → An Intelligent student who delete the record is notified that others records are not delete then violation.
(If they don't delete → then, violation.)

(2) The view of total database content is

- ~~a) Conceptual view~~ b) Internal view
- c) External view d) physical view.

→ This is on 3 schema architecture.

* Internal view & External view,

external view is basically related to outer view.

→ User who total view not want.

[User has partial view (data only he need)]



→ physical view at ~~most~~ 3rd level view at ~~21211~~.
(Database storage related)

→ In partial case, External view is Ans.

(3.) In Relational Model, Cardinality is →

- | | |
|-------------------------|------------------------------|
| <u>a)</u> No. of tuples | <u>b)</u> No. of attributes |
| <u>c)</u> No. of Tables | <u>d)</u> No. of Constraints |

→ If degree, then no. of attributes.

The no. of rows in a Table, called as Cardinality.

(4.) _____ Constraint is used to maintain consistency among tuples in two relations.

- | | |
|---------------------------------|----------------------------|
| <u>a)</u> key | <u>b)</u> domain |
| <u>c)</u> Referential Integrity | <u>d)</u> Entity integrity |

→ Domain basically deals with that which type of data we put into Table.
(Integer, varchar, character, etc.)

→ Entity integrity, related with primary key.

→ key deals with uniqueness.



Date _____
Page No. _____

105. Imp. Ques' on Advance DBMS :-
(Big Data & Data Warehouse)

Q-1: What do data warehouse support?

- a) OLAP
- b) OLTP
- c) OLAP & OLTP
- d) Operational Database.

→ Data Warehouse, where we integrate data at one place from all diff sources.

Ex:- Big BAZAAR, - their outlets mostly found in Every City.

↳ (at the end of day, all data integrated from all cities & kept) i.e. Data Warehouse Se.

- We store this data, to analyse on it later. So that, they can also play ADS - to diff users acc. to their data of purchasing items.
(Apply Mining algorithms on these data)

- OLAP → online Analytical processing.
- OLTP → online Transacⁿ processing.

↳ It works on Current Data.

→ (that where to Remove store, where user new store, remove or add items) — all these based on Analysis.

② Hadoop is framework that works with variety of related tools. Common groups include —

- ~~A~~ Map Reduce, Hive & Hbase
- ~~B~~ Map Reduce, MySQL and Google Apps
- ~~C~~ Map Reduce, Hadoop, Iguana
- D Map Reduce, Hadoop, Truport.

→ Hadoop, basically work on Big Data.
(It is a Tool of Big Data).

↳ Like Google, Facebook. Big data deals with the multiple petabytes of the data. Now, to process this much of data. We don't use normal tool like SQL Server, Oracle. We use Hadoop. (Yes, this data is also unstructured).

→ (Hadoop Ecosystem / Framework includes many small tools like map reduce. (used to process the data, reduce the data (divide) & then works on bench processing — i.e., works on Multi-processing).

→ Hive → (If we write to SQL commands in Hadoop, then use Hive).

→ Hbase → (helps in data storage).

→ also tool like Zookeeper, flume, pig, etc.

2) Google App, it is part of cloud.

3. All of the following accurately describe Hadoop, except:

- A) open Source we also (listed in apache, install it)
- ✓ B) Real time ~~data~~.
- C) Java based
- D) Distributed Computing approach

→ Hadoop, It is basically batch processing.
Mean we first need data & then we analyse on it.

→ (In Real time, we use SPARK).

4. Which of the following does not comes under five V's of Big Data?

- a) Volume (how much amount of data)
- b) Velocity (with which velocity, data ↑)
- c) Variety (Structured, unstr, Semi-struct)
- ✓ d) Visualization ~~is~~

To Note:- Let, it there is 'x' amount of total data.

where, x → is all data on Earth, then

(80% - 90%) of x is created in last 5-6 years. Mean,

Data is increasing with so much speed.

Unstructured → photos, videos,
Semi-structured → XML based data.

- Value → (ie, value of our data).
Veracity → (means, trustworthiness).

↓
(how much capable our data to believe on it.)

→ 5 V's of Big DATA: →

- 1.) Volume
- 2.) Velocity
- 3.) Variety
- 4.) Value
- 5.) Veracity.

→ Visualizing, is a part of Data Analytics. Where we visualise our data with help of graphs (pie chart, bar chart, - etc.)

106.

Deferred Database Modification: →

→ This topic comes under log based Recovery.

↓
(If there is any failure inside our system, then either we can recover that system or not).

→ log, is basically a file (small sized file) in which we store our actions. that (What Trans. performs, we stored in logs)

(deferred $\rightarrow$ delayed, postponed, belated)

- (like, History in our Browser)
- When our system fails, to recover Trans.
→ (either we have to Roll back them
or Modify them).

→ We do that, by seeing the log.

→ 2 Methods, by seeing the log $\rightarrow$

- 1.) Deferred Database Modification
- 2.) Immediate

(24.12)

Database (M.D.)	T_1	Transac ⁿ Log
A = 100 200 B = 200 400	R(A) A = A + 100 W(A) - 200 R(B) B = B + 200 W(B) - 400 Commit	< T ₁ , <u>Start</u> > < T ₁ , A, 200 > new value. < T ₁ , B, 400 > new value. < T ₁ , <u>Commit</u> > <u>Redo</u>

- Here, it don't update in Database hand-to-hand. It updates in Database after Commit, (i.e., deferred - late, postponed).

After commit, database updates.

- How we Recovery in Deferred? Let's say system fails after Commit.
- So, when Recovery Manager comes, it first check trans. log file & check (Start & Commit) in it.

Then, Recovery Manager.

REDO

Means, update the new values in the database.

(Let, ^{due to} fail $\rightarrow$ our database also not there).
then,

Recovery Manager puts the A & B value from log in database.

A = 200
B = 400

(If it already in database, then overwrite it).
or wipeout

Case II

A = 100
B = 200

T₁
w(A) = 200
w(B) = 400
* fail.

Transacⁿ log
<T₁, start>
<T₁, A, 200>
<T₁, B, 400>

Roll back

Now, fails before Commit. So, now database value is same as before.

Now,

after failure occur, when Recovery Manager opens the log file. $\rightarrow$ It sees T₁ starts but not commit. So, Here Recovery Manager don't do anything. He simply Roll back.

Means, पुराना value out हो रहा, उसे out open करने की विचार नहीं है।

→ In Deferred Modification also known as No UNDO/REDO method. =

Ex.:-

< T ₁ , Start >	}	<u>REDO</u>
< T ₁ , A, 200 >		
< T ₁ , B, 400 >		
< T ₁ , Commit >		
< T ₂ , Start >	}	<u>(No Action)</u>
< T ₂ , C, 500 >		

Roll back at Commit

* In Deferred, we store only new values.

107 Like Command in SQL! →

1) We use Like Command, generally to Search the Data.

- Q:-
- 1) Find Employee detail whose name starting with 'A'.
 - 2) Find Emp detail whose name ending with 'n'.
 - 3) whose name contains 'ee'.
 - 4) whose name contain 'a' in 2nd place.
 - 5) whose name contain 'a' in 2nd place.
- & name should contain total five Characters.

% → Any value (anything) change it.
In this 'o' character is not, it's problem not.

5.1

Emp.	
30	Narve
1.	Varun
2.	Arun
3.	Ravina
4.	Anrit
5.	Ranjeet
6.	Ajeet

% → any value
in length.

— → reserved for
a value.

1.) Select * from Emp where name like 'A%';
Output → Arun, Anrit, Ajeet

2.) — like '%n';
Output → Varun, Arun

3.) — like '%ee%';
Output → Ranjeet, Ajeet

Note: If Name → eean, Arunee — any thing
then, these also comes, but since 'ee' we need.

% → also 'o' character include.

4.) — like '_a%';
Output → Varun, Ravina, Ranjeet.

5.) — like 'a_____';
Output → Varun

In, C++
Here

⇒ =
⇒ :=

11-108



Date

Page No.

108:

Basic PL-SQL programming with Execution :-

→ Program 1: Find the Sum of 2 numbers.

```
⇒ declare
a int;
b int;
c int;

begin
a := &a;
b := &b;
c := a + b;
dbms_output.put_line('Sum of a and b = ' || c);
end;
```

Declarative.

Executable part

// value given by user
(to take input from user)

// cant in C++
// 3 in C++

① Program 2: Greatest of 2 numbers →

```
→ declare
a int;
b int;
begin
a := &a;
b := &b;
if (a > b)
then
dbms_output.put_line('a is greater'); //a
else
dbms_output.put_line('b is greater'); //b
```

SQL live में हमें value रखने की देनी पड़ती है
इसमें बाद में user value नहीं दे सकता।

(69)

end if;
end;

(both code works fine). ✓

109. PL-SQL :- (while, for loop) :-

program 3 :- for loop :-

```
→ declare
  a number (2);
begin
  for a in 0..10
  loop
    dbms_output.put_line (a);
  end loop;
end;
```

// = 0 to 10.
// By default,
increment of 1 by 1.

program 4 :- while loop :-

```
→ declare
  a int;
  b int;
begin
  a := 0;
  b := 10;
  while a < b
  loop
    a := a + 1;
    dbms_output.put_line (a);
  end loop;
end;
```

// print from a to b.

Output → 1 to 10

then,
→ output → a to 9
(Now, first printing
done increment
→ output → 10)

4 In oracle sometimes there shows
error in output. - then write.

Set ServerOutput on

// but in Live SQL
it is By default

Code

⇒ Both code works fine. ✓

110. Single Row & Multi Row functions in SQL :-

1. Single Row! :-

If our func. is applicable on single row, and only apply on single row.
& gives an ^{single} output corresponding to that row. → then, it is single row func.

2. Multi-Row! :- func.

func. that apply on more than one row,
& gives an ^{single} output corresponding to all these rows.

- | | |
|-------------|---------------------------------------|
| ⇒ Round | (Round-off value) |
| ⇒ Mod. | (gives remainder after division) |
| ⇒ lower | (convert a string into lower letters) |
| ⇒ initcap. | (Capital the initial letter) |
| ⇒ Concat | (Add 2 string & make 1) |
| ⇒ LPAD/RPAD | (for left & right padding) |
| ⇒ NVL, NVL2 | (to take care of NULL values) |

SQL functions -

Single-Row funcⁿ

Number func.

- Round
- Trunc
- Mod

Case Manipulation

- lower
- upper
- initcap.

Character functions

Character Manipulation.

- Concat
- Substr
- length
- Instr
- LPAD/RPAD
- Trim
- Replace.

Date function

- Months-between
- Add-Months
- NEXT-DAY
- LAST-DAY
- Round
- TRUNC

General functions

- NVL
- NVL2
- NULL IF
- COALESCE
- CASE
- DECODE

Data-Type Conversion.

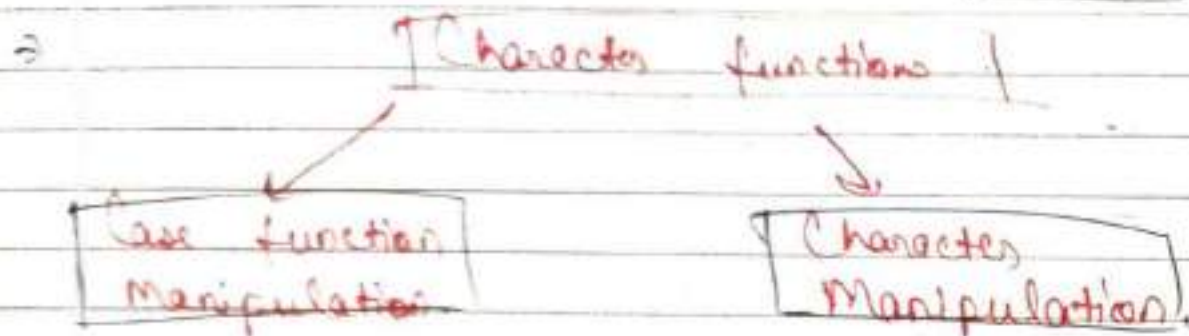
- TO-CHAR
- TO-NUMBER
- TO-DATE

Multi-Row funcⁿ

Aggregate.

- SUM
- MAX
- MIN
- COUNT
- AVG

Character functions in SQL with Execution! →



- lower
- upper
- Init Cap
- Concat ('Varun', 'Singh') Varun Singh
- Substr ('Varun', 2, 4) arun
- Instr ('Varun', 'u') 4th
- length ('Varun') 5
- LPAD ('Varun', 10, '*') *****Varun
- RPAD ' ' (10-length)
- TRIM ('V' from 'Varun') Arun
- REPLACE ('Varun', 'V', 'T') Tarun

ANURAG
1 2 3 4 5 6

Execution → If we want to implement these func, so, first we have to make schema (table).

Output Table! → select * from emp;

Id	NAME
1	Crash Smashers
2	Varun Singh



⇒ Create table emp
(
id int,
name varchar 2 (20)
);

Table Created.

2 Rows

insert into emp values (1, 'Gate Smashers');
insert into emp values (2, 'Varun Singh');

2 Rows

Select
1 by 1
Each
Row,
to
see
the
Output

Select * from emp;
Select lower (name) from emp;
Select upper (name) from emp;
Select initcap (name) from emp;
Select concat (id, name) from emp;
Select SUBSTR (name, 2, 5), INSTR (name, 'v') from emp;
Select length (name) from emp;
Select lpad (name, 15, 'x') from emp;
Select rpad (name, 15, 'x') from emp;
Select trim ('v' from name) from emp;
Select replace (name, 'v', 't') from emp;

Output:-

REPLACE (NAME, 'V', 'T')
Gate Smashers
Varun Singh

Output:-

Lpad (NAME, 15, 'x')
***GATE_SMASHERS
***Varun_Singh

Here, it counts (-) Space also.

Output:-

SUBSTR (NAME, 2, 5)	INSTR (NAME, 'v')
ate S	0
arun	1

(It don't count (-) here).

ie,

GATE	SMASHERS
1 2 3 4	5 6 7 8 9 10 11 12
ate S	

11/20 View in Database! →

(Oracle, SQL Server Views)

- What is view in Database?

→ Virtual Table! →

The Table we create,
Create Table Xyz

It takes physical space in memory (H.O.)

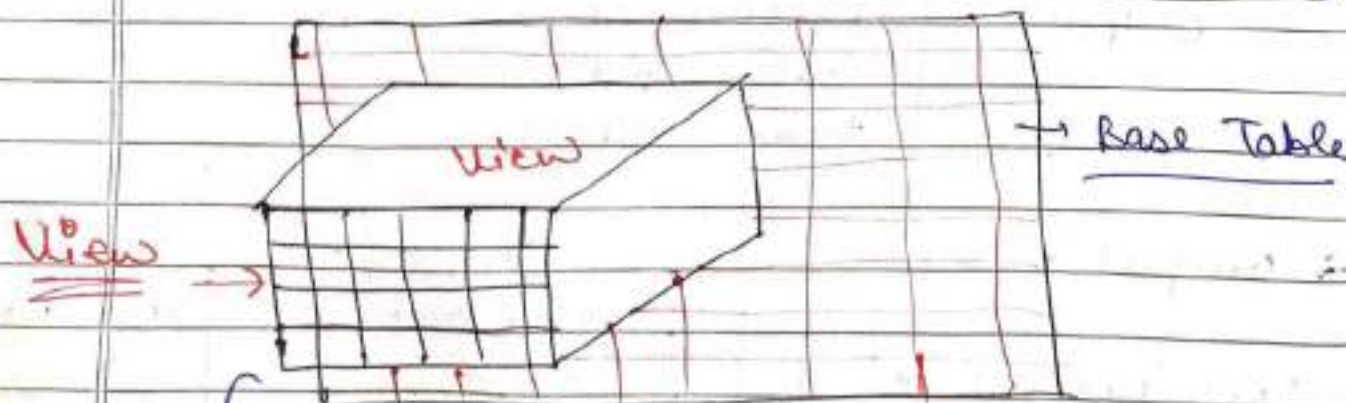
2) But, view is the virtual Table.

It looks like a Table, but it is not actually a table. It don't take space.
any.

→ View is the result set of a stored query.

[Query! →]

✱ Create view V, as select id from student;



This view has no physical existence.
We don't store it anywhere.

VI - View



Page No.

25

(#) The Execute code of this query, stores after compile. &

after compile, when we write
Select * from V1

then,

it shows the data like a table from the
view

(#) So, actually we just store this little query, rather than Result. We don't store Result.

That's why it's a virtual Table.

→ Read-only (vs) updatable Views! →

→ If we made any changes in Base Table & that col^m is also in our view, then, obviously that also changes.

→ Same, If we delete from Base Table. Then also deletes from View.

(#) → But, if we change anything in View! →

And we want that changes to execute also in the Base Table, then updatable Views. &

If we shut down the (Insert, Delete & update) DDL commands on view, i.e., we disable these commands, so, that it can't operate on View. Then, we made Read-only Views for it.

2. Materialized View :-

type of updated version.

(If our data is on the remote server, & I want a copy of that on my local server / machine. i.e., I want a snapshot of that remote data on my local server. So, that is called Materialised View.)

→ (This takes space, but takes less space as comparatively to that data).

① We can't apply any DDL Command (Alter, ex.) on view.

But, apply only DML Commands if we make the updatable view.

② We can insert the data of more than 1 table, in a View.

i.e.

(View can also take data from Multiple Tables.)

③ We also can take any particular row from Table to make view, like

where address = 'delhi';

So, all delhi students come into the view.

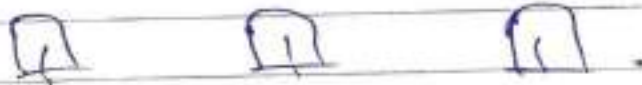
④ Advantages of View :-

1.) To restrict the Data Access.

(like, we don't give the access of original Table to our user, we just give view to them of some data.)

2.) To make complex queries easy.

Ex: 1) Like, we have data from 3 tables.



then,

By default, we apply Join or Nested Query.

So,

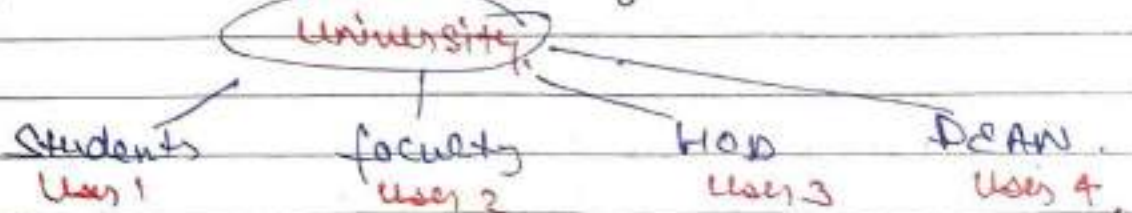
It's better to just make the view from these tables. Rather, than to write complex query.

3.) To provide data Independence,

(We just give a particular access to a user of a table, rather than giving the full database access).

4.) To present diff views of the same data.

Ex: 1)



(They all have diff privileges).

So,

→ We give diff views to all of them of the same data (table).

X X



SQL CHEAT SHEET :->

Examples ->

1.) Select all rows from table with filter applied

-> Select * From tbl where col1 > 5;

2.) Select first 10 rows for 2 columns
Select col1, col2 from tbl limit 10;

3.) Select all rows with multiple filters applied

-> Select * from tbl where col1 > 5 AND col2 < 2;

4.) Select all rows from col1 and col2
Ordering by col1

- Select col1, col2 from tbl Order By 1;

5.) Return count of rows in table

- Select Count (*) from tbl;

6.) Return sum of col1

- Select Sum (col1) from tbl;

7.) Return max value from col1

- Select MAX (col1) from tbl;

8.) Computes summary statistics by grouping col2

- Select AVG (col1) from tbl Group By col2;

9.) Combine data from 2 tables using a left Join

→ SELECT * FROM tbl1 AS t1
LEFT JOIN tbl2 AS t2 ON t2.col1 = t1.col1;

10.) Aggregate & filter Results

→ SELECT
col1,
AVG (col2) = AVG (col3) AS total
FROM tbl
GROUP BY col1
HAVING total > 2;

11.) Implementation of CASE statement →

→ SELECT col1,
CASE
WHEN col1 > 10 THEN 'more than 10',
WHEN col1 < 10 THEN 'less than 10',
ELSE '10'
END AS NewColumnName
FROM tbl;

*) ORDER OF EXECUTION →

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT



④. Create

```
CREATE DATABASE MyDatabase ;
```

```
CREATE INDEX IndexName  
ON TableName (col1);
```

```
CREATE TABLE OurTable (  
    id int,  
    name varchar (12)  
);
```

• UPDATE TABLE

```
UPDATE OurTable  
SET col1 = 56  
WHERE col2 = 'something';
```

• DELETE

```
DROP DATABASE OurDatabase ;
```

```
DROP TABLE OurTable ;
```

• DELETE Records

```
→ Delete from OurTable  
WHERE col1 = 'something';
```

#

Add / Remove Column

ALTER TABLE OurTable
ADD cols int;

ALTER TABLE OurTable
DROP column cols;

x

x